University of Milano-Bicocca

Department of Informatics, Systems and Communication

Master's degree in Data Science

Gossip learning for distributed training of performance models in Edge FaaS Systems

Supervisor: Prof. Michele Ciavotta
Co-supervisor: Dr.ssa Federica Filippini

Master's degree thesis by:

Adonis Kingsley Granita

student ID number 861280

Academic Year 2025-2026

Abstract

Decentralized Function-as-a-Service (DFaaS) systems allow heterogeneous edge nodes to cooperate in executing and redistributing serverless workloads. Supporting workload-offloading decisions requires models that estimate resource usage and overload risk for both the local node and potential destinations. However, models trained only on local observations may generalize poorly to other node types, while centralized training requires collecting data at a common location. This thesis investigates Gossip Learning for decentralized training of an adopted permutation-invariant multi-task model that jointly predicts node-level resource usage and overload status. Nodes retain their training records locally and exchange model parameters without a central training coordinator.

The study evaluates four existing merging strategies and introduces a node-type-aware approach that uses received models to generate synthetic targets for local workloads represented on remote node types. Different parameter-update rules and synthetic-data management configurations are compared through simulations of fully connected networks containing nine and eighteen nodes.

In the eighteen-node experiments, combining 70% of the local model parameters with 30% of the received parameters before each training update provides the best overall results among the evaluated configurations. Averaged across the eighteen nodes, a selected configuration achieves a common-test accuracy of 0.8950, an overloaded-class F1-score of 0.6147, and a mean squared error of 0.02346 on scaled targets, compared with an average accuracy of 0.5553 and an average MSE of 0.20081 for local-only training. As expected, centralized training achieves higher accuracy (0.9666) and lower MSE (0.00693). The decentralized approach nevertheless substantially improves upon local-only training while retaining data on individual nodes and avoiding a central training coordinator.

These findings show that the proposed Gossip Learning configuration improves prediction beyond local data distributions in the evaluated setting. The resulting models provide a predictive component that could support workload offloading; their integration into an operational offloading policy remains future work.

Contents

Abstract **3**

1 Introduction **11**
1.1 Background 13
1.1.1 Edge Computing 13
1.1.2 Serverless Computing and Function-as-a-Service 16
1.1.3 Edge Function-as-a-Service and Decentralized Function-as-a-Service 18
1.1.4 Workload Forecasting and Performance Prediction . . . 23
1.1.5 Deep Learning Foundations 24
1.1.6 Distributed learning 30
1.1.7 Statistical and System Heterogeneity 34

2 State of the Art **37**
2.1 Performance Prediction in Serverless and Edge FaaS Systems . . 37
2.1.1 Resource and Overload Prediction in Edge FaaS 38
2.1.2 Multi-Target and Permutation-Invariant Performance Prediction 41
2.1.3 Multi-Task Performance Prediction with Joint Regression and Classification 43
2.2 Distributed Training of Predictive Models 47
2.2.1 Federated Learning for Performance Prediction 47
2.2.2 Gossip Learning: From Distributed Classification to Edge Workload Forecasting 49

3 Problem statement **53**
3.1 Research Problem 53
3.2 Research Questions 55
3.3 Assumptions and Constraints 57
3.3.1 System and Communication Assumptions 57
3.3.2 Experimental Constraints 58
3.4 Contributions 59

4 Methodology . . . 61
4.1 Methodological Overview . . . 61
4.2 Network and Data Distribution . . . 62
4.2.1 Network . . . 62
4.2.2 Data Allocation to Nodes . . . 62
4.3 Gossip Learning Simulation Environment . . . 64
4.3.1 Model Initialization and Local Training . . . 64
4.3.2 Peer Communication and Asynchronous Execution . . . 65
4.4 Model-Merging Strategies . . . 66
4.4.1 Reference Strategies . . . 66
4.4.2 Proposed Strategies . . . 68
4.5 Experimental Design and Configuration . . . 72
4.5.1 Sequence of Experiments . . . 72
4.6 Evaluation . . . 73

5 Dataset . . . 75
5.1 Origin and Collection of the Datasets . . . 75
5.1.1 Source Domain . . . 75
5.1.2 Target Domain . . . 76
5.1.3 Input Variables and Prediction Targets . . . 76
5.1.4 Dataset Composition and Class Distribution . . . 77
5.2 Data Preprocessing . . . 80
5.2.1 Initial Cleaning and Domain Harmonization . . . 80
5.2.2 Local Training, Validation, and Test Splits . . . 80
5.2.3 Training-Set Overload Balancing . . . 81

6 Experimental Results . . . 83
6.1 Results with Existing Merging Strategies . . . 83
6.1.1 Number of Gossip Updates . . . 84
6.1.2 Task Loss Weights . . . 88
6.1.3 Number of Merged Models . . . 89
6.1.4 Effect of Longer Training and Motivation for Node-Type-Aware Learning . . . 91
6.2 Results with the Proposed Node-Type-Aware Strategy . . . 92
6.2.1 Results with Keep Local and Overwrite . . . 94
6.2.2 Results with Interpolation . . . 101
6.3 Summary of Experimental Findings . . . 107

7 Conclusion 113
7.1 Answers to the Research Questions 114
7.1.1 RQ1: Effectiveness and Generalization of Decentralized Training 114
7.1.2 RQ2: Influence of Model-Merging Strategies 114
7.1.3 RQ3: Evaluation of Gossip Learning Configurations . . . 115
7.2 Limitations 116
7.3 Future Work 117

List of Figures

1.1 Architecture of the Cloud Computing paradigm. Source: Shi et al. [1]. 14
1.2 Architecture of the Edge Computing paradigm. Source: Shi et al. [1]. 14
1.3 Serverless computing architecture. Source: [2]. 17
1.4 Reference scenario for decentralized FaaS in a federated edge environment. Source: Ciavotta et al. [3]. 20
1.5 High-level architecture of a DFaaS node. Source: Ciavotta et al. [3]. 21

2.1 Architecture of the Janossy-inspired multi-target neural model. Source: Filippini et al. [4]. 42
2.2 Janossy-based multi-task architecture for performance regression and node-overload classification. Source: Khan [5]. 46

4.1 Fully connected nine-node network used in the initial experiments. The geographic background visualizes the node positions; the edges represent possible direct model exchanges in the simulation. 63

5.1 Number of prepared observations by domain and node type. . . 78
5.2 Percentage distribution of non-overloaded and overloaded observations by domain and node type. The value above each bar is the percentage of overloaded observations. 79
5.3 Distribution of the number of active functions per observation in the Source and Target Domains. A function is considered active when its request rate is greater than zero. 79

6.1 Classification results for node 8 using Overwrite with 100 updates. Columns show local training, local validation, local test, and common-test results, respectively. Confusion matrices appear above the corresponding proportional diagrams. 87

6.2 Classification results for node 5 using Overwrite with 100 updates. Columns show local training, local validation, local test, and common-test results, respectively. Confusion matrices appear above the corresponding proportional diagrams. 87
6.3 Classification results for Node 4 using Keep Local with Aggregate. Columns show local training, validation, local test, and common-test results, respectively. 98
6.4 Percentage of overloaded observations in the datasets used to train Node 4 with Keep Local and Aggregate. Panels show the local training data, individual synthetic node types, the combined synthetic data, and the complete local-plus-synthetic dataset. . . 99
6.5 Classification results for Node 11 using Overwrite with Aggregate. Columns show local training, validation, local test, and common-test results, respectively. 100
6.6 Classification results for Node 13 using Overwrite with Aggregate. Columns show local training, validation, local test, and common-test results, respectively. 101
6.7 Regression predictions for Node 8 using Interpolate with Substitution. Rows show local training, validation, local test, and common-test results; columns show CPU usage, RAM usage, and RAM usage percentage. The dashed diagonal represents perfect agreement. 105
6.8 Percentage of overloaded observations in the local-plus-synthetic training dataset of Node 1 over gossip updates, using Interpolate with Substitution. 105
6.9 Percentage of overloaded observations in the local-plus-synthetic training dataset of Node 0 over gossip updates, using Interpolate with Substitution. 106
6.10 Accuracy of the local-only baseline models on the original training datasets of all eighteen nodes. Rows identify models and columns identify datasets. Outlined diagonal entries correspond to each model's own local training dataset. 109
6.11 Accuracy of the models obtained using Interpolate with Substitution and $\alpha = 0.3$ on the same original training datasets used in Figure 6.10. Rows identify models and columns identify datasets. 110

1 Introduction

Function-as-a-Service (FaaS) enables applications to execute as event-driven functions while delegating the management of their execution environments to the underlying platform. Extending this paradigm to edge infrastructures brings computation closer to users and data sources, but introduces challenges associated with limited resources, heterogeneous nodes, and uneven workload demand. In Decentralized Function-as-a-Service (DFaaS) systems, neighbouring nodes cooperate through peer-to-peer communication and can redistribute function requests to exploit available capacity across the infrastructure.

Effective workload redistribution requires more than knowledge of the resources currently available at the local node. Before offloading requests, a node should estimate how potential destinations would respond to the additional workload, including their expected resource usage and risk of overload. Since neighbouring nodes may have different computational characteristics, the same combination of functions and invocation rates can produce different outcomes depending on where it is executed. Predictive models must therefore capture relationships between workload composition, node characteristics, and performance across heterogeneous execution environments.

Performance prediction addresses this requirement by estimating the effects of a given workload on a computing node. It complements workload forecasting, which estimates future incoming demand, and can provide information to an offloading mechanism that selects where requests should be processed. This thesis focuses on the training of the performance-prediction component. It does not implement the workload-forecasting stage or evaluate an operational offloading policy.

Previous work has developed machine-learning models for predicting resource usage and overload in Edge FaaS systems. In particular, the multi-task architecture developed by Khan [5] combines continuous resource predictions and binary overload classification within a shared neural model. Its permutation-invariant representation allows a workload to be described through its active functions without making predictions depend on their arbitrary input order. This architec-

ture provides the predictive foundation adopted in this thesis, which considers node-level CPU usage, RAM usage, RAM usage percentage, and overload status.

Training such models becomes more challenging when observations are distributed among participating nodes. A model trained exclusively on one node's data may perform well under familiar conditions but remain insufficiently accurate for other node types. This limitation is particularly relevant to offloading, where a node needs predictions about potential destinations as well as itself. Centralized training provides access to the combined observations, but requires data collection at a common location. An alternative is to allow nodes to learn collaboratively while retaining their training records locally.

Gossip Learning provides a fully decentralized mechanism for this collaboration. Each node maintains its own model, exchanges parameters with selected peers, and performs local optimization using the information available to it. Knowledge propagates through repeated peer-to-peer interactions without a central aggregation server. The focus is on the organization of training and the incorporation of remote knowledge, using the adopted base multi-task architecture.

The main contributions of this thesis are:

- The adaptation and implementation of a Gossip Learning procedure for decentralized training of the adopted permutation-invariant multi-task performance model.
- An experimental comparison of reference merging strategies and selected training configurations for joint resource regression and overload classification.
- The design and evaluation of a new merging strategy that combines model-parameter updates with locally generated synthetic observations associated with remote node types.

Structure of the Thesis. The remainder of this chapter introduces the background on Edge FaaS, performance prediction, neural architectures, and distributed learning. Chapter 2 reviews related work on performance modeling and distributed training. Chapter 3 defines the research problem, research questions, assumptions, and contributions. Chapter 4 describes the Gossip Learning simulation, model-merging strategies, and evaluation methodology. Chapter 5 presents the datasets and their preprocessing. Chapter 6 reports the experimental results and analyses the behaviour of the evaluated configurations. Finally, Chapter 7 answers the research questions, discusses the limitations, and outlines future work.

1.1 Background

This section provides the conceptual and technological foundations required to understand the problem addressed in this thesis. It first introduces Edge Computing and explains how bringing computation closer to users and data sources can mitigate some of the latency, bandwidth, and service-continuity limitations of exclusively cloud-based infrastructures. It then presents Serverless Computing and discusses Function-as-a-Service (FaaS) as one of its main service models, before describing its extension to edge environments through Edge FaaS and Decentralized Function-as-a-Service (DFaaS). The latter constitutes the reference application scenario considered in this work.

The section subsequently distinguishes workload forecasting from performance prediction and explains their role in supporting proactive resource-management and offloading decisions. It then introduces the deep-learning concepts required to understand the adopted predictive model, including recurrent neural networks, multi-task learning, and permutation-invariant architectures. Finally, it presents the foundations of learning from distributed data by distinguishing centralized, federated, and fully decentralized coordination, as well as synchronous and asynchronous learning. Statistical and system heterogeneity are also discussed, as they constitute fundamental challenges when data distributions, workloads, computational resources, communication conditions, and node availability differ across the infrastructure. Together, these concepts establish the background for the decentralized gossip-based training of a multi-task performance prediction model investigated in this thesis.

1.1.1 Edge Computing

Cloud Computing is a widely adopted paradigm for delivering computing and storage resources through large, remotely managed data centers. According to Satyanarayanan [6], Cloud Computing provides two main benefits. On the one hand, concentrating computational resources in large data centers enables organizations to exploit economies of scale, thereby reducing system administration and operational costs. On the other hand, organizations can avoid the high initial investments (Capital expenditure, CAPEX) required to build and maintain their own data centers by relying on services provided by cloud providers.

In recent years, the widespread adoption of mobile devices and the Internet of Things (IoT) has substantially increased the volume of data generated by devices such as smartphones and sensors. Transmitting these data to remote data centers can increase end-to-end latency, generate additional network traffic,

and consume bandwidth, potentially affecting the Quality of Service of latency-sensitive applications. Moreover, processing all collected data in the cloud may be inefficient in IoT scenarios involving a large number of data-generating devices. These limitations motivate the execution of at least part of the processing closer to the corresponding data sources [1, 6].

To address these challenges, Edge Computing has emerged as a distributed computing paradigm in which computing and storage resources are deployed at the edge of the network, close to end devices. These resources may take different forms, but share the same objective: to bring computation closer to data sources by reducing the physical and logical distance between where the data are generated and the computational resources responsible for processing them [1, 6].

Edge Computing does not replace Cloud Computing but complements centralized cloud infrastructures. Shi et al. [1] define the edge as the collection of computational and networking resources located along the path between data sources and cloud data centers. In the traditional cloud architecture shown in Figure 1.1, data generated by end devices are transferred to remote data centers for storage and processing. By contrast, the Edge Computing architecture illustrated in Figure 1.2 introduces an intermediate edge layer, allowing part of the computation and storage to be performed closer to the data sources.

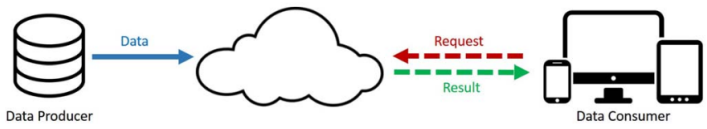


Figure 1.1. Architecture of the Cloud Computing paradigm. Source: Shi et al. [1].

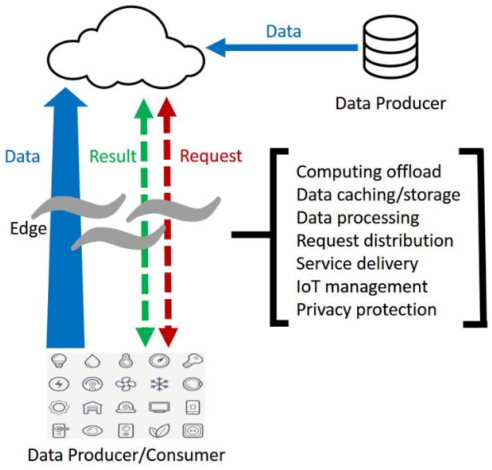


Figure 1.2. Architecture of the Edge Computing paradigm. Source: Shi et al. [1].

As illustrated in Figure 1.2, the edge layer can support several functionalities, including computation offloading, data processing, caching and storage, request distribution, service delivery, IoT management, and privacy protection. These capabilities can reduce latency and cloud-directed traffic, lower bandwidth consumption, and improve the scalability of the overall infrastructure [1].

Edge resources can also improve service continuity when access to the cloud is temporarily disrupted. For example, if a cloud service becomes unavailable due to network or cloud failure, a nearby edge resource can act as a temporary fallback and continue to provide critical functionality [6].

Beyond performance improvements, Edge Computing can support data privacy by allowing sensitive information to be filtered, anonymized, or aggregated close to its source before being transmitted to the cloud. However, its distributed and heterogeneous infrastructure also introduces additional security challenges, since a larger number of geographically dispersed nodes and communication links must be protected [1, 6].

Despite its advantages, Edge Computing introduces several design and management challenges. Unlike centralized cloud data centers, edge infrastructures are composed of geographically distributed nodes that may differ significantly in terms of processing capacity, memory, storage, network connectivity, and energy availability. Edge nodes are therefore typically heterogeneous and resource-constrained. Consequently, the same workload may produce different resource-consumption patterns and performance conditions depending on the node on which it is executed. Furthermore, the distributed nature of the infrastructure increases the complexity of service management, security enforcement, task allocation, and resource coordination [1, 7].

The management of edge resources is further complicated by the dynamic nature of the environment. Edge workloads may vary considerably over time as a consequence of user mobility, changes in request patterns, and the geographical distribution of users and applications. At the same time, edge nodes and communication links may become temporarily unavailable, causing changes in both the available computational capacity and the network topology. These characteristics make it difficult to maintain a complete and continuously updated view of the state of the system [1, 7].

This variability makes resource allocation and task offloading particularly challenging. Decisions based exclusively on the current resource state may be inadequate in the presence of rapidly changing workloads, since corrective actions may occur only after performance degradation or resource saturation has begun. Predictive information about future workload and resource utilization can instead support proactive resource-management decisions [7].

The geographical distribution and operational autonomy of edge nodes also motivate distributed resource-management mechanisms. Approaches relying on a continuously available central coordinator may be difficult to apply when nodes communicate through unreliable links or dynamically join and leave the infrastructure. Instead, decentralized solutions allow nodes to make local decisions and cooperate with neighboring peers, although they must operate without a complete and continuously updated global view [7].

The following sections introduce Serverless Computing and Function-as-a-Service, before discussing their deployment in decentralized edge environments.

1.1.2 Serverless Computing and Function-as-a-Service

Serverless Computing is a cloud computing model in which developers focus primarily on implementing the application logic, while the cloud provider manages application deployment, resource provisioning, monitoring, fault tolerance, logging, and scalability [8, 9].

The term *serverless* does not imply the absence of servers. Rather, it indicates that server provisioning and management are abstracted from application developers [8, 9].

Jonas et al. [9] identify automatic scaling without explicit resource provisioning and fine-grained, usage-based billing as defining characteristics of Serverless Computing. These characteristics allow applications to dynamically adapt resource consumption to workload demand while reducing operational overhead.

In the definition adopted by Jonas et al. [9], Serverless Computing combines Function-as-a-Service (FaaS) with Backend-as-a-Service (BaaS):

- **Function-as-a-Service (FaaS)** provides computational capabilities through the execution of event-driven functions.
- **Backend-as-a-Service (BaaS)** provides managed backend services that support the development and execution of serverless applications.

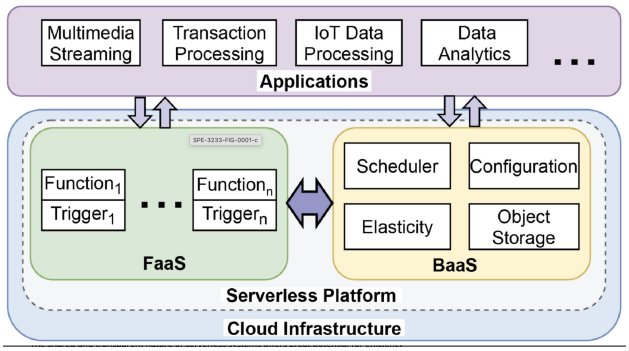


Figure 1.3. Serverless computing architecture. Source: [2].

Figure 1.3 illustrates the relationship between applications, serverless services, and the underlying cloud infrastructure. FaaS provides the computational component responsible for function execution, while BaaS provides specialized managed services, such as storage, configuration, and messaging [2, 9].

1.1.2.1 Function-as-a-Service

Function-as-a-Service (FaaS) is one of the main execution models of Serverless Computing. It enables developers to deploy application logic as small functions that are executed in response to specific events, while the underlying platform manages their deployment, resource provisioning, scaling, monitoring, and fault tolerance [8, 9].

So FaaS follows an event-driven execution model. Functions may be triggered by events such as HTTP requests, database updates, file uploads, message-queue events, or data produced by IoT devices. Instead of running continuously, a function is activated only when a corresponding event occurs [9].

Functions are generally designed to be stateless, meaning that they do not maintain persistent application state within their execution environment across successive invocations. When persistent state is required, it is typically stored in external services, such as databases or object-storage systems. This separation allows the platform to create, replicate, or remove function instances according to the current workload [8, 9].

Unlike traditional applications, which are commonly deployed as continuously running services, FaaS platforms allocate computational resources dynamically in response to function invocations. When the incoming request rate increases, the platform may create additional function instances; when demand decreases, unused instances can be released, potentially scaling the allocated resources down to zero. This execution model provides elasticity while reducing the infrastructure-management responsibilities of application developers [8, 9].

By abstracting resource provisioning and runtime management, FaaS simplifies the development of event-driven applications. However, the execution model originally emerged in resource-rich cloud environments, whereas its deployment at the edge must account for limited and heterogeneous computational resources.

1.1.3 Edge Function-as-a-Service and Decentralized Function-as-a-Service

1.1.3.1 Edge Function-as-a-Service

The Function-as-a-Service paradigm was initially developed for centralized and resource-rich cloud data centers. In this setting, FaaS providers can rely on large pools of computational resources to instantiate function execution environments, distribute invocation requests, and adapt the number of active instances to changes in workload demand. The increasing adoption of latency-sensitive and data-intensive applications has created a growing interest in deploying serverless functions outside traditional cloud data centers and closer to users and data sources [10].

This evolution can be framed within the Edge-Cloud compute continuum, which comprises computational, storage, and networking resources distributed across different infrastructure layers, ranging from end devices and edge nodes to centralized cloud data centers. These resources provide different and complementary properties. Cloud data centers offer high computational capacity, scalability, and reliability, whereas edge nodes provide greater proximity to users and locally generated data. Applications can therefore exploit different parts of the continuum according to their latency, resource, privacy, and communication requirements [10].

Deploying FaaS platforms at the network edge enables function invocations to be processed closer to the point at which events and data are generated. This is particularly relevant for latency-sensitive and data-intensive Internet-of-Things applications, where continuously transferring information to remote cloud data centers may increase response times and generate considerable network traffic.This approach allows at least part of the processing to be performed locally, reducing communication with the cloud and enabling applications to react more rapidly to incoming events [10, 11].

Extending FaaS towards the edge is not merely a matter of relocating an existing cloud platform. Cloud FaaS systems are generally designed for environments characterized by abundant computational resources, relatively homogeneous servers, and high-speed internal networks. Edge infrastructures are instead

composed of geographically distributed nodes that may differ significantly in processing capacity, memory availability, storage, hardware architecture, energy constraints, and network connectivity. Consequently, the resources and time required to execute the same function may vary depending on the node selected for its execution [10].

Moreover, the resources consumed by the FaaS platform reduce the capacity available for executing user functions. Edge FaaS platforms must therefore limit the overhead introduced by function isolation, invocation dispatching, monitoring, and resource management. They must also schedule invocations across nodes whose availability and connectivity may vary over time [10, 11].

In this context, *function placement* determines which nodes host the execution environment required by a function, whereas *invocation scheduling* determines where a specific function request should be executed. A related mechanism is *function offloading*, through which a node forwards an invocation request that it cannot or should not serve locally to another computational resource. Offloading can be horizontal, when requests are forwarded between nodes at the same infrastructure level, or vertical, when they are transferred between different levels of the continuum, for example from an edge node to the cloud [10].

An example of a FaaS platform designed for resource-constrained edge environments is *tinyFaaS*, proposed by Pfandzelter and Bermbach [11]. Its architecture targets single-node deployments and aims to limit the computational and memory overhead introduced by the platform, so that most of the available resources remain dedicated to user functions. Its design also considers requirements arising from IoT environments, including support for lightweight communication protocols, function isolation, and extensibility across different hardware and runtime environments [11].

Although tinyFaaS demonstrates how the FaaS model can be adapted to constrained edge nodes, it does not directly address coordinated workload distribution across multiple nodes. In particular, a single-node platform cannot autonomously exploit spare computational capacity available elsewhere when its local workload exceeds the available resources. Coordinating function execution across geographically distributed FaaS-enabled nodes therefore requires additional resource-discovery, scheduling, and offloading mechanisms [10, 11].

This limitation becomes particularly relevant because edge workloads are rarely distributed uniformly. User mobility, traffic fluctuations, and application-specific request patterns can cause some nodes to receive more invocations than they can process, while other nodes retain unused computational capacity. Processing all requests exclusively at the node where they initially arrive may consequently result in resource underutilization, service degradation, or request

rejection. Distributed and decentralized FaaS architectures seek to address this problem by coordinating function execution and workload distribution across multiple nodes [3, 10].

1.1.3.2 Decentralized Function-as-a-Service

Decentralized Function-as-a-Service (DFaaS) is an architecture designed to support cooperative serverless execution across federated edge nodes. In DFaaS, multiple FaaS-enabled nodes form a federation in which each participant remains autonomously managed while making part of its computational capacity available to other members. This organization allows nodes to cooperate without transferring control of their local resources to a single centralized orchestrator [3].

A federation may include nodes operated by the same infrastructure provider or by different providers that agree to share computational capacity. Each node receives function-execution requests from users connected to its corresponding access point. When sufficient local resources are available, requests can be processed by the local FaaS platform. When the node becomes overloaded or cannot accommodate additional invocations, part of the incoming workload can instead be transparently forwarded to another federation member with sufficient available capacity [3].

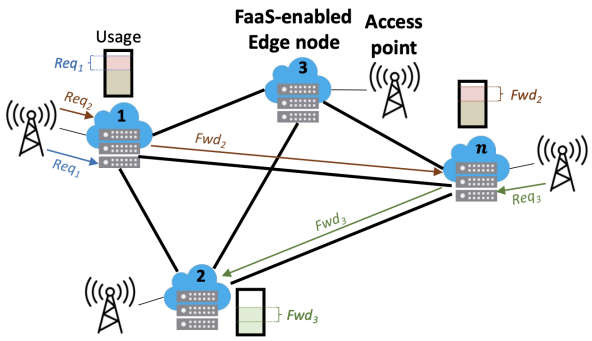


Figure 1.4. Reference scenario for decentralized FaaS in a federated edge environment. Source: Ciavotta et al. [3].

Figure 1.4 illustrates a DFaaS federation composed of geographically distributed edge nodes. Requests are initially directed to the node associated with the corresponding access point. For example, a request delivered to a node with sufficient capacity is executed locally. Subsequent requests that would exceed the local capacity can be forwarded to other nodes in the federation. This mechanism implements horizontal offloading because invocation requests are redistributed between peer nodes belonging to the edge layer [3, 10].

Horizontal offloading allows the system to exploit computational resources that would otherwise remain unused. It can also reduce the number of rejected requests and prevent an overloaded node from experiencing excessive resource saturation. At the same time, offloading is not free of cost: forwarding a request introduces additional network communication, and the receiving node must have sufficient capacity to execute both its local workload and the externally received invocations. An effective load-management mechanism must therefore determine when local execution remains preferable and when the benefits of offloading compensate for its communication and execution costs [3, 10].

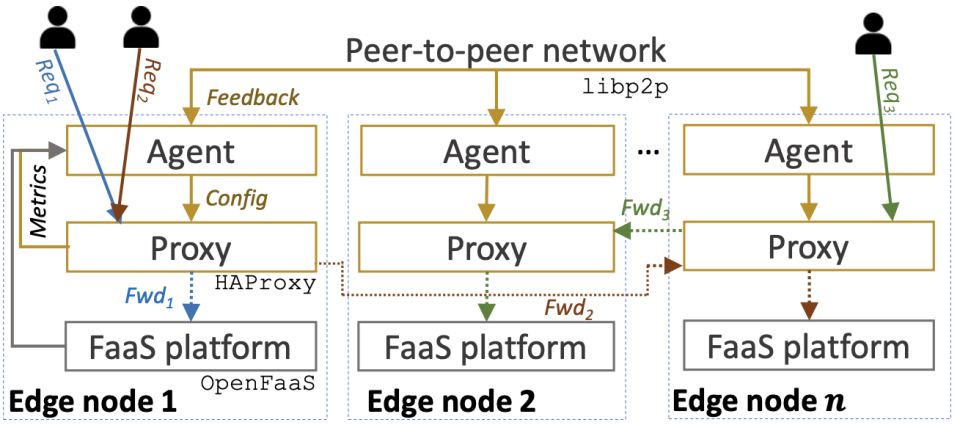


Figure 1.5. High-level architecture of a DFaaS node. Source: Ciavotta et al. [3].

The high-level architecture of a DFaaS node is shown in Figure 1.5. Each participating node contains three main components: a FaaS platform, a Proxy, and an Agent. The FaaS platform provides the runtime environment responsible for instantiating and executing serverless functions [3].

The Proxy acts as the entry point for function-execution requests. It receives incoming invocations and distributes them between the local FaaS platform and remote nodes in the federation. Requests forwarded by other nodes are processed through the same mechanism. The Proxy configuration therefore determines which fraction of the workload is executed locally and which fraction is redirected to neighbouring nodes [3].

The Agent implements the decentralized control logic of each DFaaS node and communicates with the Agents running on the other nodes through a peer-to-peer overlay network. Its responsibilities include discovering neighbouring peers, detecting changes in the federation, monitoring the state of the local FaaS platform, and collecting information about resource utilization and Quality-of-Service metrics. The information exchanged among Agents allows each node to

construct a partial view of the federation and to participate in resource-allocation decisions without relying on a centralized coordinator [3].

In the architecture proposed by Ciavotta et al. [3], the Agent is also designed to estimate the workload expected in the following time interval and the corresponding resource demand. On the basis of this information, it can determine whether the local node has sufficient capacity, negotiate computational resources with other members of the federation, and configure the Proxy to distribute incoming invocations accordingly. Workload prediction is therefore not an independent functionality, but a component of the decentralized resource-management process: inaccurate estimates may cause unnecessary offloading when demand is overestimated or insufficient resource allocation when it is underestimated.

Unlike centralized architectures, DFaaS does not rely on a single component that maintains the complete state of the federation and determines how all requests should be distributed. Instead, each node makes autonomous decisions based on local measurements and information received from its peers. This organization is consistent with the geographically distributed nature of edge infrastructures and avoids introducing a single centralized coordination point. Decentralized architectures can therefore reduce the risk of communication bottlenecks and single points of failure, although they must operate using information that may be incomplete, delayed, or limited to a subset of the network [3, 10].

The quality of the local information available to each Agent is consequently fundamental. Underestimating future demand may cause a node to accept more requests than it can process, leading to overload, increased response time, or request rejection. Conversely, overestimating resource demand may cause unnecessary offloading, increasing communication latency and leaving local computational capacity underutilized. Effective DFaaS operation therefore benefits from predictive models capable of estimating how incoming function workloads will affect the future resource utilization and operational state of heterogeneous edge nodes [7].

The DFaaS architecture provides the reference system context for the work presented in this thesis. In particular, this study investigates the decentralized training of multi-task models designed to predict node-level resource utilization and overload conditions. These predictions can support DFaaS Agents in making proactive resource-management and offloading decisions. Gossip Learning is adopted to allow the predictive models maintained by different nodes to incorporate knowledge originating from distributed local datasets without relying on a centralized training coordinator.

1.1.4 Workload Forecasting and Performance Prediction

Effective resource management in distributed computing environments can benefit from information about the conditions that are likely to occur in the near future. As mentioned in the previous sections, this is particularly relevant in edge environments, where computational resources are limited and incoming demand may vary considerably over time. Predictive information can support proactive resource-management decisions before resource saturation or service degradation occurs [4, 7].

Although *workload forecasting* and *performance prediction* are closely related, they address two distinct problems. Workload forecasting aims to estimate future demand using the information available when the forecast is produced. In a time-series setting, future values are commonly estimated from previous observations of the variable of interest, possibly together with additional explanatory variables [12]. In a FaaS environment, workload can be represented through quantities such as the number or rate of function invocations observed during consecutive time intervals [7].

Let w_t denote the workload observed during time interval t. A workload-forecasting model can be represented at a high level as

$$\hat{w}_{t+h} = f\left(w_t, w_{t-1}, \ldots, w_{t-k+1}, \mathbf{z}_t\right), \tag{1.1}$$

where $\hat{w}_{t+h}$ is the estimated workload for a future time interval, h is the forecasting horizon, k determines the number of previous observations considered by the model, and $\mathbf{z}_t$ represents any additional contextual information [12].

Performance prediction instead estimates how a computing system will behave when processing a given workload. A performance-prediction model relates workload characteristics and system configuration to indicators describing the resulting operational state. In FaaS-enabled edge environments, relevant outputs may include CPU utilization, memory consumption, energy consumption, execution latency, and overload conditions [4].

Let $\mathbf{x}$ describe the workload and let $\mathbf{s}$ contain information about the system configuration. A generic performance-prediction model can be represented as

$$\hat{\mathbf{y}} = g\left(\mathbf{x}, \mathbf{s}\right), \tag{1.2}$$

where $\hat{\mathbf{y}}$ contains the predicted performance or resource-utilization indicators. Workload forecasting therefore estimates *what demand is expected to arrive*, whereas performance prediction estimates *what effect a given workload is expected to have on the system* [4, 7].

The two models may be organized as consecutive stages of a proactive resource-management pipeline. The first stage estimates the workload expected during a future interval, while the second estimates the effect of that workload on the computing node:

$$\{w_t, w_{t-1}, \ldots\} \xrightarrow{\text{workload forecasting}} \hat{\mathbf{x}}_{t+h} \xrightarrow{\text{performance prediction}} \hat{\mathbf{y}}_{t+h}. \tag{1.3}$$

In a DFaaS environment, the resulting information can support decisions concerning local execution, resource allocation, and workload offloading [3, 4].

Performance prediction can involve different types of learning tasks. Resource-utilization indicators, such as CPU and memory consumption, are continuous quantities and can therefore be estimated through regression. Overload detection can instead be formulated as a classification problem in which the model determines whether a given workload is expected to produce a normal or overloaded state. Since these outputs describe related effects of the same workload, they may be learned jointly using a multi-task architecture. The foundations of this learning paradigm are introduced in the following section.

1.1.5 Deep Learning Foundations

Deep learning models are artificial neural networks composed of multiple processing layers. Each layer transforms its input into a new representation through a set of learnable parameters and a nonlinear activation function. By composing several transformations, a neural network can approximate complex relationships between input variables and prediction targets [13].

A feedforward neural network defines a mapping from an input vector $\mathbf{x}$ to an output $\hat{\mathbf{y}}$. Information flows from the input layer through one or more hidden layers until it reaches the output layer, without recurrent connections. A generic hidden layer can be expressed as

$$\mathbf{h}^{(l)} = \sigma\left(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}\right), \tag{1.4}$$

where $\mathbf{h}^{(l-1)}$ is the input to layer l, $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ are its learnable weights and biases, and σ is an activation function. The intermediate vectors produced by the hidden layers constitute learned representations of the original input. The output layer subsequently maps the final representation to the prediction required by the learning task [13, Chapter 6].

During supervised training, the parameters are adjusted to reduce a loss function that measures the discrepancy between the predicted outputs and the corresponding targets. Backpropagation applies the chain rule to compute the

derivatives of the loss with respect to the parameters of the different layers. A gradient-based optimization algorithm can then use these derivatives to progressively update the model parameters [13].

The architecture of a neural model depends on the structure of its inputs and outputs. Recurrent networks introduce mechanisms for processing sequential or variable-length inputs. Multi-task architectures allow related prediction objectives to share internal representations, while permutation-invariant architectures are appropriate when the input represents a set whose ordering has no semantic meaning. These three concepts are introduced below because they provide the foundations required to understand the predictive models considered in this thesis.

1.1.5.1 Recurrent Neural Networks and Gated Units

Recurrent Neural Networks (RNNs) are neural architectures designed to process sequential data. Unlike feedforward networks, which process an input without retaining an internal state between sequence positions, an RNN maintains a hidden state that is updated after each element and carries information derived from the elements processed previously. The same model parameters are shared across all sequence positions, meaning that the network repeatedly applies the same transformation while processing the sequence. This parameter sharing enables RNNs to operate on sequences of different lengths, although their output remains sensitive to the order of the input elements [13].

Given an input sequence $\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_T$, a conventional recurrent update can be represented as

$$\mathbf{h}_t = \phi\left(\mathbf{W}_x \mathbf{x}_t + \mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{b}\right), \tag{1.5}$$

where $\mathbf{x}_t$ is the input at position t, $\mathbf{h}_{t-1}$ is the previous hidden state, $\mathbf{h}_t$ is the updated state, and ϕ is a nonlinear activation function. Through this recurrence, the state at a given position can contain information derived from previously processed elements. Depending on the task, the network may produce an output at every sequence position or use the final state as a fixed-dimensional representation of the complete sequence [13].

The repeated application of the recurrent transformation creates a computational path extending across the complete sequence. During training, gradients must be propagated backwards through this path. For long sequences, they may progressively decrease towards zero or grow to excessively large values. These phenomena are known respectively as the *vanishing-gradient* and *exploding-gradient* problems. In particular, vanishing gradients make it difficult

for a conventional RNN to learn relationships between elements separated by long intervals [13, 14].

Long Short-Term Memory (LSTM) networks were introduced to address the difficulty of learning long-term dependencies with conventional recurrent architectures [14]. An LSTM unit maintains an internal cell state whose evolution is regulated by gating mechanisms. In the commonly used formulation, an input gate controls the contribution of newly computed information, a forget gate controls the amount of previous state that is retained, and an output gate controls how the internal state contributes to the hidden representation. Since the values of these gates depend on the current input and the previous hidden state, the network can learn when information should be stored, retained, or exposed [13].

A related architecture is the Gated Recurrent Unit (GRU), introduced by Cho et al. [15]. Like the LSTM, a GRU uses gates to regulate the propagation of information, but it adopts a more compact internal structure. In particular, it maintains a single hidden state and employs two main gates: an update gate and a reset gate. The update gate controls the balance between the previous state and a newly computed candidate representation, whereas the reset gate determines how much of the previous state contributes to the computation of that candidate [13, 15].

Using $\mathbf{z}_t$ for the update gate and $\mathbf{r}_t$ for the reset gate, the GRU can be expressed as

$$\mathbf{r}_t = \sigma\left(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1}\right), \tag{1.6}$$

$$\mathbf{z}_t = \sigma\left(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1}\right), \tag{1.7}$$

$$\widetilde{\mathbf{h}}_t = \phi\left(\mathbf{W}\mathbf{x}_t + \mathbf{U}\left(\mathbf{r}_t \odot \mathbf{h}_{t-1}\right)\right), \tag{1.8}$$

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + \left(1 - \mathbf{z}_t\right) \odot \widetilde{\mathbf{h}}_t, \tag{1.9}$$

where σ is the logistic sigmoid function and $\odot$ denotes element-wise multiplication. Although notational conventions may differ between implementations, the central principle remains the same: the update gate determines how much of the previous hidden state is retained in the current hidden state, whereas the reset gate determines how much information from the previous hidden state is considered when computing the candidate state [13, 15].

LSTM and GRU units therefore represent two gated alternatives to conventional recurrent units. Both provide mechanisms that allow the model to control the flow of information across sequence positions. However, an LSTM maintains separate cell and hidden states and uses a larger set of gates, whereas a GRU combines memory and hidden representation into a single state. These differences do

not imply that one architecture is universally superior: their suitability depends on the learning problem, data, and model configuration [13].

1.1.5.2 Multi-Task Learning

Traditional single-task learning trains a model to optimize one prediction objective. When several related objectives must be learned from the same or closely connected inputs, they may instead be addressed jointly through Multi-Task Learning (MTL). MTL is a form of inductive transfer in which knowledge learned from one task can support the learning of other related tasks. This occurs because the training signals of the different tasks jointly influence the shared representations learned by the model. The objective is to exploit common information across tasks to improve learning and generalization [16].

Let $\mathbf{x}$ denote an input associated with K prediction tasks. A multi-task neural architecture can first generate a shared representation

$$\mathbf{z} = f_{\boldsymbol{\theta}}(\mathbf{x}), \tag{1.10}$$

where $f_{\boldsymbol{\theta}}$ is parameterized by $\boldsymbol{\theta}$. The prediction for task k can then be produced by a task-specific function:

$$\hat{\mathbf{y}}_k = g_{\boldsymbol{\phi}_k}(\mathbf{z}), \qquad k = 1, \ldots, K, \tag{1.11}$$

where $\boldsymbol{\phi}_k$ contains parameters used exclusively by the corresponding task. The shared component is commonly referred to as the *shared backbone*, while the task-specific components are usually called *task-specific heads*.

A common implementation of MTL is *hard parameter sharing*. In this configuration, the hidden layers are shared by all tasks, whereas each task retains a separate output layer or head. Consequently, every training objective contributes to updating the common parameters. Parameter sharing can act as a form of regularization because the shared representation must support more than one prediction objective rather than becoming specialized exclusively for a single task [16, 17].

An alternative is *soft parameter sharing*. In this case, each task has its own model parameters, but the learning objective includes constraints or regularization terms that encourage the task-specific representations to remain similar. Hard sharing therefore imposes a common representation directly, whereas soft sharing preserves separate representations and encourages their parameters to be related [17].

The complete training objective of a multi-task model can generally be written as a weighted combination of task-specific losses:

$$\mathcal{L} = \sum_{k=1}^{K} \lambda_k \mathcal{L}_k, \tag{1.12}$$

where $\mathcal{L}_k$ is the loss associated with task k, and λ_k determines its contribution to the overall objective. This formulation allows heterogeneous prediction objectives to be optimized jointly. For example, continuous quantities can be learned through regression losses, while categorical outputs can be learned through classification losses.

The effectiveness of MTL depends on the relationships among the tasks. Related tasks may provide complementary training signals and promote representations that capture information useful to more than one objective. However, sharing information between unrelated or conflicting tasks can reduce performance, a phenomenon commonly referred to as *negative transfer*. Consequently, the selection of jointly learned tasks, the architecture of the shared representation, and the relative weighting of the losses are relevant design decisions [16, 17].

In performance prediction, multiple indicators may describe different effects of the same input workload. A multi-task architecture can therefore use a common representation of the workload while assigning separate output components to different regression and classification objectives. This makes it possible to capture common workload–resource relationships without requiring every output to be modeled through an entirely independent neural network.

1.1.5.3 Permutation-Invariant Models

Many neural architectures process inputs represented as vectors or sequences, for which the position of each element may carry information. Other learning problems are instead defined over sets. A set is an unordered collection: changing the order used to present its elements does not change the object represented by the input. A model operating on such data should therefore avoid producing different predictions solely because the same elements are supplied in a different order [18].

Let $X = \{x_1, \ldots, x_n\}$ be a set and let π be any permutation of its elements. A function f is permutation-invariant if

$$f(x_1, \ldots, x_n) = f(x_{\pi(1)}, \ldots, x_{\pi(n)}) \tag{1.13}$$

for every possible permutation π. Invariance must be distinguished from *permutation equivariance*. In the equivariant case, permuting the input produces a corresponding permutation of the output rather than leaving the output unchanged [18].

One way to construct an invariant model is to aggregate element-level representations using a symmetric operation such as summation, averaging, or maximum. These operations produce the same aggregate independently of the order in which their operands are presented. The Deep Sets architecture studied by Zaheer et al. [18] applies a shared transformation ϕ to every set element, combines the resulting representations through summation, and processes the aggregate through a second transformation ρ:

$$f(X) = \rho\left(\sum_{x \in X} \phi(x)\right). \tag{1.14}$$

Because both the transformation ϕ and its parameters are shared acrossthe elements, and because summation is insensitive to their order, the resulting function is permutation-invariant. This structure can also operate on sets with different cardinalities, since the aggregation always produces a fixed-dimensional representation [18].

Simple symmetric aggregation is computationally efficient, but it may constrain the way in which interactions among elements are represented. Janossy Pooling provides a more general construction by applying a permutation-sensitive function to different orderings of the input and then averaging the resulting representations [19]. For a set containing n elements, an exact Janossy representation can be written as

$$f(X) = \frac{1}{n!} \sum_{\pi \in \Pi_n} g\left(x_{\pi(1)}, \ldots, x_{\pi(n)}\right), \tag{1.15}$$

where Π_n denotes the collection of all $n!$ permutations and g is a function that may be sensitive to input order. Although the value produced by g may differ among permutations, averaging over the complete permutation set makes the final result invariant [19].

This construction allows permutation-sensitive models, including recurrent networks, to be used as internal encoders. Each ordering can be processed by the same recurrent model, producing a permutation-specific representation. Pooling these representations removes the dependence of the final output on the particular ordering initially assigned to the elements [19].

Exact enumeration requires $n!$ evaluations and therefore becomes computationally prohibitive as the number of elements increases. Janossy Pooling can consequently be approximated through mechanisms such as canonical orderings, restrictions on the maximum interaction order, or stochastic sampling of permutations. These alternatives reduce computational cost but introduce assumptions or approximations relative to averaging over every possible ordering [19].

Permutation invariance is relevant when a workload is represented as a collection of active functions rather than as a sequence with an intrinsic order. Reordering the descriptions of the same functions should not, by itself, alter the predicted resource utilization or overload condition. An invariant architecture makes this property explicit instead of requiring the model to learn an arbitrary dependence on the positions assigned to the functions.

1.1.6 Distributed learning

Machine-learning training is traditionally described as an optimization process performed over a dataset available within a single computational environment. However, in many real-world systems, data are naturally generated and stored across multiple devices, servers, or geographically distributed nodes. Transferring all observations to a central location may be undesirable or impractical because of communication costs, limited connectivity, privacy requirements, or the scale of the data involved. Distributed learning addresses this setting by allowing multiple computing entities to contribute to the training process while retaining at least part of the computation, and potentially the training data themselves, at their original locations [20, 21].

Let $\mathcal{D}_k$ denote the local dataset stored by participant k, containing n_k observations, and let $n = \sum_{k=1}^{K} n_k$ be the total number of observations distributed among K participants. A common formulation expresses the global learning objective as

$$\min_{\mathbf{w}} F(\mathbf{w}), \qquad F(\mathbf{w}) = \sum_{k=1}^{K} \frac{n_k}{n} F_k(\mathbf{w}), \tag{1.16}$$

where $\mathbf{w}$ denotes the model parameters and F_k is the objective computed from the data available at participant k. In other words the global objective $F(\mathbf{w})$ is computed as a weighted average of the local objectives $F_k(\mathbf{w})$ of all participants, where the contribution of each participant is weighted according to the number of observations in its local dataset. The goal is to find the model parameters $\mathbf{w}$ that minimize this global objective. This formulation does not prescribe a particular system architecture: the local contributions may be collected by a central component, combined hierarchically, or exchanged directly among peers. Consequently, distributed learning systems can be distinguished according to where the model state is maintained, how participants communicate, and how their locally computed contributions are combined [20–22].

Distributing the learning process introduces dependencies between computation and communication. Each participant operates on only a portion of the

available data, and the information derived from these local observations must therefore be propagated through the system. The resulting protocol determines whether training produces a single explicitly maintained global model or a collection of local models that progressively incorporate information from other participants. It also affects communication overhead, sensitivity to slow or unavailable nodes, and the consistency of the model states observed across the system [20, 22, 23].

1.1.6.1 Centralized, Federated, and Decentralized Learning

In centralized learning, training data are collected in a central computing environment, where both model optimization and model-state management are performed. This organization simplifies access to the complete dataset and avoids the need to reconcile independently trained models. Nevertheless, it requires the transfer and storage of raw observations at the central site. Such a requirement may be problematic when data are large, generated continuously, sensitive, or produced by devices connected through constrained or intermittent networks [20, 21].

Distributed computation does not necessarily eliminate centralized coordination. For example, the parameter-server architecture separates the system into workers and one or more server components. Workers process their assigned data partitions and compute parameter updates, whereas the parameter servers maintain and update the shared model state. The computations are therefore distributed, but communication and model management still depend on a logically centralized service. Parameter servers can be partitioned or replicated for scalability and reliability, without changing their coordinating role in the training process [23].

In the standard server-coordinated Federated Learning architecture, computation is distributed, while coordination remains centralized. Training data remain on participating clients, while a central server orchestrates a sequence of communication rounds. During a round, the server distributes the current model to a subset of clients; each selected client performs local optimization using its own dataset and returns the resulting update or model parameters. The server then aggregates these contributions to obtain the model used in the following round. Thus, although the data and local computations are distributed, the standard Federated Learning architecture retains a central aggregation point [21].

This distinction is important because the expression *decentralized data*, used to indicate that observations remain on clients, does not necessarily imply a fully decentralized learning protocol. In the Federated Averaging algorithm, for example, local training is performed by the clients, but model dissemination,

client participation, and aggregation are coordinated by a server. The server therefore remains part of both the communication structure and the training procedure [21].

In fully decentralized learning, instead, there is no central server that maintains and manages what is considered the authoritative or global version of the model. Participants are connected through a communication graph and exchange information directly with selected neighbours. Each node updates its own model using local observations and information received from other nodes. So knowledge is propagated through repeated local interactions rather than through a sequence of broadcasts and collections performed by a central coordinator [22, 24].

Decentralized stochastic-gradient methods commonly combine a local optimization step with a consensus or mixing step. The local step incorporates information from the node's dataset, while the mixing step brings its parameters closer to those maintained by neighbouring nodes. Repeating these operations allows information to spread across the communication graph even when no node has direct access to all datasets or models. The convergence and efficiency of this process depend in part on the topology and connectivity of the underlying graph [22].

Gossip Learning represents a fully distributed realization of this principle. In the approach proposed by Ormándi et al. [24], multiple models are propagated across a peer-to-peer network through direct exchanges between participating nodes. When a model reaches a node, it is updated using locally available data and can be combined with other models. Raw observations remain at their original nodes, whereas the models act as a mechanism through which the learned information is disseminated. This removes the need for an always available central coordinator and makes communication a sequence of local peer-to-peer interactions [24].

1.1.6.2 Synchronous and Asynchronous Coordination

Another relevant distinction concerns the temporal coordination of computation and communication. In synchronous learning, training proceeds through coordinated iterations or communication rounds that require participants to synchronize before advancing to the next model state. Participants compute their contributions from a defined model state, after which the system waits for the required results before producing the next shared state. This organization makes the sequence of model versions explicit and limits differences between the parameters used by different workers during the same iteration [20, 23].

The synchronization barrier can nevertheless reduce system efficiency when participants have heterogeneous computation or communication capabilities. The completion time of an iteration may be determined by its slowest required participant, commonly referred to as a straggler. This issue is especially relevant when training involves devices with different hardware resources or unstable network connections [20, 21].

In asynchronous learning, participants are not required to complete each computation step at the same time. Updates may be generated and incorporated as workers become available, allowing faster participants to continue without waiting for a global synchronization barrier. Parameter-server systems can support this behaviour through consistency policies that regulate how old a worker's local parameter state may be relative to the state maintained by the server [23].

Asynchronous execution introduces a different difficulty: an update can be computed from parameters that have already been modified by other participants. Such an update is usually described as *stale*. The absence of a global barrier can improve resource utilization and reduce waiting time, but the learning procedure must tolerate differences among the model versions used concurrently throughout the system. Synchronism and consistency are therefore related but distinct properties: removing a synchronization barrier does not remove the need to define when and under which model state an update is accepted [20, 23].

Gossip-based protocols can support decentralized asynchronous coordination. In such implementations, nodes perform local operations and exchange models with peers without requiring every participant to enter the same communication round. In the original Gossip Learning formulation, models follow independent random walks and are updated whenever they visit a node. The resulting progress is driven by local message exchanges rather than by a globally synchronized sequence of iterations [24].

This execution model is relevant to geographically distributed edge infrastructures, where nodes may differ in computational resources, workload characteristics, and availability.

1.1.7 Statistical and System Heterogeneity

Learning from distributed data requires accounting for differences among the participating nodes. These differences can concern both the data used for training and the characteristics of the underlying infrastructure. Accordingly, the literature commonly distinguishes between *statistical heterogeneity*, which arises when local datasets follow different distributions, and *system heterogeneity*, which concerns variations in the computational and communication capabilities of the participating devices [25].

Although these two forms of heterogeneity describe different properties of a distributed learning system, their effects are not necessarily independent. System constraints determine which nodes can participate, how much local training they can complete, and when their updates become available. Consequently, they can also alter the effective composition of the data represented during training. For example, systematically excluding slower participants can reduce the contribution of the data distributions associated with those nodes [25].

1.1.7.1 Statistical Heterogeneity

Many traditional distributed optimization methods assume that training examples are independently and identically distributed across workers. Under this assumption, the local dataset assigned to each participant can be regarded as a sample drawn from the same underlying population. Therefore, a local objective is expected to approximate the global objective constructed from all available observations [21].

This assumption is frequently violated when data are generated locally by different users, devices, or services. Let $\mathcal{D}_k$ denote the dataset stored at node k, and let $F_k(\mathbf{w})$ be the corresponding local objective. In a statistically heterogeneous setting, the datasets $\mathcal{D}_k$ may be generated according to different distributions. As a result, the local objectives F_k don't represent the same learning problem equally well, and a local objective may provide a poor approximation of the objective associated with the complete population [21, 25].

A distinction must also be made between non-IID and unbalanced data. The former expression concerns differences in the statistical distributions represented at the nodes, whereas the latter indicates that participants possess different numbers of training observations. A system can therefore contain balanced but non-IID datasets, unbalanced but similarly distributed datasets, or exhibit both properties simultaneously [21].

In practical terms, statistical heterogeneity may concern the examples, features, labels, or patterns observed by individual participants. Since each node

optimizes its model using its own dataset, local training can move different models in different directions. This effect becomes particularly relevant when several local optimization steps are performed before model aggregation or exchange. In their study of federated optimization in heterogeneous networks, Li et al. [25] discuss how performing many local training epochs can cause local models to move too far from their shared starting point, potentially leading to unstable convergence when data are non-identically distributed across devices.

The degree of statistical heterogeneity is not determined only by the existence of physically separate datasets. Instead, it depends also on how the local data distributions differ from one another and from the overall population. For example, McMahan et al. [21] distinguish an IID partition, obtained by randomly distributing examples among clients, from a pathological non-IID partition in which clients receive examples associated with only a limited subset of labels. This illustrates how the same original dataset can lead to different distributed learning problems depending on the partitioning mechanism.

1.1.7.2 System Heterogeneity

System heterogeneity arises because participating devices do not necessarily provide equivalent computing, storage, energy, or networking capabilities. Differences in processors and memory affect the amount and speed of local computation, while variations in connectivity influence the time and reliability of model communication. Battery constraints can further limit the availability or participation of individual devices [25].

These differences can cause participants to complete different amounts of local work within the same time interval. In a synchronously coordinated system, a slow participant can delay the completion of a training round and is commonly described as a *straggler*. A possible response is to impose a deadline and discard updates from nodes that do not complete the required work. However, this reduces the set of nodes that effectively contribute to training and can introduce bias when the excluded nodes are associated with particular data characteristics [25].

System heterogeneity also includes differences in node availability and communication conditions. McMahan et al. [21] describe distributed clients as potentially offline or connected through slow or costly communication links. In addition, node availability may be related to the characteristics of locally generated data. If slower or intermittently connected nodes participate less frequently in training, their local data distributions will be underrepresented in the resulting model. Consequently, node availability is not merely a technical property: it may also

affect the statistical composition of the data that effectively contribute to the learning process.

These issues are especially relevant at the network edge. Edge nodes are geographically distributed, often have limited and heterogeneous capacities, and may enter or leave the network because of unpredictable node or link failures. Furthermore, edge workloads are volatile because they depend on factors including user mobility, infrastructure location, and the application being executed. The considered infrastructure may therefore vary both in the resources offered by its nodes and in the workload patterns observed at their locations [7].

2

State of the Art

This chapter reviews the literature relevant to performance prediction and distributed model training in serverless and edge environments. The discussion follows two complementary research directions. The first examines approaches for predicting resource utilization and overload in Edge FaaS systems, including multi-target, permutation-invariant, and multi-task models. Particular attention is given to the predictive architecture adopted in this thesis. The second examines collaborative training approaches, considering Federated Learning for performance prediction and Gossip Learning for distributed classification and edge workload forecasting.

The selected studies are analyzed in terms of their objectives, methods, experimental findings, and limitations. This comparison provides the basis for defining the specific problem addressed in the following chapter.

2.1 Performance Prediction in Serverless and Edge FaaS Systems

Performance prediction can support the management of serverless infrastructures by providing information about the expected behaviour of applications and computing resources. The performance of function invocations may depend on several factors, including the characteristics of the incoming workload, the resources available on the execution node, the underlying hardware, and the concurrent execution of other functions. These factors can affect application performance, resource consumption, and the resource-management decisions taken by the platform [26, 27].

Data-driven approaches address this complexity by using measurements collected under different workloads and system configurations to learn relationships between execution conditions and performance indicators. The studies reviewed in this section address prediction at different levels of serverless infrastructures,

ranging from the resource consumption of function instances to node-level resource utilization and overload conditions in Edge FaaS environments. The discussion also examines models that address variable workload compositions, multiple continuous prediction targets, and joint regression and classification [4, 5, 26, 27].

2.1.1 Resource and Overload Prediction in Edge FaaS

As discussed in the previous chapter, the deployment of FaaS platforms at the network edge introduces performance-modeling requirements that differ from those encountered in resource-rich cloud data centers. Edge nodes generally provide limited computational capacity and must process workloads whose intensity and composition can change over time. In this context, estimating the resources required by the expected workload is important for deciding whether incoming requests can be processed locally or whether part of the workload should be redirected to other nodes. For this purpose, Filippini et al. [27] propose a machine-learning-based framework for predicting resource consumption and overload conditions in FaaS-enabled edge environments.

The proposed framework occupies a specific position within a proactive load-management pipeline. A workload-forecasting component first predicts the number of function requests that a node is expected to receive. The performance-prediction component then estimates the resources required to process that workload and determines the expected operational state of the node. Finally, a load-management component can use these predictions to decide which requests should be processed locally, redirected to another node, or rejected.

The work of Filippini et al. [27] focuses on the second component of this pipeline, rather than on forecasting the arrival rate or computing the final workload-distribution policy.

The need to account for multiple functions executing on the same computing node also emerges in *Interless*, proposed by Ma et al. [26]. Interless considers the resource interference that may occur among co-located function instances and uses their historical request rates and request concurrency to predict future vCPU utilization and network-bandwidth consumption. The model is evaluated on a Docker Swarm cluster and obtains a MAPE of 3.2% over the considered 36-hour prediction horizon [26].

Although both approaches consider multiple functions sharing the same computing infrastructure, their prediction objectives differ. Interless forecasts the resource consumption of co-located function instances, whereas the framework

proposed by Filippini et al. estimates function and node-level resource metrics and the binary node-overload state from a given workload configuration [26, 27].

The importance of node-level performance prediction arises from the limitations of static capacity thresholds defined independently for each function. Such thresholds do not represent the combined resource demand generated when several functions are executed concurrently on the same node. For example, treating the capacity of each function independently may classify a node as overloaded when one function slightly exceeds its predefined request threshold, even though the remaining functions are inactive and resources are still available. Moreover, the simultaneous execution of several individually admissible workloads may generate excessive aggregate resource consumption. Node-level performance models can instead represent the relationship between the complete workload composition and the resulting state of the computing node [3, 27].

The framework proposed by Filippini et al. [27] is organized into data collection, model development, and inference stages.

During data collection, a *Samples Generator* performs controlled load tests by varying the request rates of the deployed functions. Measurements collected at both function and node levels are used to train the performance-prediction models and determine the binary overload label [27].

Since this thesis reuses data collected in that experimental campaign, the data-collection procedure, recorded metrics, and criteria used to define overload are described in greater detail in Chapter 5.

A distinctive component of the framework is the *Function Profiler*, which groups functions according to their observed resource-consumption patterns. For each function and tested workload level, summary statistics are computed from the monitored metrics and combined into a feature vector. After normalization, Principal Component Analysis is applied to obtain a lower-dimensional representation, which is subsequently processed using K-Means clustering. The resulting clusters represent functions exhibiting similar resource-usage profiles [27].

This profiling mechanism is intended to reduce the dependence of the performance models on the identity of individual functions. When a previously unseen function is introduced, the *Profile Predictor* assigns it to the cluster whose resource-consumption behaviour most closely matches its profile. The system can then select prediction models associated with that cluster instead of requiring a separate set of models to be trained for every new function. Consequently, the framework considers both function-specific models and more general models constructed from function classes [27].

The *System Forecaster* receives the incoming request rates and the node type as input features and predicts performance metrics at either function, function-class,

or node level. A separate model is trained for each target variable. The authors use AutoGluon for an initial comparison of candidate algorithms and select LightGBM for the reported experiments. Continuous quantities are estimated through standard or quantile regression, whereas the node operating state is predicted through classification. Quantile regression is included to provide estimates that can be used more conservatively in resource-management decisions, for example by considering a high quantile of the expected resource consumption rather than only its conditional average [27].

The framework is evaluated using OpenFaaS through an experimental campaign involving six functions and three virtual node configurations. Different workload combinations are tested to collect normal and overloaded operating conditions [27].

The profiling experiments identify three function groups corresponding to different resource-utilization patterns. The regression results show that the class-based models can estimate node-level CPU utilization and power consumption with performance close to that of models using the identities of the individual functions. For standard regression, the reported class-based node models obtain R^2 values of 0.97 for CPU utilization, 0.77 for memory usage, and 0.93 for power consumption. The corresponding function-specific models obtain R^2 values of 0.98, 0.95, and 0.95, respectively. The larger difference observed for memory indicates that aggregating functions into classes may discard information that remains useful for predicting highly variable metrics [27].

For node-state classification, the model is evaluated on 45,370 test instances and achieves an overall accuracy of 97%. Both the overloaded and non-overloaded classes obtain an F1-score of 0.97. The recall is 0.99 for overloaded nodes and 0.96 for non-overloaded nodes, indicating that the classifier detects most overload conditions while maintaining a limited number of false alarms [27].

These predictions can support load-management decisions by estimating whether a given workload configuration is expected to overload the node [27].

The work demonstrates the feasibility of learning the relationship between incoming FaaS workloads, resource consumption, and node overload conditions. Nevertheless, its experimental scope is limited to six functions and three virtual node configurations, with each virtual machine evaluated separately to avoid interference.

Each target variable is handled by a separate regression or classification model. The framework therefore does not provide a unified multi-target architecture. Its function-specific models also rely on a fixed representation of the deployed functions, limiting their flexibility when workload compositions change. These

aspects are addressed by the extended framework discussed in the following subsection [4, 27].

2.1.2 Multi-Target and Permutation-Invariant Performance Prediction

The performance-modeling framework introduced by Filippini et al. [4] extends their earlier work [27]. The initial framework trains a separate regression or classification model for each target variable. Although these specialized models can provide accurate predictions, their number grows with the performance metrics and levels of granularity considered. Function-based models are tied to workload compositions represented in the training data and cannot directly handle every possible combination of functions that may subsequently be deployed on a node [4].

This limitation is particularly relevant in FaaS environments because the set of functions active on a node can vary over time. Enumerating all possible workload compositions during data collection would become impractical as the number of available functions increases. Furthermore, the order in which functions are represented in an input sequence has no physical meaning: a node hosting functions f_1, f_2, and f_3 is in the same configuration regardless of the order in which these functions are supplied to the predictive model. A suitable architecture should therefore process variable compositions while producing predictions that do not depend on an arbitrary ordering of their elements [4].

To address these requirements, Filippini et al. [4] introduce a multi-target neural regressor inspired by Janossy pooling. Instead of training an independent model for each resource metric, the proposed model produces several continuous outputs simultaneously. In the reported evaluation, these outputs correspond to node-level CPU utilization, memory consumption, and power consumption. The shared model can therefore learn a common internal representation of the relationship between workload composition, node characteristics, and multiple aspects of resource usage [4].

Let a workload deployed on a node contain k functions. Each function f_i is represented by a tuple containing its expected request rate λ_{f_i}, an embedding derived from its previously observed execution profile, and an encoding of the node type. The function embedding is obtained from the profiling process described in the previous work, where resource-consumption measurements are summarized and transformed through Principal Component Analysis. Consequently, the model receives information about both the expected workload and the characteristic resource behaviour of each function [4].

The desired mapping can be represented as

$$\phi : \{(\lambda_{f_1}, \mathbf{e}_{f_1}, \mathbf{n}), \ldots, (\lambda_{f_k}, \mathbf{e}_{f_k}, \mathbf{n})\} \longrightarrow \left(\widehat{\mathrm{CPU}}_n, \widehat{\mathrm{RAM}}_n, \widehat{\Pi}_n\right), \tag{2.1}$$

where $\mathbf{e}_{f_i}$ denotes the profile-based representation of function f_i, $\mathbf{n}$ encodes the node type, and the outputs are the predicted node-level CPU utilization, memory consumption, and power consumption. Since the input represents a collection of simultaneously deployed functions, the expected output should remain unchanged under any permutation of its elements [4].

The architecture processes the function representations through a fully connected input layer with 64 neurons, followed by three stacked GRU layers with 80 hidden units each. A multilayer perceptron containing a 128-neuron fully connected layer and dropout regularization produces the resource predictions. The network is trained using Adam by minimizing a joint mean squared error over the target variables. During training, the input functions are randomly permuted, encouraging the network to produce consistent predictions for different orderings of the same workload composition and thereby approximate permutation-invariant behaviour [4].

Figure 2.1 illustrates this processing pipeline, from the construction and permutation of the input function representations to the generation of multiple performance predictions.

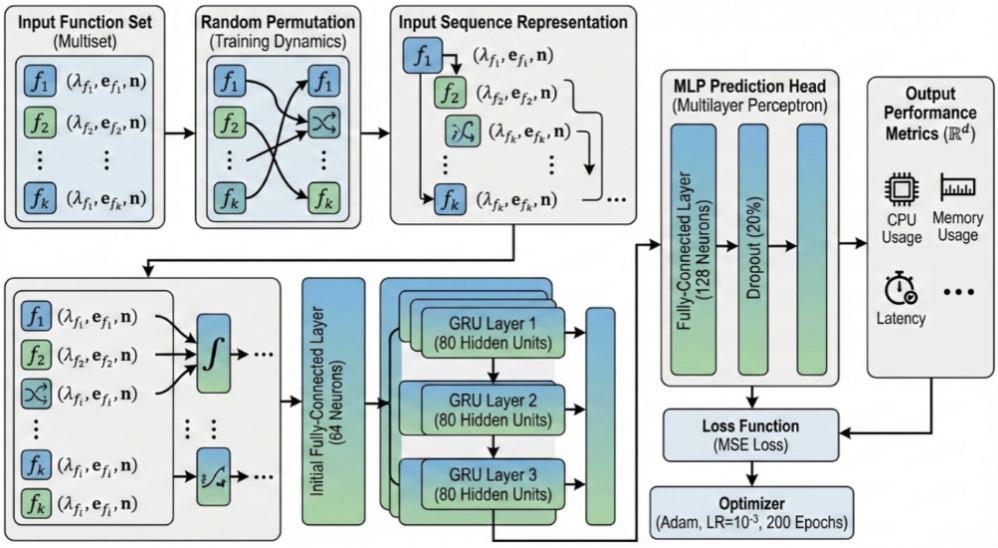


Figure 2.1. Architecture of the Janossy-inspired multi-target neural model. Source: Filippini et al. [4].

This training procedure follows the principle underlying Janossy pooling, where a function defined over a set is represented by considering permutations of its elements. Computing the output over every possible permutation would, however, become prohibitively expensive as the number of functions increases. The adopted implementation therefore relies on sampled permutations during training rather than explicitly enumerating the complete permutation space. The resulting model is consequently described as a Janossy-inspired neural network [4].

The experimental results indicate that the multi-target model can estimate the three node-level resource metrics without requiring a separate neural network for each output. On the test data, the authors report a MAPE of 5.93% for CPU utilization, 5.42% for memory consumption, and 12.12% for power consumption. The higher error associated with power consumption is attributed to the greater complexity and non-linearity of its relationship with the workload. Overall, the results are reported as being comparable with those achieved by the specialized single-target regressors evaluated in the same study [4].

2.1.3 Multi-Task Performance Prediction with Joint Regression and Classification

The permutation-invariant architecture presented by Filippini et al. [4] demonstrates that multiple node-level resource metrics can be estimated jointly from a variable composition of FaaS workloads. The model is formulated as a multi-target regression system: it predicts several continuous resource quantities but does not integrate the prediction of the binary node-overload state into the same neural architecture.A related approach is developed by Khan [5], who proposes a unified multi-task model that simultaneously performs resource regression and overload classification.

The motivation for this unified formulation is that the considered prediction targets are related. Resource utilization at function level contributes to the aggregate state of the node, while the resulting node-level conditions influence whether a workload is classified as sustainable or overloaded. Training completely independent models would require each model to learn partially overlapping relationships between function workloads, hardware characteristics, resource consumption, and service degradation. The multi-task architecture therefore uses a shared representation to capture common workload–resource dynamics before specializing its predictions through separate output branches [5].

Unlike the profile-based representation described in the preceding subsection, Khan's model represents each function through an explicit one-hot encoding of its identity rather than a PCA-derived execution profile [4, 5].

For each observation, the original vector of function request rates is transformed into a sequence containing only the active functions. Each function is represented by

$$\mathbf{x}_{f_i} = [\lambda_{f_i}, \mathbf{id}_{f_i}, n], \tag{2.2}$$

where λ_{f_i} is the request rate of function f_i, $\mathbf{id}_{f_i}$ is a one-hot encoding of its identity, and n identifies the node type. Since the dataset contains six functions, the one-hot function identifier has six components; together with the request rate and node type, each active function is represented by an eight-dimensional vector [5]:

$$\underbrace{1}_{\text{request rate}} + \underbrace{6}_{\text{one-hot function identifier}} + \underbrace{1}_{\text{node type}} = 8.$$

Inactive functions, whose request rate is zero, are removed from the sequence. Consequently, the number of elements processed by the model depends on the number of functions active in a particular workload configuration. Zero padding is subsequently applied to obtain equally sized inputs within each batch, while a masking layer prevents the padded elements from influencing the learned representation [5].

As in the preceding permutation-invariant model, the order assigned to the active functions has no semantic meaning. The datasets used were collected through experiments including combinations of three functions selected from a pool of six. After excluding functions with zero request rates, each sample therefore contains at most three active functions. This makes it computationally feasible to enumerate all possible orderings with a maximum of

$$3! = 6 \tag{2.3}$$

permutations. Unlike an approximation based on randomly sampled permutations, this procedure explicitly considers every possible ordering of the active functions during both training and evaluation [5].

Each function representation is first processed by a shared fully connected embedding layer containing 64 units with ReLU activation. The same transformation is applied independently to every function and every permutation. The embedded sequence is then processed by a GRU containing 80 hidden units. For each permutation, the final GRU state provides a fixed-dimensional representation

of the corresponding ordered sequence. The GRU parameters are shared across all permutations, ensuring that every ordering is processed by the same recurrent encoder [5].

The permutation-specific representations are combined using Janossy pooling. Given K permutations and their corresponding recurrent representations $\mathbf{h}_1, \ldots, \mathbf{h}_K$, the aggregated representation is

$$\mathbf{z} = \frac{1}{K} \sum_{k=1}^{K} \mathbf{h}_k. \tag{2.4}$$

Because the average is computed over all possible orderings of the active functions, $\mathbf{z}$ does not depend on the ordering initially assigned to them. At the same time, processing each permutation with a recurrent encoder allows the model to represent interactions among functions before the different orderings are aggregated [5].

The resulting permutation-invariant vector is passed to a shared backbone consisting of a fully connected layer with 128 units and a hyperbolic tangent activation, followed by dropout with rate 0.2. This shared backbone produces the common latent representation used by the two prediction tasks. The architecture then divides into a regression head and a classification head, allowing task-specific transformations while retaining the information learned jointly in the preceding layers [5].

The regression head contains a fully connected layer with 128 ReLU units, dropout with rate 0.2, and a linear output layer with 22 units. These outputs include function-level CPU utilization, memory consumption, and average latency for the six functions, together with node-level CPU and memory related metrics. The classification head follows a similar structure but terminates with one sigmoid unit, whose output represents the probability that the node is overloaded [5].

Figure 2.2 illustrates the complete architecture and its regression and classification branches.

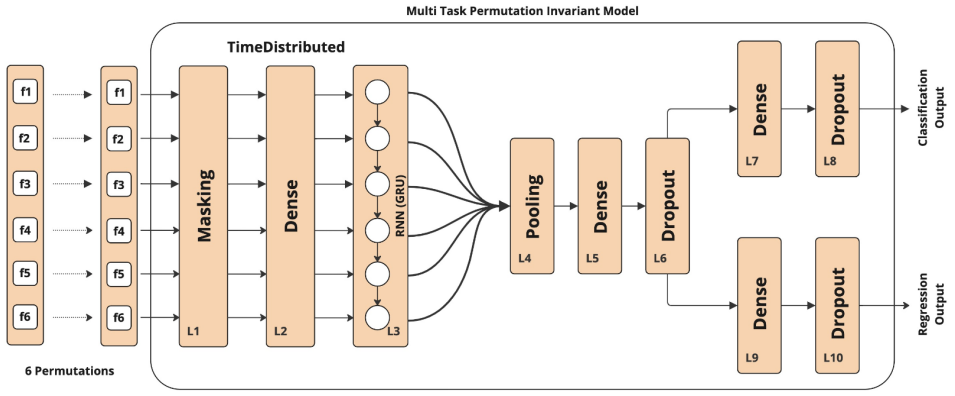


Figure 2.2. Janossy-based multi-task architecture for performance regression and node-overload classification. Source: Khan [5].

The two branches are trained jointly using a weighted objective:

$$\mathcal{L} = 5\mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{cls}}, \tag{2.5}$$

where $\mathcal{L}_{\text{reg}}$ is the masked mean squared error obtained by averaging the considered regression errors, and $\mathcal{L}_{\text{cls}}$ is binary cross-entropy for overload classification. The regression errors are therefore aggregated into a single loss before applying the weight of 5, which balances its contribution against the classification loss [5].

Masking the regression loss is required by the strategy used to mitigate the imbalance between overloaded and non-overloaded observations. Additional overloaded configurations are generated for the classification task by increasing the request rates of workloads already known to cause overload. Since these generated configurations do not have corresponding measured resource values, their regression targets are marked as missing. They contribute to overload classification but are excluded from the regression-loss calculation [5].

Khan [5] evaluates the unified architecture using grouped and stratified five-fold cross-validation. Workload configurations sharing the same feature vector are assigned to the same group so that identical configurations do not appear in both training and test subsets. When trained and evaluated within the target domain, the model obtains an aggregate MAPE of approximately 2.01% across the continuous outputs. For overload classification, it achieves an average accuracy of approximately 95.9%, while the F1-score for the minority overloaded class is approximately 0.835 [5].

The base multi-task, permutation-invariant architecture developed by Khan provides the predictive model adopted in this thesis. Its use within the decentralized training procedure is described in Chapter 4.

2.2 Distributed Training of Predictive Models

The performance-prediction models examined in the previous section are trained using measurements collected from computing infrastructures. When such measurements are generated at geographically distributed nodes or belong to different administrative domains, transferring them to a single location may be costly or restricted by data-ownership and confidentiality requirements. Distributed learning provides an alternative in which the participants collaborate in model training while retaining their respective datasets locally. This section reviews solutions that apply distributed learning to performance prediction, beginning with server-coordinated Federated Learning and subsequently considering fully decentralized Gossip Learning.

2.2.1 Federated Learning for Performance Prediction

An application of Federated Learning to performance prediction is presented by Lan et al. [28] in a multi-operator telecommunication scenario. The considered services comprise components deployed across geographically distributed data centers, which may belong to the same operator or to different operators [28]. Predicting service-level performance may require measurements produced by several components, but directly transferring the corresponding infrastructure data across administrative domains can introduce communication overhead and data-governance concerns. The authors investigate whether a shared predictive model can be trained by exchanging model parameters instead of the original operational measurements [28].

The proposed approach represents each service component running on a server as a learning agent. Infrastructure measurements collected from the execution environment of an agent constitute the input features, whereas measurements describing the performance of the application service constitute the prediction targets. The agents train local copies of a common neural model and periodically send their parameters to a coordinating server. The server applies Federated Averaging to produce an updated global model, which is subsequently returned to the participants for further local training [28].

The evaluation uses measurements collected over time on a testbed at KTH University running a Video-on-Demand application and a Key-Value store. The experiments consider periodic workloads, sudden increases in incoming requests, and separate or concurrent execution of the two services. The measurements are partitioned into 24 federated agents for each service. Inputs describe server-level conditions, including CPU utilization, memory usage, and disk operations,

whereas the prediction targets describe application performance: read and write latencies for the Key-Value store, and network read, display, and audio-playback delays for Video-on-Demand [28].

The authors also examine data heterogeneity across agents. Their analysis identifies differences in both infrastructure-feature distributions and prediction targets, while machines performing similar roles tend to exhibit more similar input distributions [28].

The study compares local learning, in which each agent trains only on its own data; centralized learning, in which all training data are pooled; and Federated Learning, in which agents collaborate through server-side parameter aggregation. The comparisons use the same training and test partitions and the same feedforward neural architecture, allowing the evaluation to focus on the organization of training rather than differences between models [28].

A central aspect of the evaluation is the relationship between local training and communication. Federated training is limited to 20 communication rounds, while the number of local epochs performed before each aggregation is varied among 10, 50, and 200. These configurations are denoted respectively as FL-10, FL-50, and FL-200. Local and centralized models are trained for 200 epochs. The authors additionally consider two dataset sizes, referred to as the *Small Set* and the *Tiny Set*, to study how limited local data affect the comparison [28].

The results show that Federated Learning can outperform models trained only on local data and, for some prediction tasks, approach the performance of centralized learning. In the Video-on-Demand experiments, all the evaluated federated configurations improve over local learning. However, FL-200 provides the best overall performance for this service, although no single federated configuration consistently performs best across all predicted metrics [28].

A different pattern is observed for the Key-Value service. In this case, FL-50 produces the best aggregate federated result and improves over local learning, whereas configurations using fewer or more local epochs can perform worse than the local model. The authors relate these differences to the heterogeneity of agent data and to the distributions of the individual prediction targets. Consequently, no single number of local epochs is found to be optimal for every service and performance metric [28].

The experiments also show that increasing local computation does not necessarily compensate for a limited number of communication rounds. When agents perform many local epochs before aggregation, their models may become more specialized to their respective datasets, reducing the effectiveness of parameter averaging. Conversely, excessively frequent aggregation would require more communication rounds, which are assumed to be constrained in the scenario

considered. The appropriate balance therefore depends on the data distributions and on the prediction task [28].

Overall, the study shows that Federated Learning can support performance prediction without centralizing operational data. Its effectiveness depends on the prediction task, data heterogeneity, and the balance between local training and communication.

2.2.2 Gossip Learning: From Distributed Classification to Edge Workload Forecasting

The work of Ormándi et al. [24] represents a foundational contribution to fully decentralized Gossip Learning. The authors study a peer-to-peer setting in which every node holds a single training example and multiple linear models circulate through the network. The models are updated using locally available data, and the proposed merging mechanism combines linear models by averaging their parameter vectors. The experimental evaluation, conducted on binary-classification tasks, also examines voting over locally cached models and the effects of message losses and delays. The study demonstrates that predictive models can be trained without a central coordinator [24].

A subsequent study by Hegedűs et al. [29] compares Gossip Learning directly with server-coordinated Federated Learning. To isolate the effects of their different communication structures, the authors use the same local update procedure in both approaches and allocate them comparable communication resources. The experiments employ linear classification models trained through stochastic gradient descent and consider networks containing either 100 or 1,000 nodes. The evaluation also varies the distribution of class labels among participants, distinguishing a uniform assignment from a more heterogeneous setting in which the data stored by individual nodes belong to a single class [29].

Communication efficiency is investigated by applying parameter subsampling to both approaches. The authors also evaluated the availability of participants using a churn trace collected from smartphones. Under the assumptions adopted in the study, Gossip Learning obtains results comparable to Federated Learning overall.

With uniformly distributed data and high compression, it can converge faster than the federated alternative because peer-to-peer model exchanges use the available communication resources efficiently. The comparison is less favorable to Gossip Learning in some churn conditions, as a participant returning to Federated Learning can obtain the current model directly from the central server, while a

returning gossip participant may initially interact with models containing less recent information [29].

These findings provide empirical evidence that removing the aggregation server does not necessarily prevent distributed models from attaining performance comparable to centrally coordinated Federated Learning. At the same time, the results depend on the communication assumptions, compression level, data distribution, and availability pattern considered in the experiments.

A direct application of Gossip Learning to prediction in an edge environment is presented by Tundo et al. [7]. The authors consider a Decentralized Function-as-a-Service scenario in which geographically distributed edge nodes observe different workload patterns. Each node stores a local univariate time series representing the number of invocations received by a serverless function and trains an LSTM model to predict its future incoming workload. The observations remain at the nodes where they were collected, while model parameters are exchanged through the peer-to-peer network [7].

In this approach, every participating node maintains its own forecasting model. Nodes periodically select a subset of peers and transmit their current parameters. When a node receives model updates, it combines the incoming parameters with its local model and subsequently fine-tunes the result using its private dataset. Repeated exchanges allow the models to incorporate patterns originating from different nodes without requiring the maintenance of an authoritative global model by a central server [7].

The protocol proposed by Tundo et al. [7] extends the basic Gossip Learning procedure by investigating different peer-selection and merging mechanisms. Incoming models can be evaluated on local validation data and stored temporarily in a cache. After receiving the required number of models, a subset of the best-performing models can participate in the merge.

The authors investigate different model-merging strategies [7]. These strategies are also adopted in this thesis and are described in detail in Chapter 4.

The experimental analysis varies the number of model updates, the number of local epochs performed after an update, the probability of selecting neighbouring nodes, the fraction of model weights transmitted, and the stopping criterion. The study compares Gossip Learning with local-only training, centralized training on the combined node datasets, and Federated Learning using FedAvg. Both synthetic workloads and traces derived from Microsoft Azure Functions are considered. Under the evaluated configurations, Gossip Learning generalizes better than local-only training and achieves results competitive with Federated Learning, whereas centralized training generally retains an advantage in generalization [7].

The scalability analysis shows that the average communication activity of a node is influenced primarily by its number of neighbours and by the protocol configuration rather than directly by the total number of participants. Gossip Learning distributes this traffic among the nodes instead of concentrating it at one aggregation server. The authors also simulate temporary node failures and report no substantial predictive-performance difference between failed and non-failed nodes under the evaluated conditions. The experiments therefore demonstrate the applicability of Gossip Learning to dynamic edge environments while highlighting the trade-off among model generalization, communication overhead, and local computation [7].

3
Problem statement

3.1 Research Problem

As discussed in the previous chapters, this thesis considers distributed Function-as-a-Service systems deployed across edge computing infrastructures. In the reference scenario, multiple Edge FaaS nodes cooperate to execute function requests and may redistribute part of their workload when the locally available computational resources are insufficient. Supporting this cooperation requires information about the effect that an incoming workload is expected to have on the available nodes.

The nodes considered in this work execute functions drawn from the same function pool and are described through a common set of workload and performance variables. However, they may differ in their computational capacity and system characteristics. Consequently, the same workload configuration can produce different levels of resource utilization and different operating conditions depending on the node on which it is executed. A workload that can be processed without difficulty by one node may therefore cause resource saturation or overload on another.

The experimental setting considered in this thesis is based on two previously collected Edge FaaS datasets, originally referred to as the Source Domain and Target Domain datasets. According to Khan [5], the Source Domain dataset originates from the experimental study by Filippini et al. [27], whereas the Target Domain dataset was collected as part of the thesis work conducted by Howard-Grubb [30]. The two datasets use the same pool of serverless functions and follow a comparable data-generation procedure. Both adopt the labels *Light*, *Mid*, and *Heavy* to identify their node configurations; however, equally named configurations belonging to different domains are treated as distinct node types in this thesis. This distinction reflects the differences in physical infrastructure, deployment configuration, and operating conditions under which the two datasets were collected. They therefore describe workload, resource-utilization, and overload behaviour under two different execution environments. In this thesis, the terms *Source*

Domain and *Target Domain* identify the original data collection environments and do not indicate the use of a source-to-target domain adaptation procedure. Their collection procedures, structure, and statistical characteristics are presented in Chapter 5.

Predictive models can support resource-management decisions in this setting. Given a workload composition and the characteristics of a candidate execution node, a predictive model should estimate the resulting performance and resource-utilization metrics and determine whether the node is expected to support the workload without becoming overloaded. These predictions can first be produced for the local node to assess both the amount of resources that would be consumed and the risk of overload if the incoming requests were processed locally. If local execution is not considered appropriate, the same predictions can be produced for neighboring nodes by estimating the resource utilization, performance, and overload condition that would result from transferring the workload to each candidate destination. This information can support the comparison of alternative execution nodes, avoiding destinations that are expected to become overloaded and preferring those that can accommodate the additional workload while using the resources available across the distributed infrastructure more effectively. This thesis focuses on producing the predictive information required to support such decisions; it does not implement or evaluate the offloading policy itself.

The training observations required to construct these models are distributed among the participating nodes. Each node has direct access only to the measurements contained in its local dataset and therefore observes only a portion of the operating conditions represented across the infrastructure. A model trained exclusively on local data may consequently become specialized to the workload patterns and performance behaviour observed by its own node, without incorporating knowledge available at other participants. Conversely, collecting all observations in a single location would introduce a centralized component into the training process.

Among the studies reviewed in the previous chapter, the work of Tundo et al. [7] represents the closest methodological predecessor of the decentralized training investigated in this thesis. However, its LSTM model performs workload forecasting by predicting future function-invocation volumes; it does not estimate the resource utilization or overload condition resulting from a given workload. Similarly, the comparative study by Hegedűs et al. [29] evaluates Gossip Learning using linear classification models rather than a permutation-invariant multi-task architecture. Therefore, neither study investigates the gossip-based training of a model that jointly predicts continuous performance metrics and the binary overload state of an Edge FaaS node.

The problem addressed in this thesis is therefore the completely decentralized training of the permutation-invariant multi-task architecture developed by Khan [5], which jointly predicts continuous performance metrics and the binary overload state of Edge FaaS nodes. The adopted Gossip Learning procedure is derived from the approach of Tundo et al. [7]: participating nodes retain their respective datasets locally and collaborate by exchanging and merging model parameters through peer-to-peer communication. Repeated local training and model exchange are intended to allow the models to incorporate knowledge originating from different nodes without relying on a central training coordinator.

3.2 Research Questions

The research problem defined in the previous section is investigated through the following research questions.

RQ1 – Effectiveness and generalization of decentralized training.
Can Gossip Learning train the adopted permutation-invariant multi-task model to achieve better generalization than local-only training and predictive performance comparable to centralized training?

This question compares Gossip Learning with two reference settings. Local-only models are trained exclusively on the dataset assigned to their respective nodes and therefore have no opportunity to incorporate knowledge from other participants. They may consequently perform well on observations resembling their local training data but generalize poorly to workloads and performance behaviours observed on other nodes, particularly nodes belonging to different machine types. Through peer-to-peer model exchange and merging, Gossip Learning is expected to expose each local model indirectly to knowledge originating from multiple nodes and thereby improve its ability to predict continuous performance metrics and overload conditions beyond its local data distribution. Centralized training provides the second reference because a single model can be optimized using the complete collection of observations available across the infrastructure. Although the centralized model is expected to provide a strong performance benchmark, this research question investigates whether Gossip Learning can approach its regression and classification performance while avoiding centralized data collection. The comparison therefore evaluates the trade-off between predictive quality and the advantages of retaining the nodes to which they are

assigned, like reduced need to transfer raw data, and the absence of a central training coordinator.

RQ2 – Model-merging strategies.
How do different model-merging strategies affect the regression and classification performance of the adopted multi-task model?

This question evaluates the role of model merging in determining how knowledge received from different participants is incorporated into the local models. The experimental analysis initially considers the four strategies adopted in the reference Gossip Learning approach of Tundo et al. [7]: *Overwrite*, *Improved Overwrite*, *Simple Averaging*, and *Age-Weighted*. These strategies apply different mechanisms for replacing or combining model parameters and may therefore differ in their ability to preserve locally acquired knowledge, incorporate information received from other nodes, and account for the training history accumulated by the exchanged models. If the reference strategies do not provide satisfactory results or reveal limitations when applied to the adopted multi-task architecture, additional merging strategies are designed and evaluated. The definitions, implementation details, and configuration parameters of all evaluated strategies are presented in Chapter 4.

RQ3 – Gossip Learning parameter configuration.
How do the configuration parameters of the Gossip Learning process affect predictive performance, and which of the evaluated configurations provides the most effective results?

This question investigates how the main parameters governing decentralized training interact in determining the quality of the resulting models. The amount of local training controls how extensively a model is optimized on the dataset available at a node before incorporating knowledge received from other participants. Additional local training may improve learning from local observations, but it may also increase specialization to the local data distribution. The number of Gossip Learning updates determines how frequently knowledge can propagate across the network, while the number of models participating in each merge controls how much information originating from different nodes is combined during an update. These parameters are evaluated jointly to identify, among the configurations considered in this thesis, the combination that provides the most effective balance between local optimization and distributed knowledge exchange.

Together, these questions examine whether the adopted predictive architecture can be trained through a fully decentralized process and how the principal choices governing local training and model merging influence its performance. The experimental methodology used to address these questions is presented in Chapter 4.

3.3 Assumptions and Constraints

The research questions defined in the previous section are investigated under a controlled set of system, communication, and data-distribution assumptions. These assumptions make it possible to isolate the effects of decentralized training and model merging, but they also delimit the extent to which the experimental findings can be generalized to operational Edge FaaS environments.

3.3.1 System and Communication Assumptions

Common model architecture. All participating nodes use the same permutation-invariant multi-task model architecture, including the same input representation, parameter structure, prediction outputs, and loss function. This compatibility is required because models exchanged by different nodes are combined directly at the parameter level.

Common function pool. The workloads considered by the participating nodes are composed of functions selected from the same function pool. Individual observations may contain different active functions and request rates, but function identities and their semantic meaning are shared across the infrastructure.

Consistent feature and target space. Local datasets share the same feature and target structure. Input variables and prediction targets have the same interpretation across nodes, although their values and statistical distributions may differ across node types.

Communication reachability. The communication network is assumed to be fully connected. Every node can directly exchange a model with every other participant, although an individual communication event may involve only the peers selected by the protocol.

Availability of node-type information. Raw observations remain stored locally and are not exchanged among participants. However, the additional merging strategy developed in this thesis requires each node to commu-

nicate its node-type identifier together with its model parameters. This identifier provides limited contextual information about the data-collection and execution environment associated with the sending node, allowing the merging procedure to distinguish models originating from different node types. It does not disclose the individual workload or performance observations contained in the node's local dataset.

3.3.2 Experimental Constraints

No evaluation of partially connected topologies. The experiments assume a fully connected communication network in which every participant can directly exchange models with every other node. This choice isolates the effects of local training, model exchange, and merging from those introduced by the communication topology. In partially connected networks, the propagation of knowledge would also depend on node degree, neighbourhood composition, path length, and the position of heterogeneous node types within the graph, making it more difficult to attribute observed performance differences to the Gossip Learning configurations under investigation.

IID partitioning and controlled statistical heterogeneity. For each node type, the available observations are randomly shuffled and partitioned without overlap among the nodes associated with that type. Each local partition is therefore treated as being drawn from the same node-type-specific distribution.

This IID assumption applies conditionally on the node type and does not imply that datasets associated with different node types or data-collection environments follow the same distribution.

This partitioning procedure represents a controlled simplification, since operational nodes belonging to the same node type may observe non-IID data because of differences in geographical location, user behaviour, application demand, class proportions, temporal workload patterns, and operating conditions. Consequently, the experiments preserve heterogeneity across distinct node types and data-collection environments but do not reproduce the full statistical heterogeneity that may occur among real nodes of the same type.

Static, previously collected local datasets. The Gossip Learning experiments operate on previously collected observations that are partitioned among the participating nodes before decentralized training begins. The resulting local datasets remain fixed throughout each experiment, and no new measurements are generated or added during training. Consequently, the

study does not model online data streams, evolving workloads, temporal distribution shifts, or concept drift. The experimental findings are therefore limited to the functions, node types, workload configurations, and operating conditions represented in the available datasets.

Fixed model architecture. The permutation-invariant multi-task model developed by Khan [5] is adopted as the reference architecture because it jointly addresses the two prediction tasks considered in this thesis and was previously shown to provide suitable predictive performance when trained on the complete available dataset. It therefore provides an appropriate starting point for investigating whether the same architecture can be trained effectively when observations are distributed among multiple nodes. Alternative predictive architectures are not evaluated because the study focuses on decentralized parameter exchange and model merging.

3.4 Contributions

Based on the research problem and experimental scope defined in the preceding sections, the main contributions of this thesis are:

- the adaptation and implementation of a Gossip Learning process for the decentralized training of a permutation-invariant multi-task model that jointly performs continuous performance regression and overload classification;
- an experimental comparison of local-only, Gossip-based, and centralized training, with particular attention to the ability of the resulting models to generalize beyond their local datasets;
- a systematic evaluation of how the configuration of local training, model exchanges, and merging operations affects regression and classification performance;
- a comparative analysis of the model-merging strategies inherited from the reference Gossip Learning approach;
- the design and evaluation of additional merging strategies for the adopted multi-task architecture.

4
Methodology

This chapter describes how the previously collected Source and Target datasets are prepared, partitioned among the simulated Edge FaaS nodes, and used for decentralized model training. A complete description of their original collection environments, variables, and distributions is provided in Chapter 5.

4.1 Methodological Overview

The experimental workflow begins with the generation of a network and the assignment of preprocessed data to its nodes.

Before running the Gossip Learning simulations, two training baselines are established using the same predictive architecture.

In the local-only baseline, each node trains its model exclusively on its own dataset, without receiving parameters from other nodes. This provides a reference for assessing whether model exchange helps a node learn beyond the information available locally. In the centralized baseline, a single model is trained on the combined training data of all participating nodes. This provides a reference for the predictive performance achievable when the training data are available in one place.

In the Gossip Learning simulations, each node maintains its own predictive model and retains its training observations locally. Nodes exchange model parameters with selected peers, process received models according to the chosen merging strategy, and perform subsequent local training updates. The resulting models are evaluated for both regression and overload classification.

The experiments were conducted in two settings. Initial tests used a fully connected network of nine nodes, each connected to the other eight, and considered only Source Domain data. Later tests with the configurations that performed better included both Source and Target Domain data in a fully connected network of eighteen nodes, each connected to the other seventeen.

The study considers two network sizes and progressively evaluates existing and proposed merging strategies. The network configurations and experimental sequence are described in Sections 4.2 and 4.5, respectively.

4.2 Network and Data Distribution

The experimental setting consists of simulated networks of Edge FaaS nodes. This section describes how the participating nodes and their data are organized. The event-based simulation and the training procedure are presented in the following sections.

4.2.1 Network

The network is represented as a graph in which each vertex corresponds to a participating node and each edge indicates that the two connected nodes can exchange model messages directly. The generated network is saved as a list of each node's direct neighbours. The Gossip Learning simulator reads this list to determine which exchanges are possible. The graph describes the communication relationships used in the simulation; the experiments do not deploy Gossip Learning on a physical network of Edge FaaS machines.

Two network sizes are considered, both with full connectivity. The initial experiments involve nine nodes, each connected to the other eight. Figure 4.1 shows this configuration. The later experiments involve eighteen nodes, each connected to the other seventeen, to accommodate a larger volume of data and a wider range of node types.

Connectivity determines which nodes may exchange models, but it does not require a node to send its model to every neighbour at each update.

Peer selection and message timing are specified by the Gossip Learning simulation described later in this chapter.

4.2.2 Data Allocation to Nodes

Each simulated node is assigned a node type, which determines the observations available for its local training. In the initial nine-node setting, the data originate from the Source Domain. In the eighteen-node setting, both Source and Target Domain data are used. Node types from the two domains are treated as distinct, even when they share the same descriptive name.

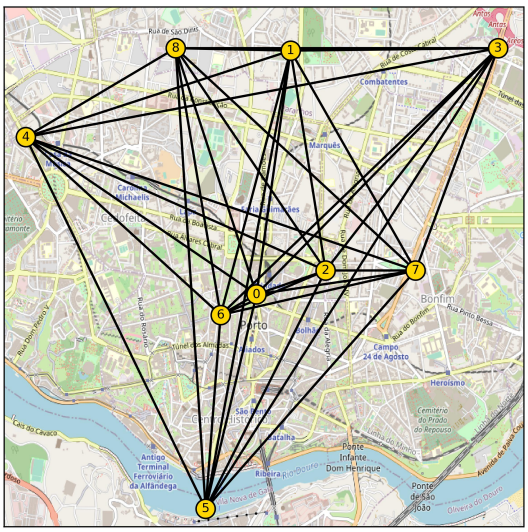


Figure 4.1. Fully connected nine-node network used in the initial experiments. The geographic background visualizes the node positions; the edges represent possible direct model exchanges in the simulation.

For each node type, the available observations are assigned to nodes of that type, so that each node receives a local dataset rather than the complete collection of observations.

The procedure also considers the binary overload label when distributing the available groups among nodes. Consequently, nodes of a given type receive separate portions of the corresponding data, while nodes of different types are associated with different sets of observations.

Each node's assigned data are divided into training, validation, and test partitions following the procedure described in Section 5.2.2. The training partition is used for local optimization, the validation partition is used to monitor predictive performance, and the test partition is reserved for evaluation.

The corresponding partitions from the participating nodes are also combined to construct the datasets used by the centralized reference model. During Gossip Learning, however, nodes retain their own observations locally: the communication protocol exchanges model-related information, not the local datasets.

4.3 Gossip Learning Simulation Environment

The Gossip Learning experiments are executed through a discrete-event simulator, which processes events such as model transmission, reception, and training completion in chronological order according to their simulated timestamps. Neural-network training is performed computationally, while communication between nodes and event timing are simulated. Each participating node maintains its own predictive model and its assigned training, validation, and test partitions. The simulator coordinates local training and model exchanges over the communication graph described in the previous section.

4.3.1 Model Initialization and Local Training

At the beginning of each simulation, a separate instance of the same multi-task predictive architecture is created for every node.

Its parameters are initialized from scratch, without loading weights from the local-only or centralized baselines.

Model parameters evolve according to the data and model exchanges experienced by each node.

Each local update starts from the parameters currently maintained by the node, after any processing of received models prescribed by the selected merging strategy.

The update consists of a configurable number of training epochs, with mini-batch size and sample shuffling controlled by the training configuration. Consequently, a gossip update may contain one or more complete passes over the training dataset available to the node depending on the configured number of epochs.

Before each local training call, training and validation inputs are prepared for the Janossy pooling procedure described in Section 2.1.3. The implementation uses all $3! = 6$ permutations of the three input positions, including padding where fewer functions are active. These permuted inputs are reused across the epochs of the local update.

Local optimization uses Adam and jointly addresses performance regression and binary overload classification. The regression head uses a masked mean squared error, excluding samples for which all regression targets are missing, while the classification head uses binary crossentropy. The optimization objective is

$$\mathcal{L} = \lambda_{\text{reg}}\mathcal{L}_{\text{reg}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}}, \tag{4.1}$$

where $\mathcal{L}_{\text{reg}}$ denotes the masked regression loss and $\mathcal{L}_{\text{cls}}$ denotes the binary crossentropy. The reference configuration uses $\lambda_{\text{reg}} = 5$ and $\lambda_{\text{cls}} = 1$, following the adopted multi-task model, giving

$$\mathcal{L} = 5\,\mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{cls}}. \tag{4.2}$$

These weights control the relative contributions of the two tasks to the optimization objective. The experiments will also investigate whether changing their values affects regression and classification performance.

For each local training call, the implementation creates a newly compiled model and copies the node's current parameters into it. Training therefore continues from the current model parameters. Within an update, the same optimizer is retained across its training epochs.

Upon completion of each local update, the parameters from its final epoch become the node's current parameters and are used in subsequent gossip exchanges and training updates.

4.3.2 Peer Communication and Asynchronous Execution

Nodes cooperate through asynchronous model exchanges over the communication graph, without a central training coordinator. At each sending event, a node randomly selects half of its available neighbours, rounding upwards, and transmits its current model parameters together with the metadata required by the merging strategy. Raw training observations remain stored locally.

Each node maintains a buffer containing at most one received model per sender; a newer model from the same sender replaces the previous one. Once the configured number of models from distinct senders has been collected, the node applies the merging strategy and performs local training. For the proposed strategy, this process also includes the generation and management of synthetic observations, as described in the following section.

Communication and training are represented through a discrete-event simulation. Initial transmissions are staggered randomly over the first 60 simulated seconds, and each directed link has a fixed transmission delay sampled between 25 and 35 simulated seconds. Local updates are assigned a fixed simulated duration, independent of the actual time required to train the model. This separates the timing of simulated network events from the execution speed of the machine running the experiments, providing a consistent timing assumption across nodes and configurations. While a node is training, incoming models are discarded,

while outgoing transmissions can still use the parameters available before the update is completed.

Nodes progress independently, without synchronized training rounds. Each node completes its gossip-training phase after the configured number of local updates, and nodes that have finished training are excluded from subsequent recipient selection.

4.4 Model-Merging Strategies

The merging strategy determines how a node incorporates information received from its peers before performing the next local training update. This section first presents the four reference strategies, which operate directly on model parameters, and then introduces the proposed node-type-aware approach. The latter separates the management of model parameters from the construction and maintenance of synthetic training data.

Let $\boldsymbol{\theta}_i$ denote the parameters currently maintained by node i, and let $\boldsymbol{\theta}_j$ denote the parameters received from node j. The notation $\widetilde{\boldsymbol{\theta}}_i$ identifies the parameters used to begin local training after the merging operation.

4.4.1 Reference Strategies

The reference strategies are Overwrite, Improved Overwrite, Simple Averaging, and Age-Weighted Averaging. They follow the approaches considered by Tundo et al. [7] and differ in how they retain, replace, or combine the local and received parameters. None of these strategies generates synthetic training observations.

Overwrite. Overwrite replaces the local parameters with those of the received model:

$$\widetilde{\boldsymbol{\theta}}_i = \boldsymbol{\theta}_j. \tag{4.3}$$

The subsequent training update adapts the received model to the receiving node's local data. This strategy processes one received model at a time and does not preserve the previous local parameters through an explicit combination.

Improved Overwrite. Improved Overwrite corresponds to the Age-Weighted Overwrite strategy described by Tundo et al. [7]. It uses the accumulated training weights associated with the local and received models to determine the probability of replacement. These weights represent training history rather than elapsed

time. In the adopted implementation, each completed local update adds the size of the node's original training partition to the model's accumulated weight.

For positive local and received weights, denoted by a_i and a_j, the probability of adopting the received model is

$$P_{\text{replace}} = \begin{cases} 1 - \left(\dfrac{a_i}{a_j}\right)^2, & a_j > a_i, \\ \left(\dfrac{a_j}{a_i}\right)^2, & a_j \leq a_i. \end{cases} \tag{4.4}$$

The node either adopts the received parameters or retains its current parameters according to this probability. Local training then proceeds from the selected model. An initially untrained local model, whose accumulated weight is zero, is replaced by the received model.

Simple Averaging. Simple Averaging combines the local and received parameters through an arithmetic mean. Let $\mathcal{R}_i$ contain the senders whose models participate in the merging operation. When complete parameter vectors are exchanged, the resulting parameters are

$$\widetilde{\boldsymbol{\theta}}_i = \frac{\boldsymbol{\theta}_i + \sum_{j \in \mathcal{R}_i} \boldsymbol{\theta}_j}{1 + |\mathcal{R}_i|}. \tag{4.5}$$

Each model contributes equally, regardless of its accumulated training weight. With one received model, the local and received parameters each contribute one half of the resulting parameter vector.

Age-Weighted Averaging. Age-Weighted Averaging assigns different contributions to the participating models according to their accumulated training weights. For positive weights and complete parameter vectors, the merging operation is

$$\widetilde{\boldsymbol{\theta}}_i = \frac{a_i \boldsymbol{\theta}_i + \sum_{j \in \mathcal{R}_i} a_j \boldsymbol{\theta}_j}{a_i + \sum_{j \in \mathcal{R}_i} a_j}. \tag{4.6}$$

Models with larger accumulated weights therefore have a stronger influence on the resulting parameters. Unlike Simple Averaging, this strategy accounts for differences in the training histories represented by the participating models.

For both averaging strategies, the accumulated weight of the merged model is set to the largest participating weight before adding the contribution of the subsequent local training update.

4.4.2 Proposed Strategies

The proposed approach, implemented as `node_type_merge`, uses the node-type information associated with each received model to extend the receiving node's training data. The received model provides predictions for workloads represented as running on the sender's node type. These predictions become synthetic targets used in subsequent local training.

The approach has two independently configurable components. The first determines how the received parameters modify the local model. The second determines how synthetic prediction blocks are retained and combined. Consequently, the same synthetic-data procedure can be evaluated with different parameter-update rules.

4.4.2.1 Node-Type-Aware Synthetic Data Generation

When node i receives a model from node j with a different node type, it constructs synthetic inputs from its original local training inputs. Workload features are preserved, while the node-type feature is replaced with the sender's type. Padding positions remain unchanged. The received model is then used to predict the targets associated with these modified inputs.

The synthetic regression targets are the predictions for node CPU usage, node RAM usage, and node RAM usage percentage. The overload probability is converted into a binary target according to

$$\widehat{y}_{\text{cls}} = \begin{cases} 0, & \widehat{p}_{\text{overload}} < 0.5, \\ 1, & \widehat{p}_{\text{overload}} \geq 0.5. \end{cases} \tag{4.7}$$

The procedure therefore retains continuous regression predictions and hard classification labels.

Each received model from a different node type generates one synthetic prediction block containing as many observations as the receiving node's original training partition. These observations are model-generated estimates rather than measurements collected from the sender, and no raw training records are exchanged.

If the sender and receiver have the same node type, no new synthetic block is generated because real local observations for that type are already available. However, the received model is still processed according to the selected parameter-update rule, and the subsequent local training update uses the currently available training dataset.

4.4.2.2 Model Parameter Update

Three alternatives determine the parameters from which local optimization starts. This choice is independent of how synthetic blocks are combined.

Keep Local. With `keep_local`, the merging operation preserves the receiving node's parameters:

$$\widetilde{\boldsymbol{\theta}}_i = \boldsymbol{\theta}_i. \tag{4.8}$$

Information from models of different node types influences subsequent optimization through the generated synthetic observations. The parameters are preserved only during the merging operation: they continue to change during local training.

Overwrite. With `overwrite`, the received parameters replace the local parameters:

$$\widetilde{\boldsymbol{\theta}}_i = \boldsymbol{\theta}_j. \tag{4.9}$$

The subsequent update trains this model on the local data together with the available synthetic observations. This distinguishes the proposed configuration from the reference Overwrite strategy, which does not maintain synthetic datasets.

Interpolation. With `interpolate`, the receiving node combines the two parameter vectors:

$$\widetilde{\boldsymbol{\theta}}_i = \boldsymbol{\theta}_i + \alpha\,(\boldsymbol{\theta}_j - \boldsymbol{\theta}_i) = (1-\alpha)\boldsymbol{\theta}_i + \alpha\boldsymbol{\theta}_j, \qquad 0 \leq \alpha \leq 1. \tag{4.10}$$

The coefficient α controls the contribution of the received model. Setting $\alpha = 0$ reproduces the parameter-update behaviour of `keep_local`, while $\alpha = 1$ reproduces `overwrite`. Intermediate values retain a contribution from both models. For example, $\alpha = 0.3$ assigns a weight of 0.7 to the local parameters and 0.3 to the received parameters.

4.4.2.3 Synthetic Data Retention and Combination

Each node maintains a separate rolling history of synthetic prediction blocks for every remote node type. The maximum number of retained blocks is controlled by a window size W. When a new block is generated, it is appended to the history of its corresponding type. If that history already contains W blocks, the oldest block is discarded.

The window counts prediction blocks for each type separately. It does not represent a fixed number of global gossip updates, and blocks associated with the same type may originate from different senders. The retained blocks are used

immediately, including when fewer than W blocks are available. Two modes determine how these histories are converted into training observations.

Concatenation. In `concatenate` mode, all retained blocks are included separately in the synthetic training dataset. Identical inputs may therefore appear multiple times with targets predicted by different received models or at different updates.

If node i has N_i local training observations and retains $B_{i,r}$ blocks for remote type r, that type contributes

$$N_{i,r}^{\text{synthetic}} = N_i B_{i,r}, \qquad 1 \leq B_{i,r} \leq W. \tag{4.11}$$

Increasing the number of retained blocks increases both the number of synthetic observations and their contribution to the training dataset.

Aggregation. In `aggregate` mode, the retained predictions are combined into one target vector for each corresponding input. Blocks associated with the same remote type contain the same workloads in the same order, allowing their targets to be aggregated directly.

For regression target q and input x, the aggregated prediction is

$$\overline{y}_q(x, r) = \frac{1}{B_{i,r}} \sum_{b=1}^{B_{i,r}} \widehat{y}_q^{(b)}(x, r). \tag{4.12}$$

Each regression output is averaged independently. Classification targets are combined through majority voting over the binary labels. If the votes are tied, the label predicted by the most recent block is selected.

Aggregation is recomputed whenever a new block is added. It does not require the window to be full: with four available blocks, regression targets are averaged over those four predictions and classification uses their four votes. Each represented remote type contributes N_i synthetic observations, regardless of the number of retained blocks.

Substitution. Substitution is the special case in which the retention window has size $W = 1$. Each node retains only the most recent synthetic prediction block for each remote node type. Whenever a new block is generated for a given type, it replaces the previously stored block for that type, while blocks associated with other types remain unchanged. Each represented remote type therefore contributes N_i synthetic observations. With a single retained block, Concatenation and Aggregation yield the same synthetic dataset: no predictions

from earlier blocks are retained or combined. The term Substitution is used in the experiments to identify this configuration.

In all the modes, the resulting synthetic datasets are concatenated with the original local training partition before optimization. The local validation partition remains unchanged. Parameter interpolation and synthetic-target aggregation therefore operate at different stages: the former combines model parameters, while the latter combines predictions used as training targets.

4.5 Experimental Design and Configuration

The experimental study follows a progressive design. The reference merging strategies are evaluated first to investigate their behaviour under different training configurations. The findings from this stage guide the selection of the settings used to evaluate the proposed node-type-aware approach. This section describes the sequence of experiments and the configuration choices investigated, while the corresponding results and their interpretation are presented in the Results chapter.

4.5.1 Sequence of Experiments

Evaluation of the reference merging strategies. The first stage considers Overwrite, Improved Overwrite, Simple Averaging, and Age-Weighted Averaging. The experiments investigate how their predictive performance changes with the relative weights assigned to the regression and classification losses, the number of received models involved in each merging operation, and the number of gossip updates performed by each node. The number of received models is varied only for strategies that support combining multiple incoming models; Overwrite and Improved Overwrite operate with one received model per update.

These experiments establish the reference performance and identify training settings to carry forward into the evaluation of the proposed approach. In particular, the number of updates specified by `fixed_updates` is selected based on the findings of this initial stage. The configurations retained and the evidence supporting their selection are reported in the Results chapter.

Evaluation of the proposed node-type-aware strategy. The second stage evaluates `node_type_merge`, starting from the training settings selected during the reference experiments. The analysis focuses on two configuration dimensions: the rule used to update model parameters upon reception of a peer model, and the mechanism used to retain and combine synthetic observations.

For model parameters, the experiments compare `keep_local`, `overwrite`, and `interpolate`. These alternatives respectively preserve the local parameters at the merging stage, replace them with the received parameters, or combine the two parameter vectors. The interpolation experiments use $\alpha = 0.3$, assigning a weight of 70% to the local parameters and 30% to the received parameters before subsequent local training. The comparison evaluates this update rule together with the selected synthetic-data management configurations.

For synthetic data, the experiments compare retaining only the most recent prediction block for each remote node type with retaining multiple blocks in a rolling window. The first configuration uses a window size of one, so each new block replaces the previous block associated with the same type. With larger windows, the retained blocks are either concatenated as separate training observations or aggregated through component-wise averaging of regression predictions and majority voting over classification labels.

The evaluated combinations of parameter-update rules, synthetic-data modes, and window sizes are used to assess how the proposed approach balances the retention of local information with the incorporation of knowledge from remote node types. Their results are compared with those obtained using the reference merging strategies.

4.6 Evaluation

The evaluated configurations are compared using metrics for both prediction tasks. Overload classification is assessed through accuracy and F1-score, while regression performance is measured using mean squared error (MSE).

Accuracy measures the proportion of correctly classified observations:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \tag{4.13}$$

where TP and TN denote correctly classified overloaded and non-overloaded observations, respectively. FP denotes non-overloaded observations incorrectly classified as overloaded, while FN denotes overloaded observations incorrectly classified as non-overloaded.

F1-score combines precision and recall through their harmonic mean. For the overloaded class, it can be expressed as

$$F1 = \frac{2TP}{2TP + FP + FN}. \tag{4.14}$$

This metric helps assess the detection of overload conditions when the two classes are imbalanced, accounting for both false overload predictions and missed overload conditions.

MSE measures the average squared difference between predicted and observed regression targets:

$$\text{MSE} = \frac{1}{n}\sum_{k=1}^{n}(y_k - \widehat{y}_k)^2, \tag{4.15}$$

where y_k is an observed target value, $\widehat{y}_k$ is its prediction, and n is the number of valid scalar target values included in the calculation. For each node, MSE is averaged across the three regression outputs and the evaluated observations, excluding observations whose regression targets are entirely missing. It is computed on the scaled target values rather than in their original physical units.

Metrics are computed separately for each node. For each configuration and simulation, the node-level results are summarized through their arithmetic mean and standard deviation. The mean describes average predictive performance, assigning equal importance to each node, while the standard deviation measures how much performance varies across nodes. Consequently, the reported mean F1-score is obtained by averaging the individual node-level F1-scores rather than by pooling predictions from all nodes into a single classification report.

Evaluation considers both each node's local test partition and a common test set. The latter is constructed by randomly sampling approximately 10% of each node's local test partition without replacement and concatenating the selected observations. All node models within a simulation are evaluated on this same dataset. The selected observations remain part of their respective local test partitions; therefore, the common test set is not an additional independently collected dataset.

Local test performance measures how well a model predicts observations associated with its own data distribution, while common-test performance assesses its behaviour across the distributions represented in the network. Additional evaluations restrict the common test set to observations with the same node type as the evaluated node, allowing performance on that type to be examined separately.

Confusion matrices complement the scalar classification metrics by reporting correctly classified observations, false positives, and false negatives. They help distinguish configurations that produce similar accuracy values but differ in their ability to identify overloaded and non-overloaded observations.

Additional diagnostic plots are used to investigate behaviours that are not fully captured by the final metrics. Information about the sender and its node type is also recorded to examine possible associations between model exchanges and changes in predictive performance.

These diagnostics support the interpretation of the experimental results and the investigation of specific behaviours, such as temporary increases in validation loss. The relevant plots and their interpretation are presented alongside the corresponding experiments in the Experimental Results chapter. Associations observed in these plots are treated as evidence for further investigation rather than, by themselves, proof of a causal relationship.

5 Dataset

5.1 Origin and Collection of the Datasets

This thesis reuses performance observations collected in two previous Edge FaaS studies, referred to as the Source Domain and Target Domain datasets. The terms identify the environments in which the measurements were originally obtained; they do not indicate that a source-to-target adaptation procedure is applied in this work. This section describes how the observations were collected, while their structure, characteristics, and preparation for the experiments are discussed in the following sections.

5.1.1 Source Domain

The Source Domain dataset originates from the experimental work of Filippini et al. [27] on performance modeling for Edge FaaS systems. The authors used OpenFaaS to deploy six functions with different resource demands: *figlet*, *shasum*, *env*, *nmap*, *curl*, and *eat-memory*. They evaluated three virtual node configurations, labelled Light, Mid, and Heavy, with respectively 2, 4, and 6 virtual CPUs and 8, 16, and 24 GB of RAM. The configurations were tested separately on a host running Ubuntu 22.04, so that measurements from one node configuration would not be affected by the concurrent execution of another [27].

A component called the *Samples Generator* submitted controlled request traffic to the deployed functions. Each test configuration specified the request rate of the functions under examination. During execution, the framework collected measurements at both node and function level, including CPU and memory usage, function response latency, and request success rate. Power consumption and the number of function replicas were also monitored. Between load tests, a cool-down period allowed the system to return close to its baseline operating state before new measurements were collected [27].

The collected observations include a binary indication of whether the tested node was overloaded. In the original framework, overload was identified when

at least one condition concerning node CPU or memory utilization, the average success rate of the deployed functions, or the success rate of an individual function was met. Thus, the overload label was assigned during the original data-collection process: it is not generated by the Gossip Learning simulation conducted in this thesis [27].

5.1.2 Target Domain

The Target Domain dataset was collected in a separate experimental study by Howard-Grubb, as reported by Khan [5]. It follows the same general data-generation procedure as the Source Domain and considers the same pool of serverless functions and the same logical Light, Mid, and Heavy node configurations. The measurements nevertheless originate from a different execution environment, so matching node-type names do not imply identical observed performance [5].

Unlike the Source Domain experiments, in which node configurations were tested separately on one host, the Target Domain setup used separate virtual machines for the three node types and an additional machine to generate the workload. The experiments could therefore run under different deployment and operating conditions. Power-consumption measurements were not collected in this setup and cannot be used as common prediction targets across the two domains [5].

The workload-generation procedure also differed in the granularity of the tested request rates. While Source Domain rates increased in steps of 10 requests per second, the Target Domain campaign initially used the same step size and later increased it to 20 requests per second. Consequently, some intermediate workload levels represented in the Source Domain are absent from the Target Domain. The criteria used to assign the node overload label remained the same across the two collections [5].

5.1.3 Input Variables and Prediction Targets

Each observation associates a workload configuration with measurements of an Edge FaaS node. The model receives seven input variables: the node type and the request rates of six functions, namely `curl`, `eat_memory`, `env`, `figlet`, `nmap`, and `shasum`. Request rates are expressed in requests per second; a value of zero indicates that the corresponding function is inactive in that observation. The `node_type` variable identifies a Heavy, Mid, or Light configuration within the Source or Target Domain. The two domains use distinct node-type codes,

so configurations with the same descriptive name are not treated as identical execution environments [5].

The model predicts three continuous node-level measurements and one binary overload label:

- `cpu_usage_node`: total CPU usage across the node's cores;
- `ram_usage_node`: node memory usage, expressed in bytes;
- `ram_usage_node_percentage`: percentage of the node's total RAM that is in use;
- `overloaded_node`: a binary label equal to 0 for non-overloaded observations and 1 for overloaded observations.

The first three variables are regression targets. The two RAM variables describe memory usage in different units, while `overloaded_node` is the classification target. The overload label follows the criteria applied during data collection; this thesis does not define it by introducing a new threshold on one of the regression targets [5]. During preprocessing, labels associated with repeated observations of the same workload configuration are harmonized through majority voting, as described in Section 5.2.1. Although the original datasets also contain measurements for individual functions, the prediction targets selected for this study concern the node as a whole.

5.1.4 Dataset Composition and Class Distribution

The statistics reported in this subsection refer to the datasets after initial data preparation and before allocation to the simulated nodes, train–validation–test splitting, and training-set oversampling. The Source Domain contains 98,732 observations and the Target Domain contains 67,573, for a total of 166,305. Table 5.1 reports the number of observations and the distribution of the overload label for each domain and node type.

The number of observations differs substantially across node types. As illustrated in Figure 5.1, Mid contributes the largest group in the Source Domain, whereas Heavy contributes the largest group in the Target Domain. Target Light is the smallest group overall. These differences reflect the available observations; the initial datasets are not modified to equalize the number of samples across node types.

Domain	Node type	Observations	Not overloaded	Overloaded	Overloaded (%)
Source	Heavy	19,423	17,938	1,485	7.6
Source	Mid	56,985	53,948	3,037	5.3
Source	Light	22,324	17,283	5,041	22.6
Source total		**98,732**	**89,169**	**9,563**	**9.7**
Target	Heavy	40,377	37,010	3,367	8.3
Target	Mid	17,192	15,608	1,584	9.2
Target	Light	10,004	6,865	3,139	31.4
Target total		**67,573**	**59,483**	**8,090**	**12.0**

Table 5.1. Number of prepared observations and overload-label distribution by domain and node type, before allocation to simulated nodes and training-set oversampling.

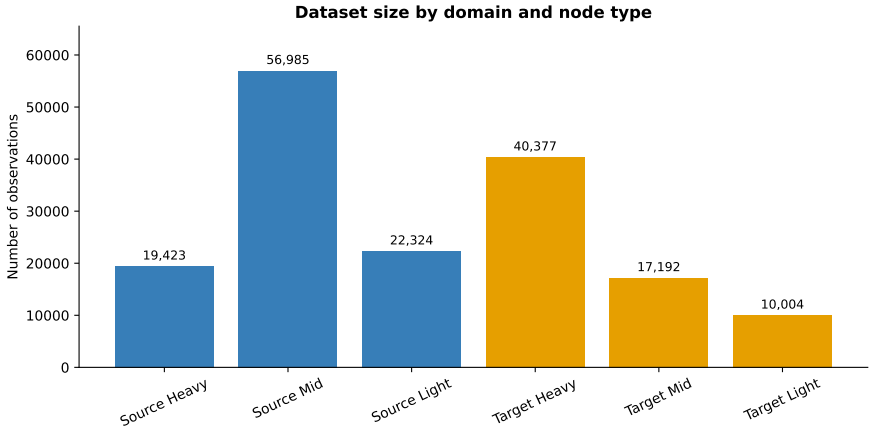


Figure 5.1. Number of prepared observations by domain and node type.

Overloaded observations form a minority in both domains: 9.7% of Source observations and 12.0% of Target observations. Figure 5.2 shows that the imbalance is not uniform across node types. Source Mid has the lowest proportion of overloaded observations, at 5.3%, while Target Light has the highest, at 31.4%. Source Light also has a comparatively high proportion, at 22.6%. These percentages describe the observed class distributions; on their own, they do not establish why the node types differ.

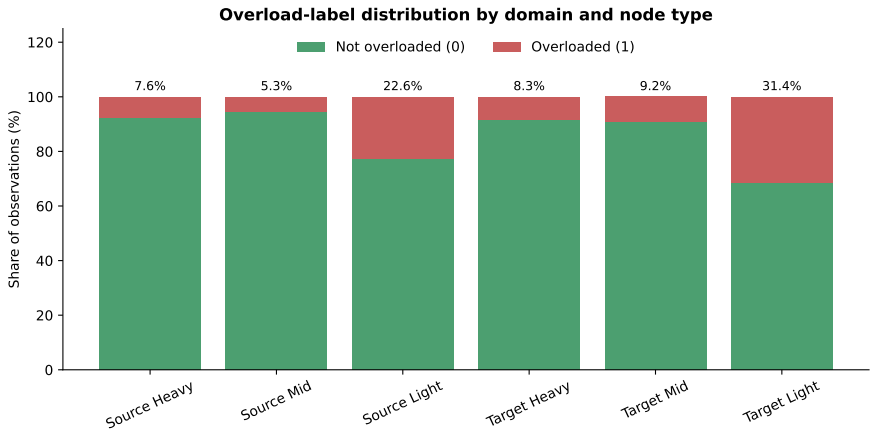


Figure 5.2. Percentage distribution of non-overloaded and overloaded observations by domain and node type. The value above each bar is the percentage of overloaded observations.

These statistics describe the prepared datasets before training-set overload balancing and therefore do not represent the class distributions of the final training partitions.

The domains also differ in the number of simultaneously active functions represented in their observations. Figure 5.3 counts a function as active when its request rate is greater than zero. Three functions are active in most observations in both domains. Conversely, observations with one or two active functions are relatively more frequent in Target. The available observations contain between one and three active functions, even though the function pool contains six functions. This is a characteristic of the experimental workloads rather than a class imbalance that requires correction; it also limits the workload compositions directly represented in the evaluation.

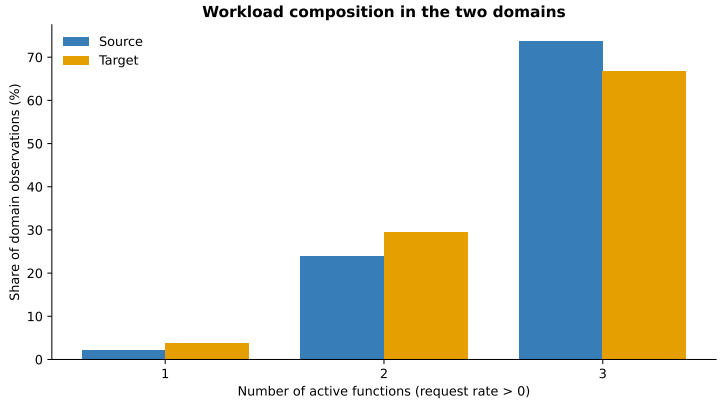


Figure 5.3. Distribution of the number of active functions per observation in the Source and Target Domains. A function is considered active when its request rate is greater than zero.

5.2 Data Preprocessing

The preprocessing pipeline converts the collected observations into datasets suitable for multi-task learning. Source and Target data are processed using a common representation while preserving their domain identity. After allocation to the network nodes, each local dataset is divided into training, validation, and test partitions. Overload balancing is subsequently applied only to the training partition.

5.2.1 Initial Cleaning and Domain Harmonization

The Source and Target datasets are harmonized using the input variables and prediction targets defined in Section 5.1.3. Columns not required by these prediction tasks are excluded from the model inputs and outputs.

A domain-specific encoding distinguishes the six node types. Source Heavy, Source Mid, and Source Light are assigned values 0, 1, and 2, respectively, while Target Heavy, Target Mid, and Target Light are assigned values 3, 4, and 5. Consequently, nodes belonging to different domains retain distinct identifiers even when they share the same hardware category.

Missing values in the selected columns are initially replaced with zero. Observations in which all function request rates are zero are removed, excluding idle workload configurations.

Repeated observations are grouped by their function request rates and node type. For each continuous regression target, values outside one standard deviation from the group mean are replaced with that mean. The overload label is replaced with the most frequent label within the group. These operations reduce variation among repeated measurements of the same workload configuration without removing the observations.

The resulting data share a common representation while retaining the distinction between Source and Target node types. Observations are then allocated to the network nodes according to the data-assignment procedure described previously.

5.2.2 Local Training, Validation, and Test Splits

Each node's dataset is partitioned separately after data allocation. Observations with identical input feature vectors are treated as a group, ensuring that repeated measurements of the same configuration within that node's dataset remain in the same partition. Stratification by the overload label is used to approximately preserve the class distribution.

For each node, the data are divided into a fixed training–test partition using a stratified group split, with approximately 20% of the observations assigned to the local test set. Observations belonging to the same group are kept together to prevent overlap between the two partitions. This partition is created once and reused throughout the experiments; cross-validation is not performed.

The remaining observations are divided into training and validation partitions through a stratified group shuffle split, assigning 20% of the remaining groups to validation. Before class balancing, the intended proportions are therefore approximately 64% training, 16% validation, and 20% testing. Actual sample proportions may differ because groups are kept intact. A fixed random seed of 42 is used for reproducibility.

After splitting, overload balancing is applied only to the training partition. Validation and test partitions remain unchanged. The resulting datasets are saved and reused across training configurations so that comparisons within the same network instance use the same local partitions.

5.2.3 Training-Set Overload Balancing

The local training partitions are balanced to reduce the predominance of non-overloaded observations. When overloaded observations are underrepresented, additional examples are generated from overloaded training samples containing three active functions. The request rate of one active function is increased while the other request rates and the node type remain unchanged. The generated examples retain the overloaded label, under the assumption that increasing the workload of an already overloaded configuration preserves its overload condition.

The additional examples are used to equalize the numbers of overloaded and non-overloaded training observations, repeating generated examples when necessary. The original training observations are retained. If no suitable examples can be generated, the partition remains unchanged. Validation and test partitions are not balanced, preserving their observed class distributions.

Since the generated examples do not have measured regression targets, these targets are left missing. They therefore contribute to the classification objective but are excluded from the masked regression loss. This preprocessing step differs from the synthetic-data generation performed by the proposed merging strategy, where received models provide predictions for additional training examples.

6
Experimental Results

6.1 Results with Existing Merging Strategies

This section presents the common-test results obtained with Overwrite, Improved Overwrite, Simple Averaging, and Age-Weighted Averaging. The experiments examine the effects of the number of gossip updates, the number of received models merged per update, and the weights assigned to the regression and classification losses.

Table 6.1 reports the node-type assignment in the evaluated nine-node network, which comprises three HEAVY, three MID, and three LIGHT nodes.

Node ID	Node Type	Node Type Name
0	0	HEAVY
1	2	LIGHT
2	0	HEAVY
3	0	HEAVY
4	1	MID
5	2	LIGHT
6	2	LIGHT
7	1	MID
8	1	MID

Table 6.1. Node IDs and corresponding node types in the evaluated nine-node network.

The results tables report accuracy, MSE, and overloaded-class F1-score as mean $\pm$ standard deviation across nodes within the evaluated simulation. The mean assigns equal importance to each node, while the standard deviation describes differences in performance among nodes. Accuracy and F1-score are expressed on a scale from 0 to 1. The analysis focuses on the observed differences between configurations; confusion matrices, prediction scatterplots, and the

overall comparison of the selected configurations are presented in a subsequent section.

6.1.1 Number of Gossip Updates

Table 6.2 compares the four strategies using 10, 20, 50, and 100 gossip updates per node. The *Updates* column indicates the number of completed local updates, each consisting of a merging operation followed by local training. Each update uses one training epoch, and one additional fine-tuning epoch is performed at the end of the gossip training process.

Merge Strategy	Updates	Accuracy	MSE	F1 (overloaded)
IMPROVED_OVERWRITE	10	0.6959 ± 0.2000	0.1886 ± 0.0907	0.2136 ± 0.0526
IMPROVED_OVERWRITE	20	0.6594 ± 0.2167	0.2008 ± 0.1079	0.1902 ± 0.0525
IMPROVED_OVERWRITE	50	0.6614 ± 0.2315	0.0957 ± 0.0432	0.2078 ± 0.0767
IMPROVED_OVERWRITE	100	0.6824 ± 0.2271	0.0724 ± 0.0211	0.2321 ± 0.0826
AGE_WEIGHTED	10	0.6101 ± 0.2705	0.2799 ± 0.1463	0.1750 ± 0.0549
AGE_WEIGHTED	20	0.6212 ± 0.2695	0.2618 ± 0.0820	0.1935 ± 0.0630
AGE_WEIGHTED	50	0.6433 ± 0.2522	0.1571 ± 0.0562	0.1997 ± 0.0736
AGE_WEIGHTED	100	0.6547 ± 0.2480	0.1018 ± 0.0200	0.2180 ± 0.0797
SIMPLE_AVG	10	0.6211 ± 0.2653	0.1550 ± 0.0998	0.1882 ± 0.0585
SIMPLE_AVG	20	0.6237 ± 0.2574	0.3389 ± 0.1455	0.1916 ± 0.0632
SIMPLE_AVG	50	0.6501 ± 0.2413	0.1522 ± 0.0466	0.2203 ± 0.0756
SIMPLE_AVG	100	0.6745 ± 0.2252	0.0819 ± 0.0183	0.2055 ± 0.0769
OVERWRITE	10	0.6720 ± 0.2148	0.0994 ± 0.0435	0.2359 ± 0.0790
OVERWRITE	20	0.6591 ± 0.2275	0.1154 ± 0.0504	0.2213 ± 0.0652
OVERWRITE	50	0.7012 ± 0.1647	0.1060 ± 0.0409	0.2292 ± 0.0612
OVERWRITE	100	0.7096 ± 0.1796	0.0897 ± 0.0453	0.2274 ± 0.0821

Table 6.2. Common-test performance for different merging strategies and numbers of updates. Values are reported as mean $\pm$ standard deviation across nodes.

Increasing the number of updates generally benefits mean regression performance, although the improvement is not monotonic. Comparing 10 and 100 updates, mean MSE decreases for all four strategies. At 100 updates, Improved Overwrite achieves an MSE of 0.0724 ± 0.0211, followed by Simple Averaging at 0.0819 ± 0.0183 and Overwrite at 0.0897 ± 0.0453. Overwrite therefore remains close to the other two strategies in terms of mean MSE, although its higher

standard deviation indicates greater variability in regression performance across nodes.

The classification results show a different pattern. Mean accuracy increases steadily with the update count for Age-Weighted Averaging and Simple Averaging, whereas the two overwrite strategies exhibit fluctuations. Overwrite reaches the highest mean accuracy in the table, 0.7096, at 100 updates, together with the lowest accuracy standard deviation among the four strategies at that update count (0.1796), indicating more consistent classification accuracy across nodes.

Overloaded-class F1 remains low across all configurations, ranging from approximately 0.175 to 0.236.

These results indicate that additional updates tend to improve mean classification accuracy and reduce mean MSE, although the improvements are not consistent across all configurations. Despite the reduction in regression error, classification performance remains limited, particularly in terms of overloaded-class F1-score.

Among the configurations evaluated with 100 updates, Overwrite appears to offer a slightly better overall trade-off. It achieves the highest mean accuracy and the lowest accuracy standard deviation, while its mean MSE remains close to those of Improved Overwrite and Simple Averaging. Its overloaded-class F1 is also close to that of Improved Overwrite, which obtains the highest value at this update count. On this basis, Overwrite with 100 updates is selected as the preferred configuration in this comparison, although its advantage is modest and does not extend to every metric.

6.1.1.1 Detailed Analysis of Overwrite with 100 Updates

Following the comparison of update counts, Overwrite with 100 updates was selected for a more detailed analysis. Table 6.3 reports the common-test accuracy and MSE of each node, showing substantial differences in predictive performance across the network.

Node	Accuracy	MSE
0	0.9104	0.1506
1	0.4324	0.0710
2	0.8765	0.0837
3	0.8744	0.1907
4	0.6648	0.0523
5	0.4132	0.0749
6	0.6091	0.0542
7	0.7808	0.0711
8	0.8248	0.0584

Table 6.3. Per-node common-test performance of Overwrite with 100 updates.

Regression performance varies substantially across nodes, with MSE ranging from 0.0523 for node 4 to 0.1907 for node 3. Nodes 0 and 3 exhibit the highest MSE values, 0.1506 and 0.1907, respectively, despite achieving high classification accuracies of 0.9104 and 0.8744. This highlights that network-wide averages can conceal substantial differences in node-level performance.

Nodes 8 and 5 are examined to illustrate contrasting classification outcomes, with common-test accuracies of approximately 82.48% and 41.32%, respectively. These cases highlight the variability across nodes rather than representing the average behaviour of the network.

The following figures report classification results for the local training, validation, and test partitions, followed by the common test set, from left to right. The common test set is constructed by randomly selecting approximately 10% of the observations from each node's local test partition and pooling them into a single dataset. These observations remain in their original local test partitions: the common test set therefore combines existing test samples rather than introducing additional data. Within each simulation, all node models are evaluated on this same dataset, which is not used to update model parameters. For each model, it contains test observations from both its own node and the other nodes, allowing classification performance across the network's data distributions to be assessed.

The upper panels show confusion matrices, with true classes on the rows and predicted classes on the columns. Class 0 denotes non-overloaded observations, while class 1 denotes overloaded observations. Diagonal entries represent correct predictions, and off-diagonal entries represent false positives and false negatives. The lower panels display the same outcomes as proportional diagrams: column widths reflect the frequencies of the true classes, while the blue and orange

regions indicate the proportions predicted as non-overloaded and overloaded, respectively.

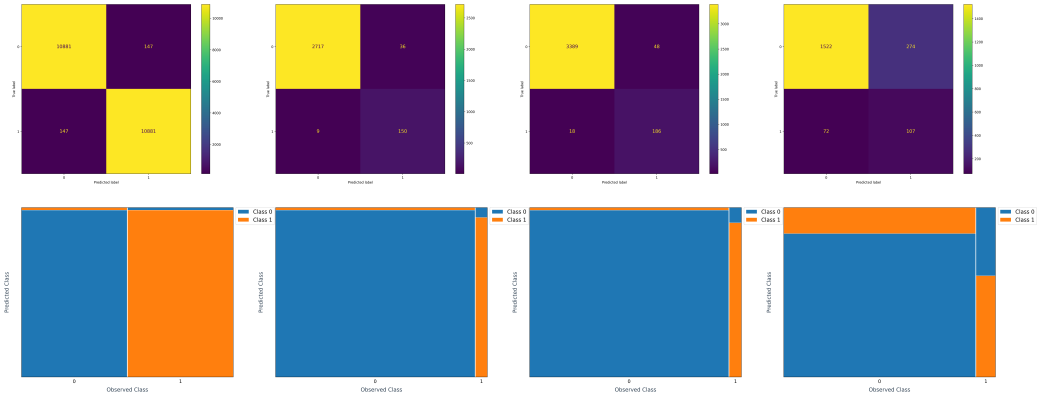


Figure 6.1. Classification results for node 8 using Overwrite with 100 updates. Columns show local training, local validation, local test, and common-test results, respectively. Confusion matrices appear above the corresponding proportional diagrams.

Figure 6.1 shows that node 8 performs well on its local data, achieving approximately 98.67% training accuracy and 98.19% local test accuracy. Performance decreases on the common test set, where the model correctly classifies 1,522 non-overloaded observations and 107 overloaded observations, but produces 274 false positives and 72 false negatives. Its common-test accuracy is relatively high compared with several other nodes, but its overloaded-class F1-score is approximately 0.382. Therefore, the accuracy value does not imply equally effective detection of overload conditions.

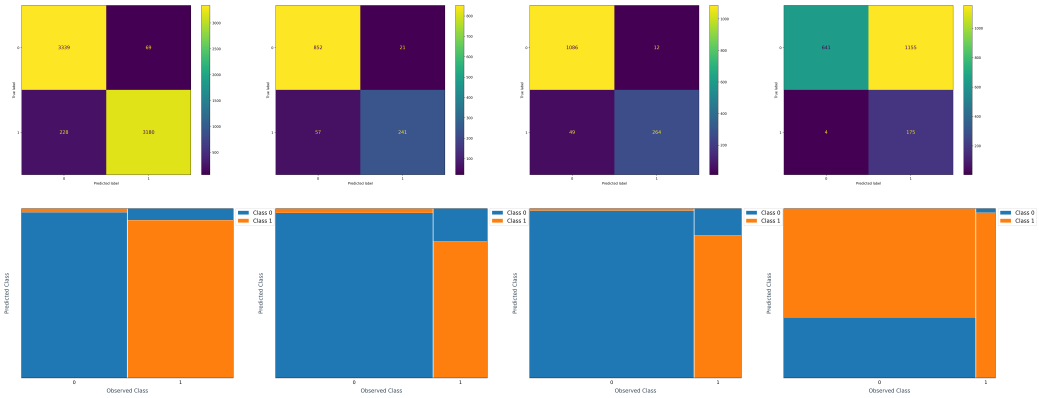


Figure 6.2. Classification results for node 5 using Overwrite with 100 updates. Columns show local training, local validation, local test, and common-test results, respectively. Confusion matrices appear above the corresponding proportional diagrams.

Figure 6.2 illustrates a more pronounced generalization limitation. Node 5 achieves approximately 95.64% training accuracy and 95.68% local test accuracy.

On the common test set, however, it incorrectly classifies 1,155 of the 1,796 non-overloaded observations as overloaded. Although it detects 175 of the 179 actual overloads, the large number of false alarms reduces accuracy to 41.32% and overloaded-class F1 to approximately 0.232. The model therefore tends to predict overload excessively when evaluated on the broader set of observations represented in the common test.

These examples show that good classification performance on local data does not necessarily translate into effective generalization across the network. Since both nodes already achieve high accuracy on their local training and test partitions, increasing the number of epochs per update was not prioritised in subsequent experiments. Instead, attention was directed towards improving how information from other nodes is incorporated.

6.1.2 Task Loss Weights

The per-head loss curves show that, after the initial training phase, the classification loss remains numerically larger than the regression loss. This observation motivated an experiment increasing the regression-loss coefficient from 5 to 10, to investigate whether assigning greater relative importance to regression would improve predictive performance.

Table 6.4 compares two loss-weight configurations with 20 updates per node. The multi-task objective adopted from Khan combines masked mean squared error for regression and binary crossentropy for overload classification:

$$\mathcal{L} = \lambda_{\text{reg}}\mathcal{L}_{\text{reg}} + \lambda_{\text{cls}}\mathcal{L}_{\text{cls}}. \tag{6.1}$$

The reference configuration uses $\lambda_{\text{reg}} = 5$ and $\lambda_{\text{cls}} = 1$. The *Loss weights* column reports these coefficients as $\lambda_{\text{reg}} : \lambda_{\text{cls}}$. The experiment increases the regression coefficient from 5 to 10 while retaining the classification coefficient at 1, thereby increasing the relative contribution of regression errors to the training objective.

Merge Strategy	Updates	Loss weights $\lambda_{\text{reg}} : \lambda_{\text{cls}}$	Accuracy	MSE	F1 (overloaded)
IMPROVED_OVERWRITE	20	$5:1$	0.6594 ± 0.2167	0.2008 ± 0.1079	0.19026 ± 0.0525
IMPROVED_OVERWRITE	20	$10:1$	0.6663 ± 0.2100	0.2177 ± 0.12081	0.1895 ± 0.0506
AGE_WEIGHTED	20	$5:1$	0.6212 ± 0.2695	0.2618 ± 0.0820	0.1935 ± 0.0630
AGE_WEIGHTED	20	$10:1$	0.6178 ± 0.2698	0.2807 ± 0.0982	0.1928 ± 0.0668
SIMPLE_AVG	20	$5:1$	0.6237 ± 0.2574	0.3389 ± 0.1455	0.1916 ± 0.0632
SIMPLE_AVG	20	$10:1$	0.6289 ± 0.2574	0.3449 ± 0.1438	0.1943 ± 0.0617
OVERWRITE	20	$5:1$	0.6591 ± 0.2275	0.1154 ± 0.0504	0.2213 ± 0.0652
OVERWRITE	20	$10:1$	0.6536 ± 0.2310	0.1262 ± 0.0417	0.2007 ± 0.06262

Table 6.4. Performance for different regression and classification loss weights. Values are reported as mean $\pm$ standard deviation across nodes.

Increasing the regression-loss weight does not improve mean MSE for any of the four strategies. For Improved Overwrite, MSE increases from 0.2008 to 0.2177; for Age-Weighted Averaging, from 0.2618 to 0.2807; for Simple Averaging, from 0.3389 to 0.3449; and for Overwrite, from 0.1154 to 0.1262. Assigning a larger coefficient to the regression objective therefore does not translate into lower common-test regression error in these experiments.

Changes in classification performance are comparatively small. Mean accuracy increases slightly for Improved Overwrite and Simple Averaging and decreases slightly for Age-Weighted Averaging and Overwrite. Overloaded-class F1 also shows no consistent benefit: it increases slightly only for Simple Averaging, while decreasing for the other strategies.

The comparison did not reveal a substantial overall improvement from increasing the regression-loss weight to $10:1$. Changes in classification performance were small, while mean MSE increased for all four strategies. The $5:1$ configuration was therefore retained for subsequent experiments, following the weighting adopted by Khan [5] for the centrally trained multi-task model.

6.1.3 Number of Merged Models

Table 6.5 compares Age-Weighted Averaging and Simple Averaging using different numbers of received models per update. The *Merged models* column counts models received from distinct neighbours, excluding the local model, which also participates in the averaging operation. A value of 1 therefore combines two models in total, while a value of 8 combines the local model with eight received models. Overwrite and Improved Overwrite are not included because their implementation accepts one received model per update. All experiments in

this comparison use 50 updates per node, one training epoch per update, and one final fine-tuning epoch.

Merge Strategy	Merged models	Accuracy	MSE	F1 (overloaded)
AGE_WEIGHTED	1	0.6433 ± 0.2522	0.1571 ± 0.0562	0.1997 ± 0.0736
AGE_WEIGHTED	2	0.6555 ± 0.2354	0.1238 ± 0.0368	0.2143 ± 0.0637
AGE_WEIGHTED	4	0.7416 ± 0.1480	0.1033 ± 0.0214	0.2384 ± 0.0711
AGE_WEIGHTED	8	0.8123 ± 0.0812	0.0576 ± 0.0093	0.2948 ± 0.1079
SIMPLE_AVG	1	0.6501 ± 0.2413	0.1522 ± 0.0466	0.2203 ± 0.0756
SIMPLE_AVG	2	0.6819 ± 0.2106	0.1376 ± 0.0499	0.2310 ± 0.0673
SIMPLE_AVG	4	0.7167 ± 0.1700	0.0867 ± 0.0175	0.2216 ± 0.0846
SIMPLE_AVG	8	0.7836 ± 0.1123	0.0657 ± 0.0120	0.2723 ± 0.0851

Table 6.5. Performance for different numbers of received models merged per update. Values are reported as mean $\pm$ standard deviation across nodes.

As the number of received models per update increases, mean accuracy improves and mean MSE decreases for both strategies in the evaluated configurations.

For Age-Weighted Averaging, accuracy increases from 0.6433 to 0.8123 as the number of received models increases from one to eight, while MSE decreases from 0.1571 to 0.0576. Simple Averaging shows the same overall trend, with accuracy increasing from 0.6501 to 0.7836 and MSE decreasing from 0.1522 to 0.0657.

The standard deviations of accuracy and MSE also decrease for both strategies, indicating that these metrics become more consistent across nodes. This reduction does not extend to overloaded-class F1, whose standard deviation is larger with eight received models than with one.

The mean overloaded-class F1 increases monotonically for Age-Weighted Averaging, reaching 0.2948 with eight received models. For Simple Averaging, the trend is less regular, but the highest value, 0.2723, is also obtained with eight received models. At this setting, Age-Weighted Averaging achieves better mean values for all three metrics than Simple Averaging.

Despite the improvements, overload detection remains a limitation: the improvement in accuracy is accompanied by an F1-score that remains below 0.30.

These observations support the usefulness of combining more received models in the evaluated setting. The configuration with eight received models represents a particular case: since the network contains nine fully connected nodes, each node must collect a model from every other node before starting an update. However, improvements in mean accuracy and MSE have already been observed with four received models, showing that these benefits are not restricted to collecting

contributions from the entire neighborhood. However, requiring more received models per update can increase the waiting time before local training begins.

6.1.4 Effect of Longer Training and Motivation for Node-Type-Aware Learning

The preceding experiments identified Age-Weighted Averaging with eight received models merged per update as a promising reference configuration. Since this configuration had initially been evaluated with 50 gossip updates per node, an additional experiment increased the number of updates to 100 to examine whether longer training could improve common-test generalization. Table 6.6 compares the two settings.

Updates	Accuracy	MSE	F1 (overloaded)
50	0.8123 ± 0.0812	0.0576 ± 0.0093	0.2948 ± 0.1079
100	0.8267 ± 0.0992	0.0390 ± 0.0103	0.3770 ± 0.0833

Table 6.6. Common-test performance of Age-Weighted Averaging with eight received models merged per update, comparing 50 and 100 gossip updates per node. Values are reported as mean $\pm$ standard deviation across nodes.

Increasing the number of gossip updates from 50 to 100 improves all three mean performance metrics. Accuracy increases from 0.8123 to 0.8267, while MSE decreases from 0.0576 to 0.0390, corresponding to a reduction of approximately 32.3%. The overloaded-class F1-score also improves, rising from 0.2948 to 0.3770. Longer training therefore provides benefits, particularly for regression, but overload detection remains limited. These results motivated investigating a complementary mechanism for incorporating information from different node types, rather than relying exclusively on additional updates with the existing parameter-merging procedure.

Although node type is included among the model inputs, it is constant within each node's original local dataset. Gossip exchanges allow the model to incorporate knowledge learned by other nodes, including relationships between workloads, resource usage, and overload conditions associated with different node types. The model is therefore not restricted to the knowledge acquired from its own observations. However, each subsequent local training update uses examples from only the receiving node's type, providing no direct variation in node type from which to learn its role as a covariate.

The proposed `node_type_merge` strategy addresses this potential limitation by introducing explicit training observations associated with remote node types. It preserves locally available workload features, replaces the node-type feature with the sender's type, and uses the received model to generate corresponding synthetic targets. Subsequent local optimization can therefore include observations associated with multiple node types without exchanging raw data. These targets remain model-generated estimates, so their usefulness depends on the information provided by the received models. The following section evaluates whether this mechanism improves generalization beyond the receiving node's original data distribution.

6.2 Results with the Proposed Node-Type-Aware Strategy

The results obtained with the previous merging strategies revealed limitations in common-test generalization, particularly in overload detection, despite good classification performance on local data.

These findings motivated the development of `node_type_merge`, which uses the sender's node type, transmitted alongside its model parameters, to guide the incorporation of remote knowledge.

The main experiments with the proposed strategy extend the evaluation from nine to eighteen fully connected nodes and include both Source and Target Domain data, whereas the earlier experiments considered only the Source Domain. This setting increases the amount of available data and introduces additional node types, allowing the proposed approach to be assessed under greater data heterogeneity.

Table 6.7 reports the node-type assignment in the evaluated eighteen-node network. Each domain contributes nine nodes, with three HEAVY, three MID, and three LIGHT nodes per domain.

Node ID	Node Type	Node Type Name
0	1	SOURCE_MID
1	5	TARGET_LIGHT
2	2	SOURCE_LIGHT
3	0	SOURCE_HEAVY
4	2	SOURCE_LIGHT
5	3	TARGET_HEAVY
6	4	TARGET_MID
7	0	SOURCE_HEAVY
8	5	TARGET_LIGHT
9	0	SOURCE_HEAVY
10	4	TARGET_MID
11	1	SOURCE_MID
12	5	TARGET_LIGHT
13	4	TARGET_MID
14	3	TARGET_HEAVY
15	2	SOURCE_LIGHT
16	3	TARGET_HEAVY
17	1	SOURCE_MID

Table 6.7. Node IDs and corresponding node types in the evaluated eighteen-node network.

As described in Section 4.4.2, the proposed strategy combines two components: the generation and management of synthetic training observations, and the rule used to update the receiving node's model parameters. Models received from nodes of different types are used locally to generate synthetic targets for inputs associated with those remote node types. This extends the receiving node's training dataset beyond its own node type without exchanging raw observations.

The first subsection presents the experiments with `keep_local` and `overwrite`, which respectively preserve the local parameters or replace them with the received parameters before local training. These experiments examine how the two update rules interact with synthetic-data management. The second subsection evaluates interpolation, introduced subsequently to allow a configurable combination of local and received parameters through the coefficient α.

6.2.1 Results with Keep Local and Overwrite

The experiments compare `keep_local` and `overwrite` with three synthetic-data strategies: *aggregate*, *substitution*, and *concatenate*. Table 6.8 reports their common-test performance. Values are reported as mean ± standard deviation across nodes.

Model-update rule	Synthetic-data strategy	Accuracy	MSE	F1 (overloaded)
OVERWRITE	AGGREGATE	0.8643 ± 0.0541	0.1163 ± 0.0680	0.5442 ± 0.0804
OVERWRITE	SUBSTITUTION	0.8684 ± 0.0426	0.0490 ± 0.0566	0.5268 ± 0.0626
OVERWRITE	CONCATENATE	0.8449 ± 0.0504	0.1040 ± 0.0627	0.5196 ± 0.0656
KEEP_LOCAL	AGGREGATE	0.8162 ± 0.1291	0.0299 ± 0.0171	0.5052 ± 0.1079
KEEP_LOCAL	SUBSTITUTION	0.7959 ± 0.1236	0.0369 ± 0.0184	0.4686 ± 0.0953
KEEP_LOCAL	CONCATENATE	0.8028 ± 0.1130	0.0291 ± 0.0144	0.4854 ± 0.1061

Table 6.8. Common-test performance of the proposed node-type-aware strategy in the eighteen-node setting. Values are reported as mean ± standard deviation across nodes.

Table 6.8 reveals a trade-off between the two model-update rules. Overwrite achieves higher mean accuracy and overloaded-class F1-score than Keep Local for every synthetic-data strategy. Its mean accuracy ranges from 0.8449 to 0.8684, with standard deviations between 0.0426 and 0.0541, indicating more consistent accuracy across nodes. Substitution achieves the highest mean accuracy, whereas Aggregate obtains the highest mean overloaded-class F1-score, at 0.5442. Nevertheless, the differences in mean accuracy and F1-score between Substitution and Aggregate are small, indicating similar average classification performance in this experiment.

Keep Local provides better regression performance, with mean MSE values between 0.0291 and 0.0369. However, its classification performance is weaker and more variable across nodes. Mean accuracy ranges from 0.7959 to 0.8162, with standard deviations between 0.1130 and 0.1291, substantially higher than those observed with Overwrite.

Conversely, Overwrite produces higher regression errors than Keep Local, particularly with Aggregate and Concatenate, whose mean MSE values are 0.1163 and 0.1040, respectively. Substitution reduces the mean MSE to 0.0490, although this remains above the values obtained by all three Keep Local configurations. Its MSE standard deviation of 0.0566 also indicates substantial variation in regression performance across nodes. Overwrite therefore offers stronger and more consistent classification, while Keep Local achieves lower regression error.

The classification results are encouraging compared with the previously evaluated existing merging strategies, particularly in terms of overloaded-class F1-score, despite the larger network and the inclusion of additional heterogeneous data. To complement this comparison, an additional experiment evaluated Overwrite with Substitution using the earlier nine-node setting, with only the source dataset. Its common-test results are reported in Table 6.9.

Accuracy	**MSE**	**F1 (overloaded)**
0.8657 ± 0.0359	0.0222 ± 0.0066	0.5290 ± 0.0558

Table 6.9. Common-test performance of the proposed strategy using Overwrite with Substitution in the nine-node setting. Values are reported as mean $\pm$ standard deviation across nodes.

The nine-node experiment shows that improved performance was also observed without increasing the network size or expanding the dataset, supporting the contribution of the proposed strategy to the gains. The mean accuracy and F1-score are also close to those obtained by Overwrite with Substitution in the eighteen-node experiment, while its mean MSE and the standard deviations of all three metrics are lower.

For this configuration, mean accuracy and F1-score remain similar across the two evaluated settings, whereas the eighteen-node experiment exhibits a higher mean MSE and greater variation across nodes. This comparison supports the applicability of the approach to the larger setting.

Regarding synthetic-data management, Concatenate provides no consistent predictive advantage over the alternatives. With Overwrite, it produces the lowest mean accuracy and F1-score, while Substitution achieves a lower mean MSE. With Keep Local, Concatenate obtains the lowest mean MSE, but the difference from Aggregate is small, at 0.0291 compared with 0.0299, and Aggregate provides higher mean accuracy and F1-score. In addition, Concatenate required more computational resources and longer execution times in the preceding experiments. Given these costs and the absence of a consistent performance advantage, it was excluded from the final comparison, which focuses on *aggregate* and *substitution*.

6.2.1.1 Node-Level Analysis of Keep Local and Overwrite

The aggregate results conceal substantial differences between individual nodes. Tables 6.10 and 6.11 report common-test performance for each node using Keep Local and Overwrite with Aggregate, respectively. This analysis examines whether the advantages observed at the network level are shared across nodes and identifies the limitations that motivated the subsequent interpolation experiments.

Node	MSE	Accuracy	F1 Overloaded
0	0.0295	0.8556	0.4678
1	0.0238	0.7741	0.4490
2	0.0215	0.8054	0.4631
3	0.0484	0.6767	0.3344
4	0.0930	0.3480	0.2409
5	0.0184	0.8875	0.6063
6	0.0198	0.9236	0.6482
7	0.0294	0.7606	0.4480
8	0.0224	0.8914	0.5516
9	0.0369	0.7711	0.4105
10	0.0198	0.8701	0.4918
11	0.0186	0.8905	0.6240
12	0.0264	0.8529	0.4373
13	0.0281	0.8854	0.5934
14	0.0181	0.8496	0.5544
15	0.0349	0.8550	0.5218
16	0.0215	0.8884	0.6041
17	0.0279	0.9056	0.6472

Table 6.10. Per-node common-test performance of Keep Local with Aggregate.

Node	MSE	Accuracy	F1 Overloaded
0	0.0346	0.8496	0.5088
1	0.1488	0.8418	0.5112
2	0.0634	0.8725	0.5656
3	0.1459	0.9155	0.6474
4	0.1291	0.8656	0.5093
5	0.1190	0.8421	0.4817
6	0.0532	0.9050	0.5587
7	0.0502	0.8108	0.4728
8	0.1466	0.8033	0.4692
9	0.0781	0.8716	0.5344
10	0.1238	0.9053	0.6246
11	0.0517	0.9161	0.6508
12	0.0526	0.9083	0.6075
13	0.3264	0.6875	0.3087
14	0.0976	0.8833	0.5792
15	0.1394	0.8938	0.5647
16	0.2031	0.8860	0.5756
17	0.1295	0.8998	0.6263

Table 6.11. Per-node common-test performance of Overwrite with Aggregate.

Keep Local achieves lower MSE than Overwrite at every node in this comparison, but its classification performance is uneven. Node 4 is the most problematic case, with an accuracy of 0.3480 and an overloaded-class F1-score of 0.2409. By comparison, several other nodes exceed 0.88 accuracy. Node 4 also has the highest MSE within the Keep Local configuration, at 0.0930, indicating that its difficulties are not confined to classification.

The following figures show results for the local training, validation, and test partitions, followed by the common test set, from left to right. The upper panels contain confusion matrices, with true classes on the rows and predicted classes on the columns. Class 0 represents non-overloaded observations and class 1 represents overloaded observations. The lower panels display the same outcomes as proportional diagrams, showing the distribution of predictions within each true class.

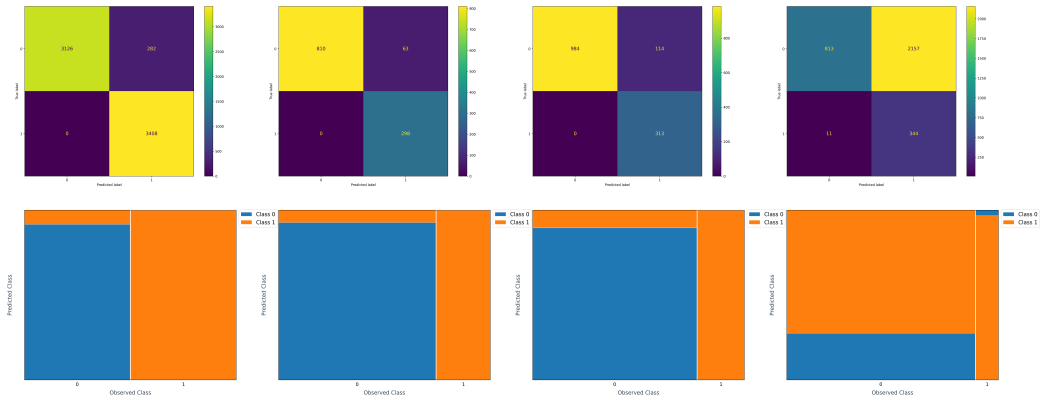


Figure 6.3. Classification results for Node 4 using Keep Local with Aggregate. Columns show local training, validation, local test, and common-test results, respectively.

Figure 6.3 shows that Node 4 correctly identifies all overloaded observations in its local training, validation, and test partitions. However, this behaviour does not transfer effectively to the common test set. Although 344 of the 355 overloaded observations are correctly detected, 2,157 of the 2,970 non-overloaded observations are incorrectly classified as overloaded. Its low common-test accuracy therefore results primarily from excessive false positives rather than a failure to detect overload conditions.

Node 4 belongs to the SOURCE LIGHT category (Table 6.7). Its high overload recall and large number of false positives on the common test set are consistent with a decision boundary that remains strongly influenced by its local node type. One possible interpretation is that conditions learned as indicative of overload for SOURCE LIGHT observations are also classified as overloaded when associated with other node types. Under Keep Local, remote knowledge is incorporated through synthetic training observations while local parameters are preserved during merging; in this case, this mechanism may not have sufficiently corrected the model's local specialization. However, node type alone does not explain the outcome: Nodes 2 and 15 also belong to SOURCE LIGHT but achieve common-test accuracies of 0.8054 and 0.8550, respectively.

To further examine this behaviour, Figure 6.4 reports the proportion of overloaded observations in the datasets used for successive gossip updates.

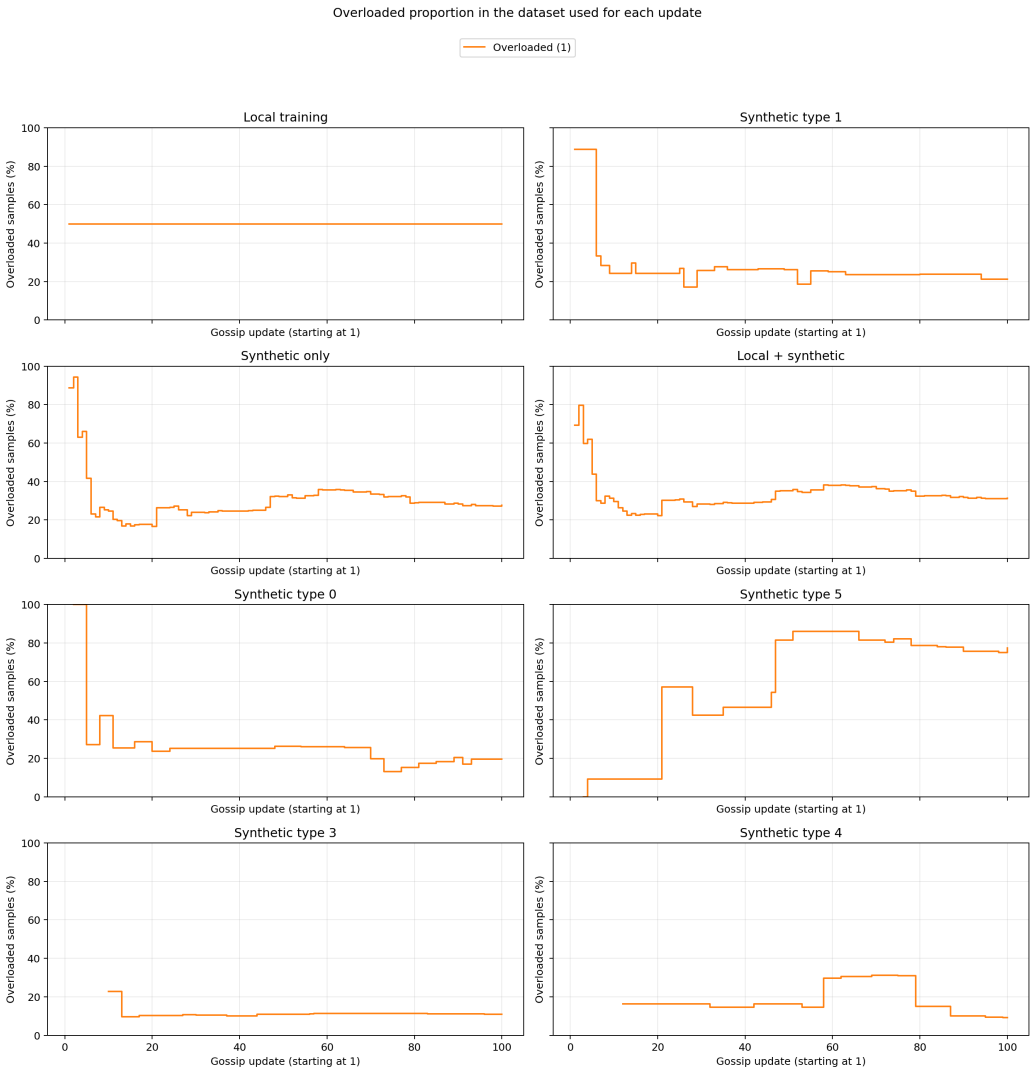


Figure 6.4. Percentage of overloaded observations in the datasets used to train Node 4 with Keep Local and Aggregate. Panels show the local training data, individual synthetic node types, the combined synthetic data, and the complete local-plus-synthetic dataset.

The local training dataset remains balanced, with 50% overloaded observations throughout the experiment. The synthetic datasets exhibit different class proportions across node types and updates. Following the initial transient, overloaded observations constitute a minority of both the combined synthetic dataset and the complete local-plus-synthetic dataset. The extensive overload predictions on the common test set therefore cannot be attributed simply to a persistent majority of overloaded training targets.

These observations suggest that the limitation concerns generalization across node types rather than class proportions alone. However, the figure describes the distribution of training targets, not their correctness or the extent to which the model learns from them. Establishing whether Node 4 retains a bias towards its

local type would require common-test errors to be examined separately for each node type.

Overwrite produces more consistent common-test accuracy across nodes in this experiment. In particular, Node 4 reaches an accuracy of 0.8656 and an F1-score of 0.5093, compared with 0.3480 and 0.2409 under Keep Local. However, this classification improvement is accompanied by an increase in its MSE from 0.0930 to 0.1291.

Node 11 illustrates one of the strongest classification outcomes with Overwrite, achieving the highest accuracy and overloaded-class F1-score in this configuration, at 0.9161 and 0.6508, respectively.

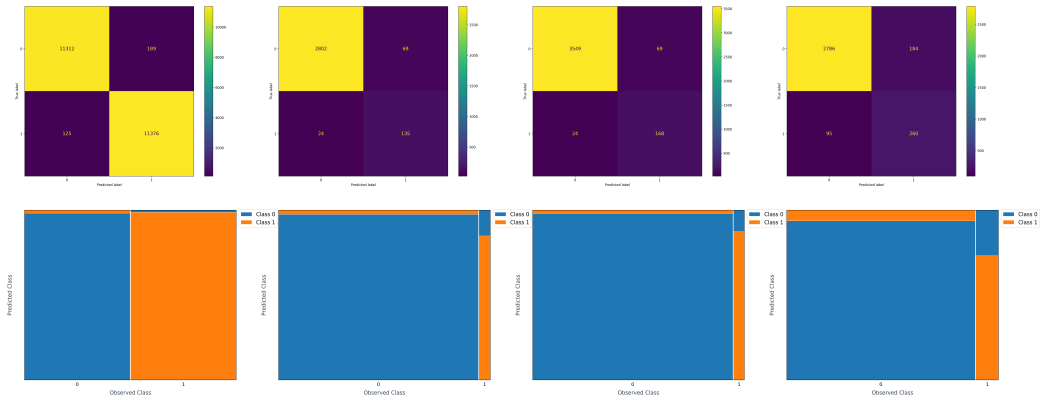


Figure 6.5. Classification results for Node 11 using Overwrite with Aggregate. Columns show local training, validation, local test, and common-test results, respectively.

On the common test set, Node 11 correctly classifies 2,786 non-overloaded and 260 overloaded observations, with 184 false positives and 95 false negatives. Compared with Node 4 under Keep Local, it avoids the extensive misclassification of non-overloaded observations. Nevertheless, the remaining false negatives and false positives show that overload detection is still imperfect despite the high overall accuracy.

Node 13 represents the weakest classification result within the Overwrite configuration, with an accuracy of 0.6875 and an F1-score of 0.3087. It also has the highest MSE, at 0.3264.

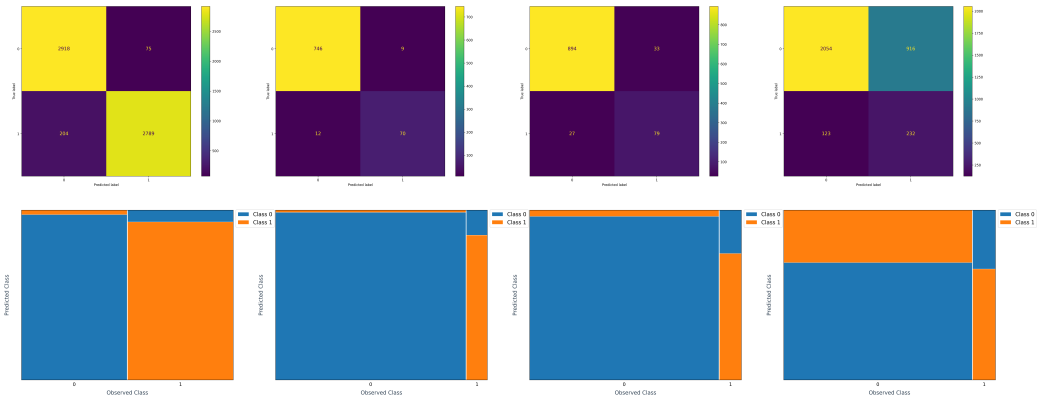


Figure 6.6. Classification results for Node 13 using Overwrite with Aggregate. Columns show local training, validation, local test, and common-test results, respectively.

Despite relatively good classification on its local partitions, Node 13 produces 916 false positives and 123 false negatives on the common test set. Its generalization therefore remains limited. However, this is a less extreme deterioration than that observed for Node 4 under Keep Local, which produces 2,157 false positives. Overwrite consequently reduces the severity of the weakest classification outcome in this comparison, without eliminating differences in performance across nodes.

The regression results reveal the main limitation of Overwrite. Its MSE is higher than that of Keep Local for every node in the comparison, with particularly large errors at Nodes 13 and 16, where MSE reaches 0.3264 and 0.2031, respectively. The classification benefits of incorporating received parameters are therefore accompanied by poorer regression performance.

These findings motivated the introduction of Interpolate as an intermediate model-update rule. Keep Local preserves the local parameters during merging and incorporates remote knowledge through synthetic training observations, whereas Overwrite replaces the entire parameter vector with the received one. Interpolate retains a contribution from the local model while incorporating received parameters through the coefficient α. The subsequent experiments investigate whether this intermediate approach can improve the balance between classification generalization and regression performance.

6.2.2 Results with Interpolation

The final experiments evaluate interpolation as an intermediate model-update rule between Keep Local and Overwrite. The aim is to retain useful local parameters while incorporating information from received models, addressing the

trade-off observed in the preceding experiments. The comparison considers Aggregate and Substitution, following the exclusion of Concatenate.

The interpolation coefficient is set to $\alpha = 0.3$, so the parameters used to begin each local training update combine 70% of the local parameters with 30% of the received parameters. This configuration preserves a larger contribution from the local model while incorporating information from the received model.

Model-update rule	Synthetic-data strategy	Accuracy	MSE	F1 (overloaded)
INTERPOLATE	AGGREGATE	0.9054 ± 0.0165	0.0283 ± 0.0082	0.6330 ± 0.0331
INTERPOLATE	SUBSTITUTION	0.8950 ± 0.0167	0.02346 ± 0.0027	0.6147 ± 0.0388

Table 6.12. Common-test performance of interpolation with $\alpha = 0.3$ using Aggregate and Substitution. Values are reported as mean $\pm$ standard deviation across nodes.

Table 6.12 shows the strongest overall results among the evaluated configurations in the eighteen-node setting. Both interpolation configurations achieve higher mean accuracy and overloaded-class F1-score, together with lower mean MSE, than the preceding Keep Local and Overwrite configurations. Interpolation therefore provides a better observed balance between classification and regression.

For Aggregate, mean accuracy increases from 0.8643 with Overwrite to 0.9054 with interpolation, while the overloaded-class F1-score rises from 0.5442 to 0.6330. Mean MSE decreases from 0.1163 to 0.0283, also providing a small improvement over the value of 0.0299 obtained with Keep Local. Substitution exhibits the same pattern: interpolation achieves an accuracy of 0.8950, an F1-score of 0.6147, and an MSE of 0.02346, improving all three mean metrics relative to the corresponding Overwrite and Keep Local configurations.

The improvements also extend to consistency across nodes. The gains in average performance are therefore accompanied by reduced variation between nodes, addressing an important limitation identified in the preceding detailed analysis.

The comparison between the two synthetic-data strategies reveals a moderate trade-off. Aggregate achieves the highest mean accuracy and overloaded-class F1-score, exceeding Substitution by 0.0104 and 0.0183, respectively. It also produces slightly lower standard deviations in both classification metrics. Substitution, however, achieves a lower mean MSE, at 0.02346 compared with 0.0283, together with a smaller MSE standard deviation of 0.0027 compared with 0.0082. Its regression predictions are therefore more accurate on average and its regression performance is more consistent across nodes.

Both synthetic-data strategies appear to be effective options when combined with interpolation. Aggregate provides the strongest observed classification results, while Substitution offers an advantage in regression accuracy and consistency.

Overall, interpolation with $\alpha = 0.3$ provides the most favourable observed balance of predictive performance and consistency across nodes in the evaluated eighteen-node experiments.

6.2.2.1 Node-Level Analysis of Interpolation

Tables 6.13 and 6.14 report the common-test performance of individual nodes using Interpolate with Aggregate and Substitution, respectively. This analysis examines the variability across nodes and complements the aggregate comparison with examples of regression predictions and training-class proportions.

Node	MSE	Acc.	F1	Node	MSE	Acc.	F1
0	0.0261	0.8577	0.5340	9	0.0295	0.9185	0.6503
1	0.0322	0.9053	0.6542	10	0.0217	0.9149	0.6458
2	0.0286	0.8941	0.6157	11	0.0286	0.9029	0.6342
3	0.0297	0.9179	0.6276	12	0.0261	0.9212	0.6450
4	0.0371	0.8761	0.5813	13	0.0208	0.9065	0.6454
5	0.0233	0.8938	0.6217	14	0.0236	0.9251	0.6876
6	0.0584	0.9113	0.6509	15	0.0279	0.9113	0.6093
7	0.0220	0.9056	0.6288	16	0.0232	0.8995	0.6557
8	0.0253	0.9128	0.6384	17	0.0259	0.9236	0.6684

Table 6.13. Per-node common-test performance of Interpolate with Aggregate in the evaluated simulation.

Node	MSE	Acc.	F1	Node	MSE	Acc.	F1
0	0.0289	0.8487	0.5149	9	0.0263	0.8992	0.6189
1	0.0228	0.9029	0.6204	10	0.0210	0.9050	0.6433
2	0.0201	0.8851	0.5919	11	0.0199	0.9056	0.6624
3	0.0287	0.9071	0.6386	12	0.0244	0.8875	0.5760
4	0.0229	0.8617	0.5818	13	0.0247	0.9095	0.6480
5	0.0227	0.8995	0.5707	14	0.0228	0.9077	0.6367
6	0.0228	0.8872	0.5410	15	0.0245	0.9143	0.5899
7	0.0264	0.9038	0.6226	16	0.0203	0.9023	0.6494
8	0.0200	0.8809	0.5892	17	0.0231	0.9029	0.6249

Table 6.14. Per-node common-test performance of Interpolate with Substitution in the evaluated simulation.

With Aggregate, accuracy ranges from 0.8577 to 0.9251 and overloaded-class F1-score from 0.5340 to 0.6876. Node 14 achieves the highest values for both classification metrics, whereas Node 0 has the lowest. Regression performance remains less uniform: Node 6 has the highest MSE, at 0.0584, despite an accuracy of 0.9113 and an F1-score of 0.6509. This illustrates that good classification performance does not necessarily imply equally good regression predictions.

With Substitution, accuracy ranges from 0.8487 to 0.9143 and overloaded-class F1-score from 0.5149 to 0.6624. Node 15 achieves the highest accuracy, while Node 11 achieves the highest overloaded-class F1-score and the lowest MSE, at 0.0199. The MSE values occupy a narrower range than in the Aggregate experiment, with a maximum of 0.0289.

Substitution provides better and more uniform regression performance in this comparison: its MSE is lower at 15 of the 18 nodes and ranges from 0.0199 to 0.0289, compared with 0.0208 to 0.0584 for Aggregate. The narrower range therefore accompanies a reduction in error across most nodes, rather than merely a reduction in variability.

Neither interpolation experiment exhibits a classification deterioration as severe as that previously observed for Node 4 under Keep Local. Similarly, the regression errors do not reach the large values observed under Overwrite. These results are consistent with the motivation for interpolation as a compromise between preserving local parameters and incorporating received parameters, although they do not establish that the same behaviour will hold across all simulations.

Figure 6.7 provides an example of regression predictions for Node 8 with Interpolate and Substitution, whose common-test MSE is 0.0200. The rows correspond to the local training, validation, and test partitions and the common test set, while the columns represent the three regression outputs. The dashed diagonal indicates agreement between the reference and predicted values.

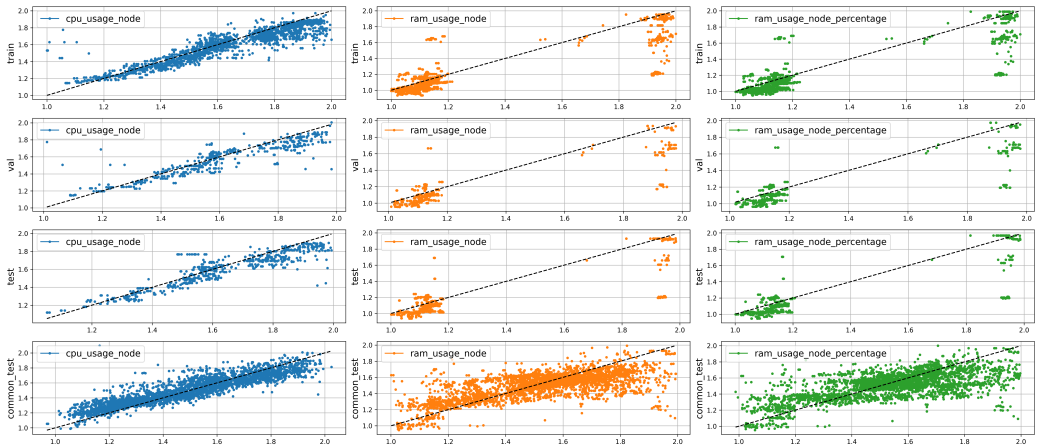


Figure 6.7. Regression predictions for Node 8 using Interpolate with Substitution. Rows show local training, validation, local test, and common-test results; columns show CPU usage, RAM usage, and RAM usage percentage. The dashed diagonal represents perfect agreement.

The CPU predictions broadly follow the diagonal across all partitions, including the common test set. The two RAM-related outputs display more distinct clusters on the local partitions and greater dispersion on the common test set. Their common-test predictions tend to overestimate some lower reference values and underestimate some higher values, indicating a tendency towards intermediate predictions. Thus, the relatively low overall MSE coexists with structured errors that remain visible when each output is inspected separately.

The training diagnostics provide a complementary perspective on classification performance. Figures 6.8 and 6.9 show the percentage of overloaded observations in the complete local-plus-synthetic training dataset for Nodes 1 and 0, respectively.

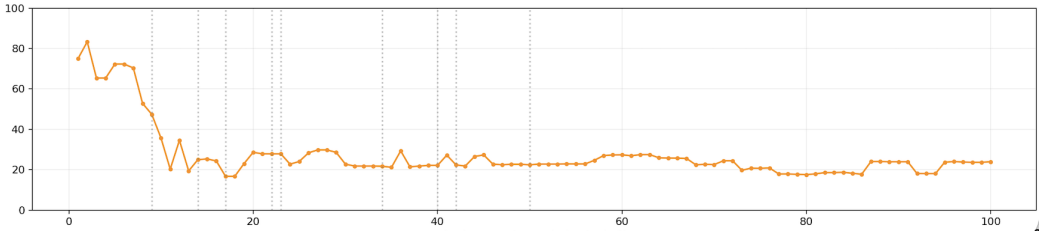


Figure 6.8. Percentage of overloaded observations in the local-plus-synthetic training dataset of Node 1 over gossip updates, using Interpolate with Substitution.

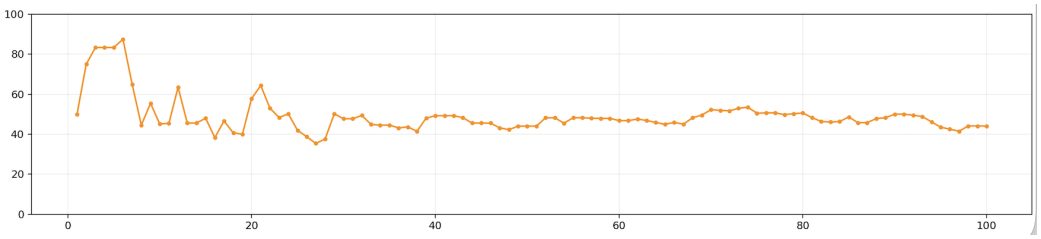


Figure 6.9. Percentage of overloaded observations in the local-plus-synthetic training dataset of Node 0 over gossip updates, using Interpolate with Substitution.

An initial concern was whether synthetic-data generation could alter the class balance established during local-data preparation. Synthetic observations reuse locally available workload inputs, associate them with remote node types, and obtain new targets from the corresponding received models. Consequently, balancing the original local labels does not guarantee that the generated targets, or the resulting local-plus-synthetic dataset, remain balanced. The diagnostics confirm that synthetic-data generation can substantially change the proportion of overloaded observations used during training.

After an initial transient, Node 1 trains on a dataset in which overloaded observations generally account for approximately 20–30% of the samples, with some intervals below 20%. Node 0 instead maintains a proportion closer to 50% during much of the experiment. Nevertheless, Node 1 achieves higher common-test accuracy and overloaded-class F1-score: 0.9029 and 0.6204, compared with 0.8487 and 0.5149 for Node 0. The improvement in overloaded-class F1-score is particularly relevant, because the comparison is not limited to overall accuracy, which can favour predictions of the majority class.

In this experiment, the imbalance introduced into the combined training dataset therefore does not prevent Node 1 from achieving good classification performance and outperforming a node with a more balanced training dataset. This does not establish that imbalance has little or no effect: the two nodes also differ in their local observations and in the remote information incorporated during training, so their comparison does not isolate the effect of class proportions. Rather, it shows that a proportion closer to 50% does not necessarily correspond to better common-test performance.

One possible interpretation is that the usefulness of synthetic observations depends on their coverage of relevant workloads and on the reliability of their targets, in addition to their class distribution. Moreover, interpolation transfers information through received model parameters as well as through synthetic observations. The plotted class proportions describe the composition of the training dataset but do not capture this additional contribution from parameter exchange.

Overall, these diagnostics confirm that the initial local class balance is not necessarily preserved after synthetic-data generation, while showing that the resulting imbalance can coexist with good common-test performance. Methods for managing class balance during synthetic-data generation remain a relevant direction for future work, particularly to investigate whether they can improve overload detection without reducing the useful information provided by remote node types.

6.3 Summary of Experimental Findings

The experiments presented in this chapter examined how different mechanisms for incorporating remote information affect predictive performance. The reference merging strategies benefited from additional gossip updates and larger numbers of received models, but overload detection remained limited. The proposed `node_type_merge` strategy extended local training with synthetic observations associated with remote node types, with its effectiveness depending on the rule used to update model parameters.

Among the configurations evaluated in the eighteen-node setting, interpolation with $\alpha = 0.3$ provided the best overall results. Table 6.15 compares the selected configuration, using interpolation with Substitution, against isolated local training and centralized training on the common test set. For local-only and Gossip Learning, the reported values are averaged across the eighteen node models.

Training approach	Accuracy	MSE
Local-only	0.5553	0.20081
Gossip Learning	0.8950	0.02346
Centralized	0.9666	0.00693

Table 6.15. Common-test performance of isolated local training, Gossip Learning with interpolation ($\alpha = 0.3$) and Substitution, and centralized training in the eighteen-node setting. Local-only and gossip results are means across node models. MSE is computed on scaled targets.

Compared with isolated local training, the selected Gossip Learning configuration increases mean common-test accuracy from 0.5553 to 0.8950 and reduces mean MSE from 0.20081 to 0.02346. These results show that the combined use of parameter interpolation and synthetic training observations substantially im-

proves prediction beyond each node's original data distribution in the evaluated setting.

As expected, centralized training achieves higher accuracy and lower regression error, benefiting from direct access to the combined training observations. The objective of the decentralized approach, however, is to improve predictive performance while retaining raw data at the participating nodes and avoiding a central training coordinator. The selected configuration provides a substantial improvement over isolated training under these constraints, although a performance gap relative to centralized training remains.

To conclude the analysis, Figures 6.10 and 6.11 compare the local-only baseline models with the models obtained using Interpolate with Substitution and $\alpha = 0.3$. This comparison illustrates how the selected approach extends classification performance beyond each node's original data distribution.

Each matrix evaluates all eighteen models on the original training datasets of all eighteen nodes. Rows identify the node whose model is evaluated, while columns identify the node whose original local training dataset is used for evaluation. The entry at row i and column j therefore represents the accuracy of model i on the training observations assigned to node j. The outlined diagonal contains evaluations on each model's own local training data, whereas off-diagonal entries describe performance on other nodes' datasets. Both figures use the same colour scale, with darker green indicating higher accuracy.

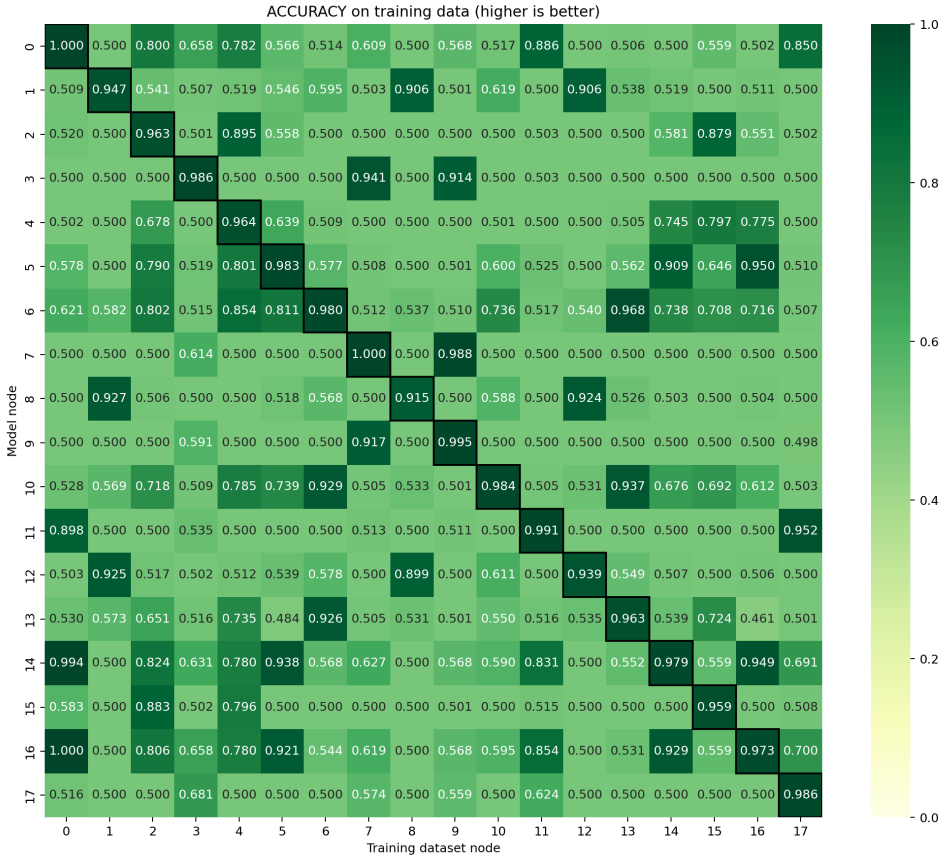


Figure 6.10. Accuracy of the local-only baseline models on the original training datasets of all eighteen nodes. Rows identify models and columns identify datasets. Outlined diagonal entries correspond to each model's own local training dataset.

The local-only matrix shows high accuracy along the diagonal and, for several models, on datasets belonging to other nodes of the same type. For example, Nodes 1, 8, and 12 all belong to TARGET_LIGHT. The local model of Node 1 achieves an accuracy of 0.947 on its own training dataset and 0.906 on the datasets of both Nodes 8 and 12. Similarly, the model of Node 3, belonging to SOURCE_HEAVY, achieves accuracies of 0.941 and 0.914 on the datasets of Nodes 7 and 9, which share its type.

This pattern is consistent with the data-allocation procedure: observations are randomly distributed among nodes of the same type, giving those nodes local datasets drawn from the same underlying type-specific distribution. Their datasets are therefore intended to be approximately IID within each type, while heterogeneity remains across types. A model trained locally can consequently perform well on another node's observations when that node shares the same type.

In contrast, many evaluations across different node types yield accuracies close to 0.5. The local-only models therefore show limited transfer across much of the network, despite fitting their own training data well.

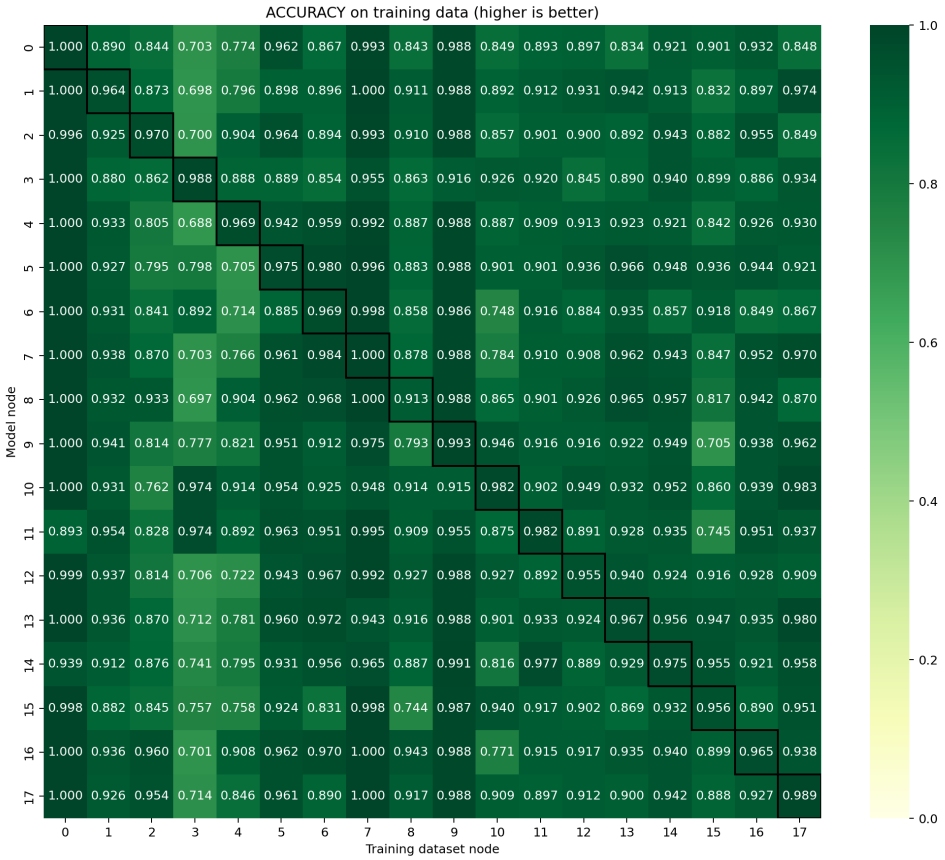


Figure 6.11. Accuracy of the models obtained using Interpolate with Substitution and $\alpha = 0.3$ on the same original training datasets used in Figure 6.10. Rows identify models and columns identify datasets.

The matrix obtained with gossip exhibits a markedly different pattern. High accuracy extends across most columns of each row, rather than being concentrated primarily on the model's own dataset and those of the same type. For example, the Node 8 model improves from 0.500 to 0.932 on the TARGET_HEAVY dataset of Node 5 and from 0.526 to 0.965 on the TARGET_MID dataset of Node 13.

These changes show that the models obtained with Interpolate and Substitution classify observations associated with other node types much more effectively than the local-only baselines. High accuracy on each node's own training dataset is broadly retained, although individual entries do not improve uniformly. Some differences between datasets also remain: for example, several models achieve lower accuracy on the dataset of Node 3 than on most other datasets. Gossip there-

fore broadens the range of distributions handled effectively without eliminating all variation in performance.

Taken together with the common-test results, the heatmaps support the conclusion that Interpolate with Substitution enables models to move beyond the type-specific behaviour observed with isolated local training. The evaluated configuration combines parameter exchange with synthetic observations associated with remote node types, allowing useful information to be incorporated without transferring raw training datasets between nodes.

7 Conclusion

This thesis investigated the decentralized training of a permutation-invariant multi-task model for resource usage prediction and overload classification in heterogeneous Edge FaaS environments. The adopted Gossip Learning procedure allows nodes to retain their observations locally while exchanging model parameters without a central training coordinator. The experimental analysis examined reference merging strategies and introduced a node-type-aware approach that complements parameter exchange with synthetic training observations associated with remote node types.

An application motivating this work is workload offloading among neighbouring Edge FaaS nodes. To decide whether and where to redirect requests, a node needs to estimate how potential destinations would respond to the additional workload, including their expected resource usage and risk of overload. This requires predictions that extend beyond the characteristics of the local node. Learning from other node types through gossip can therefore provide useful information for offloading decisions while retaining training observations locally.

The main finding is that the way received information is incorporated matters for both predictive tasks. Additional gossip updates and larger numbers of received models improved the reference configurations, but overload detection remained limited. Interpolate merging strategy provided the best overall balance among the configurations evaluated in the eighteen-node setting, improving mean predictive performance and reducing differences across nodes.

7.1 Answers to the Research Questions

7.1.1 RQ1: Effectiveness and Generalization of Decentralized Training

The first research question asked whether Gossip Learning can train the adopted multi-task model to generalize better than local-only training and achieve predictive performance comparable to centralized training.

The common-test comparison provides evidence of improved generalization with Interpolate and Substitution. Local-only training achieves a mean accuracy of 0.5553 and an MSE of 0.20081, whereas the selected gossip configuration achieves an accuracy of 0.8950 and an MSE of 0.02346. This corresponds to an increase of approximately 33.97 percentage points in accuracy and a reduction of approximately 88.3% in MSE. The improvement therefore concerns both overload classification and resource usage prediction.

Centralized training remains the strongest reference on the common test set, achieving an accuracy of 0.9666 and an MSE of 0.00693. This advantage is consistent with its direct access to the combined training observations from all participating nodes. The gossip configuration narrows the gap substantially relative to local-only training, but does not eliminate it: its accuracy remains approximately 7.16 percentage points below the centralized result, and its regression error is higher.

The answer to RQ1 is therefore that the proposed Gossip Learning configuration substantially improves generalization over isolated local training in the evaluated setting, although centralized training retains an advantage in predictive performance. A central contribution is that this improvement is obtained while keeping raw training observations on the participating nodes and avoiding a central training coordinator. Nodes exchange model parameters, and synthetic observations are generated locally using received models, without collecting the original datasets in a central repository. The objective is therefore not simply to approach centralized predictive performance, but to enable collaborative learning while maintaining local control over training data. The results demonstrate that substantial improvements over isolated training can be achieved under these constraints.

7.1.2 RQ2: Influence of Model-Merging Strategies

The second research question concerned how model-merging strategies affect regression and classification performance.

The results show that merging choices affect the two tasks differently. Among the reference strategies, Age-Weighted Averaging benefited from combining more received models, while Overwrite provided a competitive configuration when one received model was processed per update. Nevertheless, the reference experiments continued to exhibit limited overloaded-class F1-scores, motivating the introduction of `node_type_merge`.

Within the proposed approach, Keep Local generally preserved stronger regression performance but produced weaker and more variable classification results. Overwrite improved classification while increasing regression error. These findings show that preserving local parameters and replacing them with received parameters represent different compromises between retaining local knowledge and incorporating remote information.

Interpolation addressed this trade-off by combining the two parameter vectors before training on the local-plus-synthetic dataset. With $\alpha = 0.3$, the receiving node retained a contribution of 70% from its local parameters and incorporated 30% from the received model. For both Aggregate and Substitution, this configuration achieved higher mean accuracy and overloaded-class F1-score and lower mean MSE than the corresponding Keep Local and Overwrite configurations.

The answer to RQ2 is therefore that the merging rule is a central determinant of predictive performance, and its suitability must be assessed jointly for regression and classification. Among the proposed configurations evaluated in the eighteen-node setting, Interpolate provided the strongest overall results.

7.1.3 RQ3: Evaluation of Gossip Learning Configurations

The third research question concerned the comparison of selected Gossip Learning configurations to identify the most effective settings among those evaluated.

For the reference strategies, combining more received models improved mean accuracy and reduced mean MSE, with Age-Weighted Averaging achieving its best results with eight received models per update. Increasing the update count from 50 to 100 further reduced its MSE from 0.0576 to 0.0390 and increased overloaded-class F1-score from 0.2948 to 0.3770, although overload detection remained limited.

For the proposed strategy, Interpolate with $\alpha = 0.3$ provided the best overall results in the evaluated eighteen-node setting. Aggregate achieved the strongest mean classification performance, whereas Substitution produced lower and more consistent regression errors. Concatenation was excluded from the final compari-

son because its additional computational requirements were not accompanied by a consistent predictive advantage.

The experiments therefore identified interpolation as the preferred parameter-update rule, with Aggregate and Substitution offering different advantages depending on the prediction task prioritised. This conclusion applies to the configurations tested and does not establish a globally optimal parameter setting.

7.2 Limitations

The findings must be interpreted within the controlled experimental setting. Communication was simulated over fully connected networks containing nine or eighteen nodes. The study therefore does not establish how the proposed strategy behaves in larger or partially connected networks, where information may propagate more slowly and some node types may be less accessible.

The datasets were collected before training and remained fixed during each experiment. The evaluation consequently does not cover evolving workloads, concept drift, or the arrival of previously unseen node types. Observations were randomly distributed among nodes of the same type, producing an approximately IID setting within each type while preserving heterogeneity across types. Real nodes of the same type may exhibit stronger differences in workload composition and class distribution.

Synthetic-data generation can alter the class balance established during local-data preparation. The diagnostics showed that an imbalanced local-plus-synthetic dataset can coexist with good classification performance, but did not isolate the effect of imbalance from other differences between nodes. These observations therefore do not establish that class balance is irrelevant.

Finally, predictive performance still leaves room for improvement in both tasks. Although interpolation improves classification accuracy, the mean overloaded-class F1-score remains between 0.6147 and 0.6330, indicating that overload detection is not yet fully reliable. Regression errors also remain. The proposed approach therefore improves prediction across node types while retaining data locally, but further refinement can be done to reduce classification and regression errors.

7.3 Future Work

A first direction is to investigate class-balance management during synthetic-data generation. Although the original local training partitions are balanced, the predicted labels associated with remote node types can produce different overload proportions in the combined dataset. Possible approaches include weighting classification examples, sampling synthetic observations according to class and node type, or controlling the contribution of each synthetic block.

A second direction is to investigate whether differences between node types can be inferred directly from the parameters of exchanged models, without transmitting explicit node-type identifiers. Future approaches could analyse parameter patterns or similarities to identify models associated with different execution environments and use this information to guide parameter updates and synthetic-data generation. The objective would be to adapt the learning procedure to heterogeneous nodes without relying on predefined and explicitly shared node-type labels.

Extending the evaluation to larger and partially connected networks would clarify how communication structure affects the availability of knowledge from different node types. Experiments with delayed messages, intermittent participation, and changing workloads would further test the approach under conditions closer to operational systems.

A final direction is to integrate the predictive models into a workload-offloading policy. Given information about neighbouring nodes' current workloads, node types, and availability, a node could estimate the resource usage and overload risk associated with redirecting additional requests to each candidate destination. These predictions could support the selection of an offloading destination, together with communication costs and service requirements. Evaluating such a policy would determine whether the improvements in predictive performance translate into lower response times, fewer overload conditions, and better resource utilisation. This operational benefit was not measured in the present experiments.

This thesis identifies node-type-aware synthetic-data generation combined with parameter interpolation as an effective approach to decentralized multi-task performance prediction within the evaluated setting. Interpolation achieved the best overall results among the proposed configurations tested in the eighteen-node network. The main contribution is the improvement in prediction beyond local observations while retaining raw training data at the participating nodes and operating without a central training coordinator. This allows nodes to benefit from knowledge learned elsewhere in the network without pooling their original datasets. The resulting predictive capability could support workload-offloading decisions by helping nodes assess potential execution destinations. Integrating

these predictions into an operational offloading policy and measuring the resulting system-level benefits remain directions for future work.

Bibliography

[1] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.

[2] C. Denninnart, T. Chanikaphon, and M. A. Salehi, "Efficiency in the serverless cloud paradigm: A survey on the reusing and approximation aspects," *Software: Practice and Experience*, vol. 53, no. 10, pp. 1853–1886, 2023.

[3] M. Ciavotta, D. Motterlini, M. Savi, and A. Tundo, "Dfaas: Decentralized function-as-a-service for federated edge computing," in *2021 IEEE 10th International Conference on Cloud Networking (CloudNet)*. IEEE, 2021, pp. 1–4.

[4] F. Filippini, M. Savi, and M. Ciavotta, "Performance modeling in FaaS workloads at the edge using machine learning," *Cluster Computing*, vol. 29, no. 7, p. 430, 2026. [Online]. Available: https://doi.org/10.1007/s10586-026-06233-3

[5] M. N. Khan, "Domain adaptation of performance modeling in edge FaaS systems," Master's thesis in Data Science, Università degli Studi di Milano-Bicocca, 2025, academic year 2024–2025.

[6] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[7] A. Tundo, F. Filippini, F. Regonesi, M. Ciavotta, and M. Savi, "Decentralized edge workload forecasting with gossip learning," *IEEE Transactions on Network and Service Management*, May 2025, manuscript received May 13, 2025.

[8] I. Baldini, P. Castro, K. Chang, P. Cheng, S. Fink, V. Ishakian, N. Mitchell, V. Muthusamy, R. Rabbah, A. Slominski, and P. Suter, "Serverless computing: Current trends and open problems," in *Research Advances in Cloud Computing*, S. Chaudhary, G. Somani, and R. Buyya, Eds. Singapore: Springer, 2017, pp. 1–20.

[9] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, J. E. Gonzalez, R. A. Popa, I. Stoica, and D. A. Patterson, "Cloud programming simplified: A berkeley view on serverless computing," *arXiv preprint arXiv:1902.03383*, 2019. [Online]. Available: https://arxiv.org/abs/1902.03383

[10] C. Calavaro, V. Cardellini, F. L. Presti, and G. R. Russo, "Beyond cloud: Serverless functions in the compute continuum," *SN Computer Science*, vol. 6, p. 194, 2025.

[11] T. Pfandzelter and D. Bermbach, "tinyfaas: A lightweight faas platform for edge environments," in *2020 IEEE International Conference on Fog Computing (ICFC)*. IEEE, 2020, pp. 17–24.

[12] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021. [Online]. Available: https://otexts.com/fpp3/

[13] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016. [Online]. Available: https://www.deeplearningbook.org/

[14] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://www.bioinf.jku.at/publications/older/2604.pdf

[15] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder–decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Doha, Qatar: Association for Computational Linguistics, 2014, pp. 1724–1734. [Online]. Available: https://aclanthology.org/D14-1179/

[16] R. Caruana, "Multitask learning," *Machine Learning*, vol. 28, no. 1, pp. 41–75, 1997.

[17] S. Ruder, "An overview of multi-task learning in deep neural networks," *arXiv preprint arXiv:1706.05098*, 2017. [Online]. Available: https://arxiv.org/abs/1706.05098

[18] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, and A. J. Smola, "Deep sets," in *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/f22e4747da1aa27e363d86d40ff442fe-Abstract.html

[19] R. L. Murphy, B. Srinivasan, V. Rao, and B. Ribeiro, "Janossy pooling: Learning deep permutation-invariant functions for variable-size inputs," in *International Conference on Learning Representations (ICLR)*, 2019. [Online]. Available: https://openreview.net/forum?id=BJluy2RcFm

[20] J. Verbraeken, M. Wolting, J. Katzy, J. Kloppenburg, T. Verbelen, and J. S. Rellermeyer, "A survey on distributed machine learning," *ACM Computing Surveys*, vol. 53, no. 2, pp. 1–33, 2020.

[21] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, ser. Proceedings of Machine Learning Research, vol. 54. PMLR, 2017, pp. 1273–1282. [Online]. Available: https://proceedings.mlr.press/v54/mcmahan17a.html

[22] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, "Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent," in *Advances in Neural Information Processing Systems*, 2017.

[23] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, "Scaling distributed machine learning with the parameter server," in *11th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 2014, pp. 583–598. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/li_mu

[24] R. Ormándi, I. Hegedűs, and M. Jelasity, "Gossip learning with linear models on fully distributed data," *Concurrency and Computation: Practice and Experience*, vol. 25, no. 4, pp. 556–571, 2013.

[25] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., vol. 2, 2020, pp. 429–450. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2020/file/1f5fe83998a09396ebe6477d9475ba0c-Paper.pdf

[26] R. Ma, Y. Zhan, T. Yan, Y. Xia, and Y. Ali, "INTERLESS: Interference-aware deep resource prediction for serverless computing," in *2024 36th Chinese Control and Decision Conference (CCDC)*. IEEE, 2024, pp. 3783–3788. [Online]. Available: https://doi.org/10.1109/CCDC62350.2024.10588201

[27] F. Filippini, L. Cavenaghi, N. Calmi, M. Savi, and M. Ciavotta, "ML-based performance modeling in edge FaaS systems," in *Service-Oriented and Cloud Computing*, ser. Lecture Notes in Computer Science, C. Pahl, A. Janes, T. Cerny, V. Lenarduzzi, and M. Esposito, Eds. Cham: Springer Nature Switzerland, 2025, vol. 15547, pp. 112–127. [Online]. Available: https://doi.org/10.1007/978-3-031-84617-5_10

[28] X. Lan, J. Taghia, F. Moradi, M. A. Khoshkholghi, E. Listo Zec, O. Mogren, T. Mahmoodi, and A. Johnsson, "Federated learning for performance prediction in multi-operator environments," *ITU Journal on Future and Evolving Technologies*, vol. 4, no. 1, pp. 166–177, 2023. [Online]. Available: https://doi.org/10.52953/PFYZ9165

[29] I. Hegedűs, G. Danner, and M. Jelasity, "Gossip learning as a decentralized alternative to federated learning," in *Distributed Applications and Interoperable Systems*, ser. Lecture Notes in Computer Science, vol. 11534. Springer, 2019.

[30] T. Howard-Grubb, "Domain adaptation of performance modeling in edge faas systems," Master's thesis, Università degli Studi di Milano-Bicocca, Milan, Italy, 2025, department of Informatics, Systems and Communication, Academic Year 2024–2025.